

Algorithmen und Datenstrukturen

§4 Suchbäume, et al.

Prof. Dr. Georg Umlauf

Inhalt der Vorlesung

1. **Problemstellung und Motivation**
2. Binäre Bäume
3. Binäre Suchbäume
4. AVL-Bäume
5. Rot-Schwarz-Bäume
6. Heaps
7. B-Bäume*

* nicht prüfungsrelevant

§4.1 Problemstellung

Motivation

Gegeben: Menge von Datensätzen bestehend aus

- einem Suchschlüssel (**search key**) und
- den Nutzdaten (**value**).

Der Suchschlüssel muss nicht eindeutig sein.

Beispiel Telefonbuch:

(**key** ist nicht eindeutig)

Maier	Hauptstr. 1	14234
Baier	Bahnhofstr. 5	24829
Müller	Uferweg 5	53289
Anton	Mozartstr. 2	36478

Datensatz

Suchschlüssel

Nutzdaten

Beispiel Wörterbuch:

(**key** ist eindeutig)

sehen	see; look
sprechen	speak; talk
gehen	go; walk
hören	hear; listen

Hash-Verfahren

Vorteil: Suchen, Einfügen und Löschen von Daten in konstanter Laufzeit.

Nachteil: Operationen, für die die Reihenfolge der Daten wichtig ist, werden nicht unterstützt.

- Beispiele:
 - Finde Minimum/Maximum,
 - Analyse/Ausgabe der Daten in der korrekten Reihenfolge.

► Binäre Suchbäume erlauben

- sortierte Bearbeitung in $O(n \cdot \log n)$,
- Suchen, Einfügen, Finden von Maxima/Minima und Löschen in durchschnittlich $O(\log n)$.

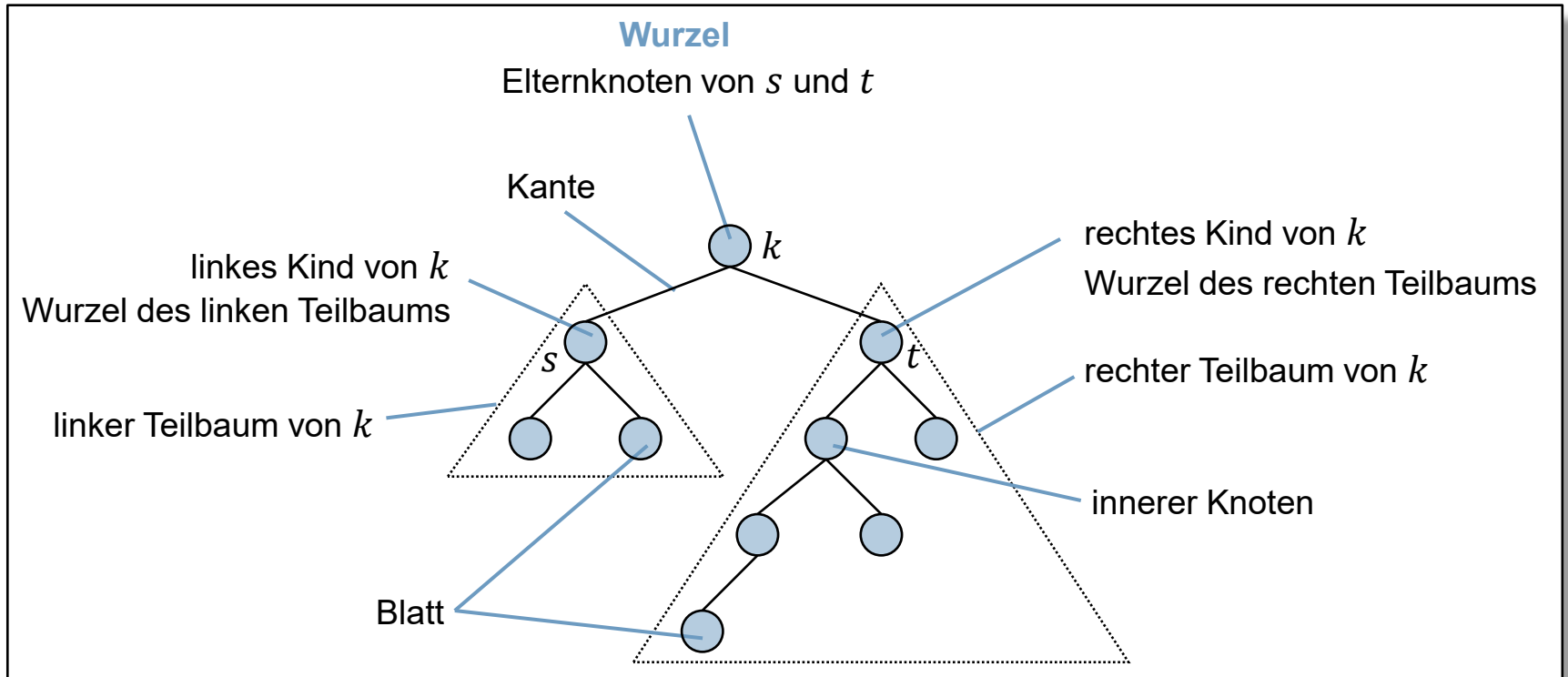
Inhalt der Vorlesung

1. Problemstellung und Motivation
2. **Binäre Bäume**
3. Binäre Suchbäume
4. AVL-Bäume
5. Rot-Schwarz-Bäume
6. Heaps
7. B-Bäume*

* nicht prüfungsrelevant

§4.2 Binäre Bäume

Bäume



Baum: Jeder Knoten kann beliebig viele Kinder haben.

Binärbaum: Jeder Knoten hat **zwei Kinder** (linkes und rechtes Kind),
oder **ein Kind** (linkes oder rechtes Kind) oder **keine Kinder**.

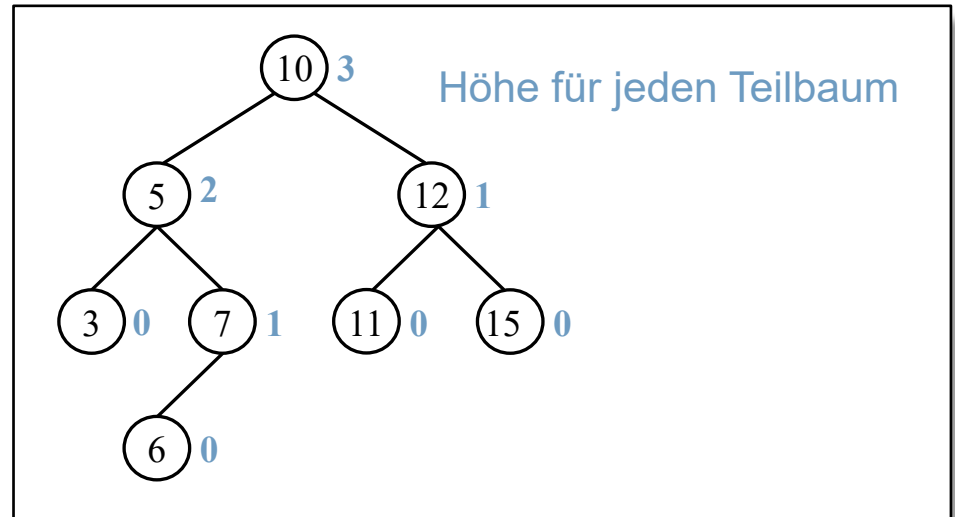
§4.2 Binäre Bäume

Bäume

Höhe eines Baumes

Maximale Anzahl von Kanten von der Wurzel zu einem Blatt.

Beispiel:



Bemerkungen:

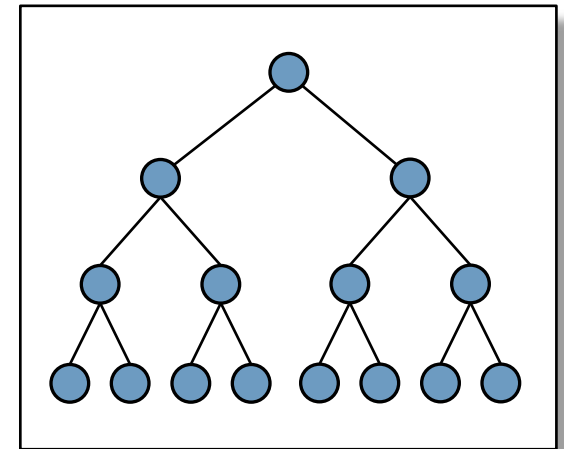
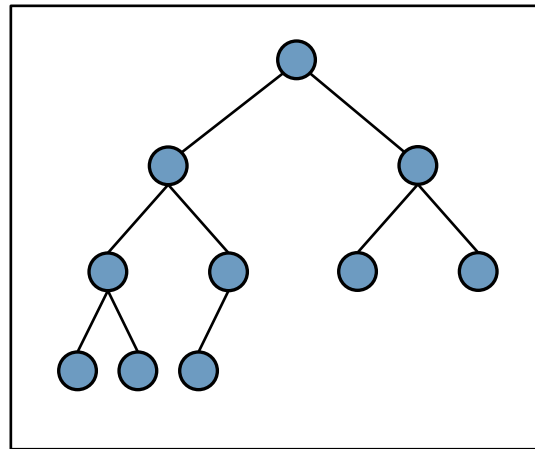
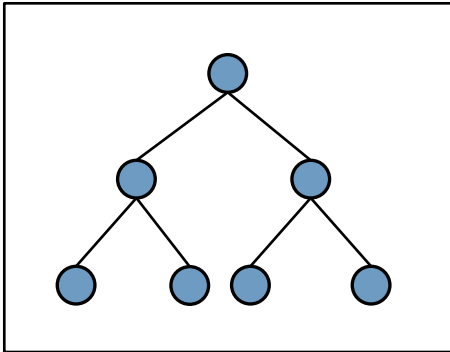
- Ein Baum, mit nur einem Knoten, hat die Höhe 0.
- Aus technischen Gründen wird die Höhe eines leeren Baums (d.h. Anzahl Knoten = 0) als -1 definiert.

§4.2 Binäre Bäume

Vollständiger binärer Baum

Bei einem vollständiger Binärbaum ist jede Ebene (bis auf die letzte) vollständig und die letzte Ebene von links nach rechts gefüllt.

Beispiele:



§4.2 Binäre Bäume

Eigenschaften vollständiger binärer Bäume:

- Ein vollständiger binärer Baum mit n Knoten hat die Höhe $h = \lfloor \log_2 n \rfloor$.
- Vollständige binäre Bäume sind (bei einer gegebenen Knotenzahl) binäre Bäume mit einer minimalen Höhe.
- Ein vollständiger binärer Baum belegt $O(n)$ Speicher.

§4.2 Binäre Bäume

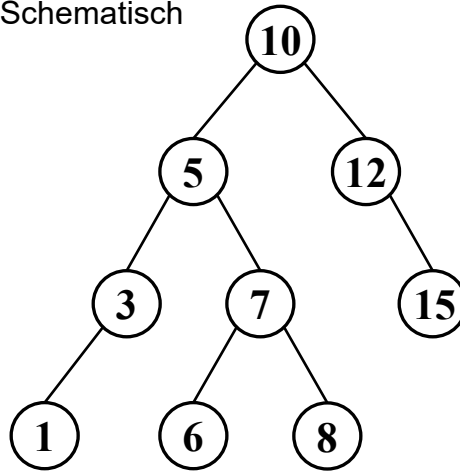
Implementierung von binären Bäumen

Jeder Knoten hat jeweils einen Zeiger auf das linke bzw. rechte Kind.

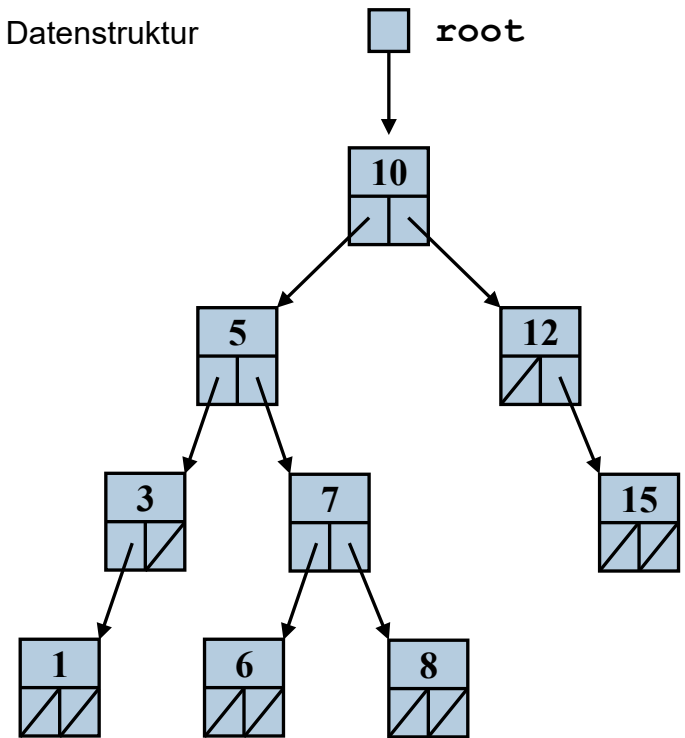
```
struct Node
{
    SomeType data;
    Node* left;
    Node* right;
};

class Tree
{
private:
    Node* root;
...
};
```

Schematisch



Datenstruktur



§4.2 Binäre Bäume

Traversieren

Das Besuchen aller Knoten in einer bestimmten Reihenfolge ist eine oft benötigte Operation.

Durchlaufreihenfolgen

- Pre-Order-Reihenfolge: Wurzel, linker Teilbaum, rechter Teilbaum.
- Post-Order-Reihenfolge: linker Teilbaum, rechter Teilbaum, Wurzel.
- In-Order-Reihenfolge: linker Teilbaum, Wurzel, rechter Teilbaum.
- Level-Order-Reihenfolge: besuche Knoten ebenenweise.

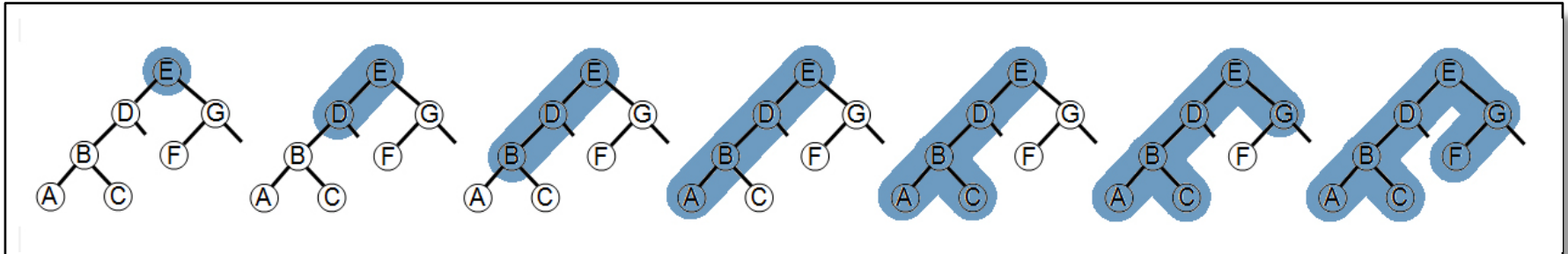
Bemerkungen

- Ein In-Order-Durchlauf ist nur für binäre Bäume sinnvoll.

§4.2 Binäre Bäume

Pre-Order-Traversierung

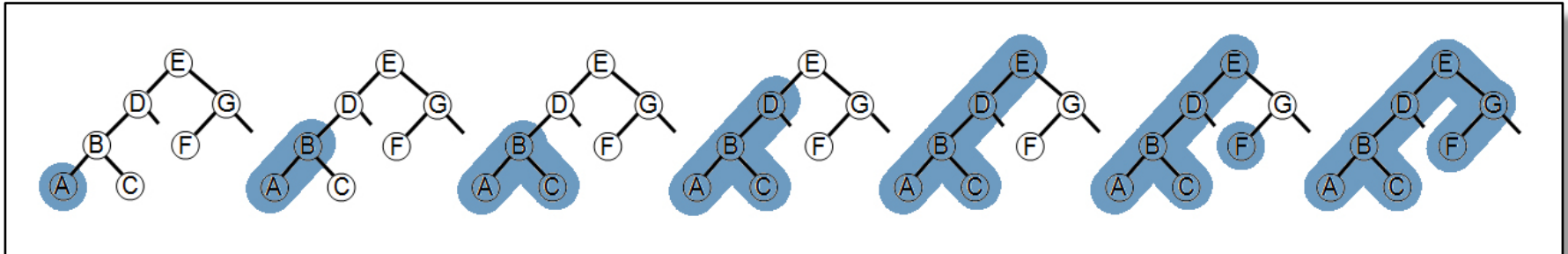
```
void PreOrder (Node* p)
{
    if ( p!= 0 ) {
        bearbeite(p->data) ;
        PreOrder (p->left) ;
        PreOrder (p->right) ;
    }
}
```



§4.2 Binäre Bäume

In-Order-Traversierung

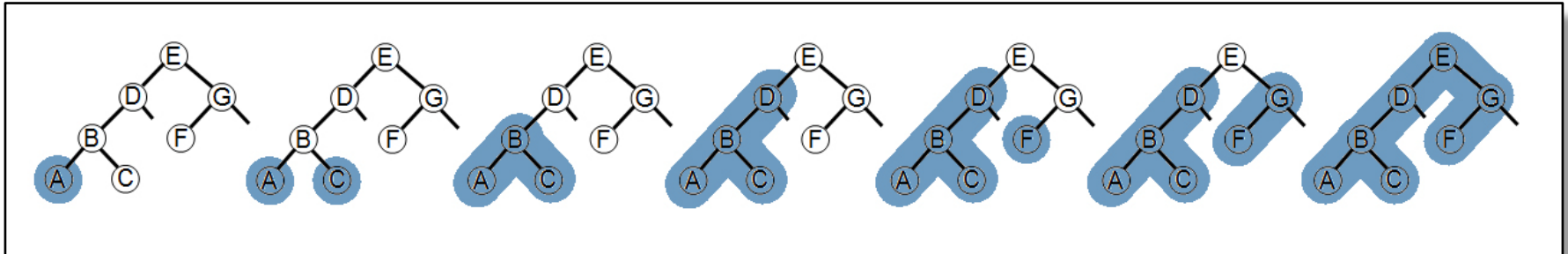
```
void InOrder (Node* p)
{
    if ( p!= 0 ) {
        InOrder (p->left) ;
        bearbeite (p->data) ;
        InOrder (p->right) ;
    }
}
```



§4.2 Binäre Bäume

Post-Order-Traversierung

```
void PostOrder(Node* p)
{
    if ( p != 0 ) {
        PostOrder(p->left);
        PostOrder(p->right);
        bearbeite(p->data);
    }
}
```



Inhalt der Vorlesung

1. Problemstellung und Motivation
2. Binäre Bäume
3. **Binäre Suchbäume**
4. AVL-Bäume
5. Rot-Schwarz-Bäume
6. Heaps
7. B-Bäume*

* nicht prüfungsrelevant

§4.3 Binäre Suchbäume

Voraussetzung

Alle Knoten in einem Baum haben einen Schlüssel (**key**) und Nutzdaten (**value**).

```
struct Node
{
    KeyType    key;
    ValueType  val;
    Node*      left;
    Node*      right;
};
```

Definition

Ein **binärer Suchbaum** ist ein binärer Baum, bei dem für alle Knoten **k** die folgenden Eigenschaften gelten:

1. Alle Schlüssel im linken Teilbaum sind kleiner als der Schlüssel im Knoten **k**.
2. Alle Schlüssel im rechten Teilbaum sind größer als der Schlüssel im Knoten **k**.

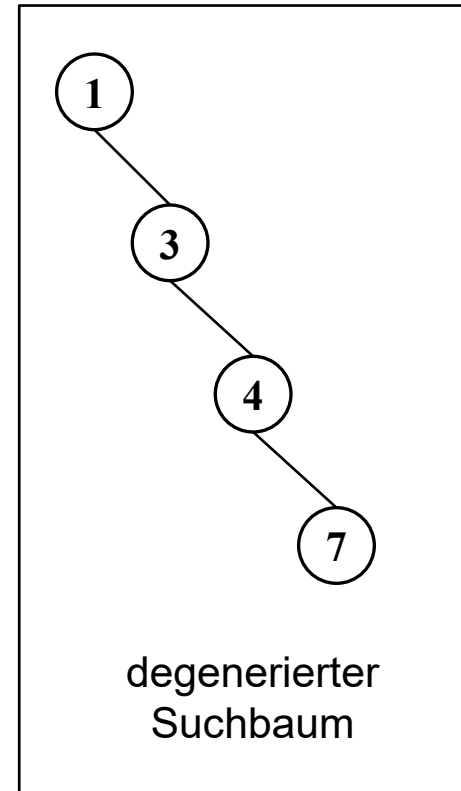
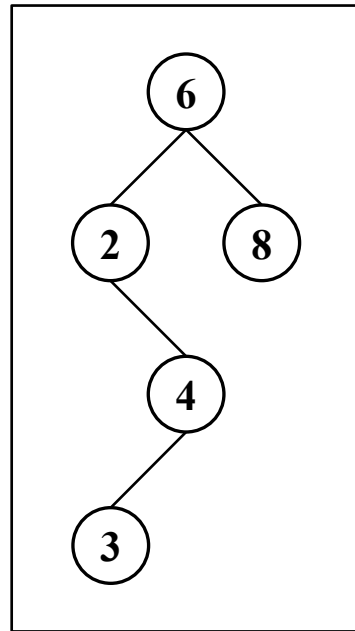
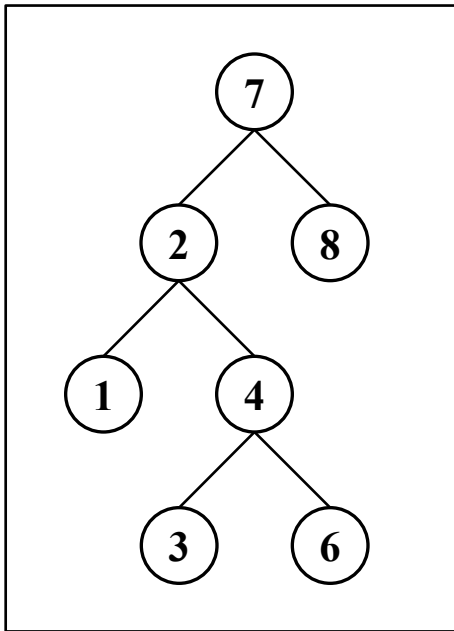
D.h.

$$k.\text{left} \rightarrow \text{key} < k.\text{key} < k.\text{right} \rightarrow \text{key}$$

§4.3 Binäre Suchbäume

Beispiele:

(In den Bäumen sind nur die Schlüssel dargestellt.)



§4.3 Binäre Suchbäume

Algorithmen

Rekursive Suche

- Suche nach einem Knoten mit Schlüssel **key** im Teilbaum **p**.
- **Falls gefunden**, wird der im Knoten abgespeicherte Datenwert **v** und der Rückgabewert **true** zurückgeliefert.
- **Falls nicht gefunden**, wird der Rückgabewert **false** zurückgeliefert.

```
bool search(KeyType key, ValueType& v, const Node* p)
{
    if (p == 0)                return false;
    else if ( key < p->key ) return search(key,v,p->left) ;
    else if ( key > p->key ) return search(key,v,p->right) ;
    else {                     // key gefunden
        v = p->val;
        return true;
    }
}
```

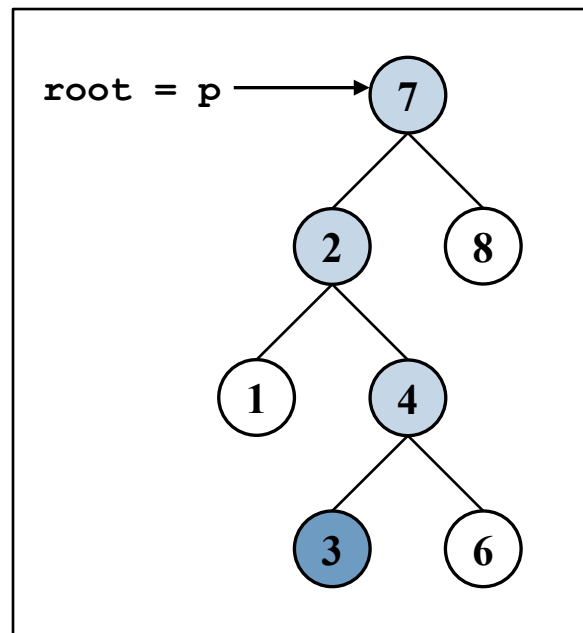
§4.3 Binäre Suchbäume

Algorithmen

Beispiel:

(In dem Baum sind nur die Schlüssel dargestellt.)

```
search(3, v, root);
```



Rekursives Einfügen

- Um einen neuen Knoten mit Schlüssel **key** und Nutzdaten **v** in einen Teilbaum **p** einzufügen, muss zuerst nach dem Schlüssel **key** im Teilbaum gesucht werden.
- **Falls gefunden,** wird kein neuer Knoten eingefügt.
- **Falls nicht gefunden,** endet die Suche erfolglos bei einem 0-Zeiger.
 - An dieser Stelle wird dann ein neuer Knoten mit Schlüssel **key** eingefügt.

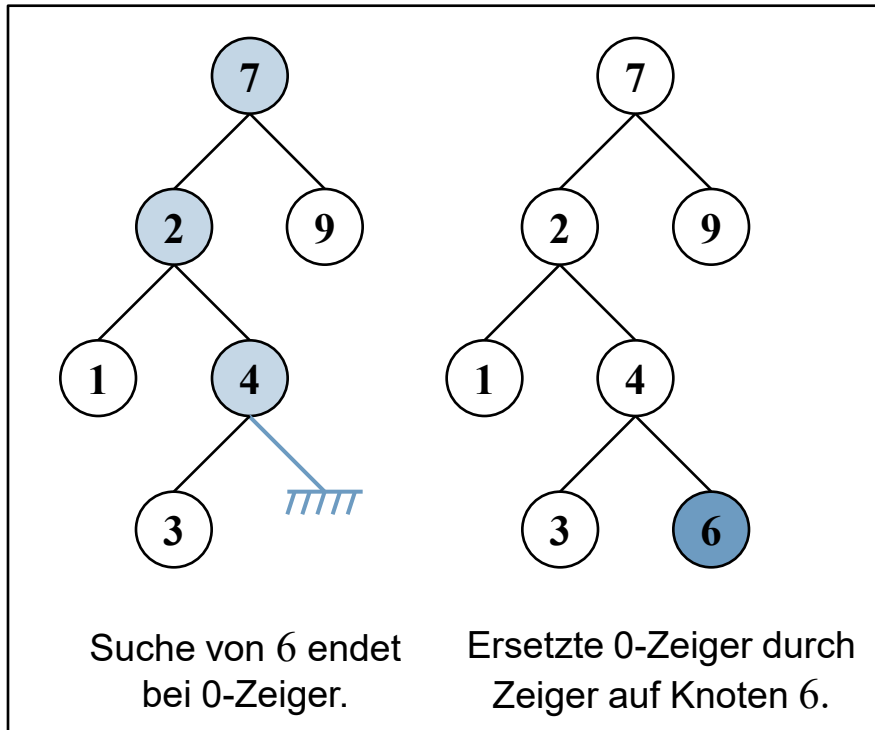
§4.3 Binäre Suchbäume

Algorithmen

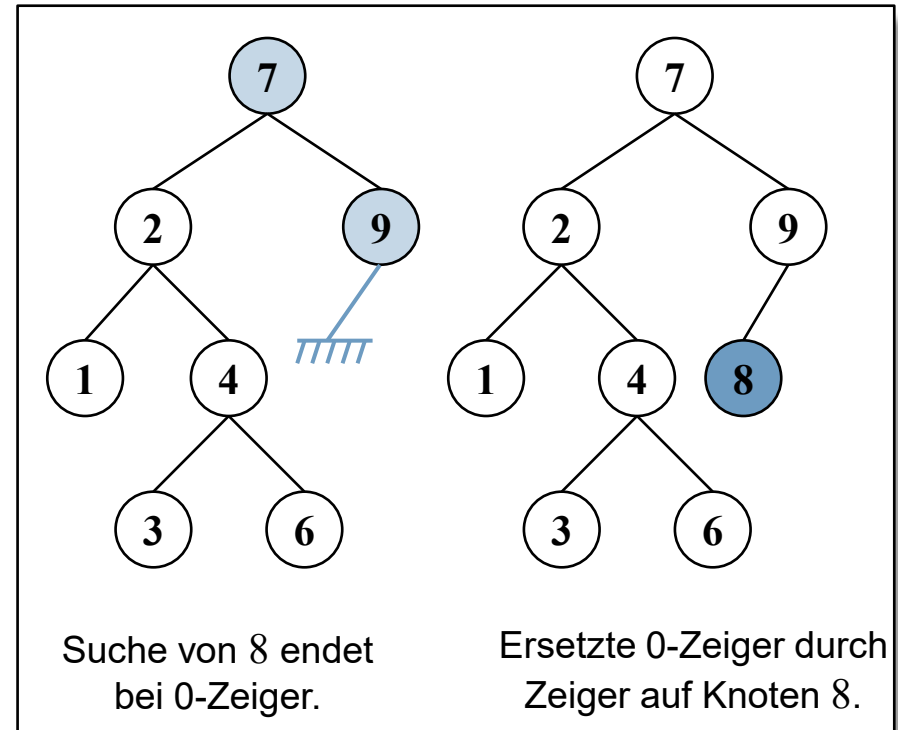
Beispiele:

(In den Bäumen sind nur die Schlüssel dargestellt.)

`insert(6, v, root);`



`insert(8, v, root);`



§4.3 Binäre Suchbäume

Algorithmen

Bemerkung:

- Der Parameter **p** ist ein **Ein- und Ausgabeparameter** und daher als Referenzparameter realisiert.
 - Der Teilbaum wird gelesen und geändert.

```
bool insert(KeyType key, ValueType& v, Node*& p)
{
    if (p == 0) {                                // key nicht vorhanden
        p      = new Node;
        p->key  = key;
        p->val  = v;
        return true;
    }
    else if ( key < p->key ) return insert(key,v,p->left);
    else if ( key > p->key ) return insert(key,v,p->right);
    else                                     return false; // key vorhanden
}
```

Rekursives Löschen

Um einen Knoten mit Schlüssel **key** zu löschen, wird zunächst nach dem Schlüssel **key** gesucht.

Es sind dann vier Fälle zu unterscheiden:

1. „**Knoten nicht vorhanden**“: Der Schlüssel **key** kommt nicht vor, d.h. es ist nichts zu tun.
2. „**Knoten hat keine Kinder**“: Der Schlüssel kommt in einem Blatt vor, d.h. der Knoten kann einfach entfernt werden.
3. „**Knoten hat nur ein Kind**“: Der Knoten mit dem gefundenen Schlüssel hat genau ein Kind, siehe nächste Seite...
4. „**Knoten hat zwei Kinder**“: Der Knoten mit dem gefundenen Schlüssel hat zwei Kinder, siehe übernächste Seite...

§4.3 Binäre Suchbäume

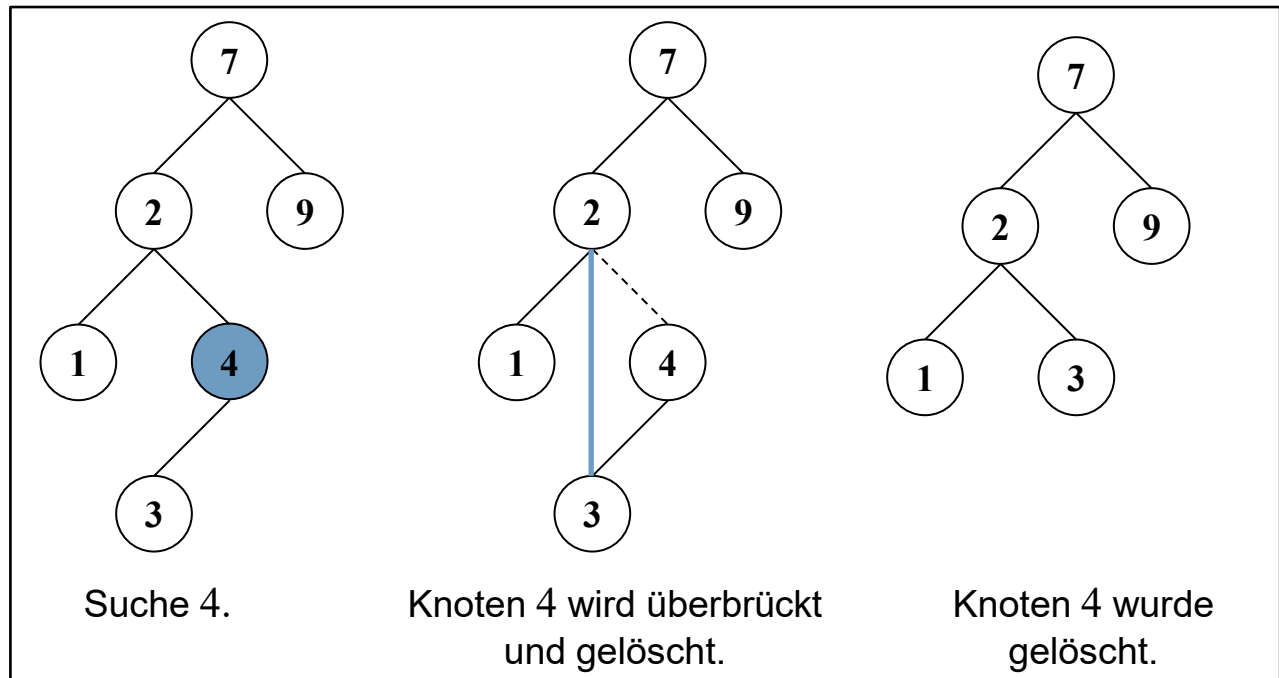
Algorithmen

Fall 3: „Knoten hat nur ein Kind“

- a. Überbrücke den Knoten **key**, indem der Elternknoten von **key** auf das Kind von **key** verzeigert wird (by-pass), und
- b. lösche den Knoten **key**.

Beispiel:

```
remove(4, root);
```



§4.3 Binäre Suchbäume

Algorithmen

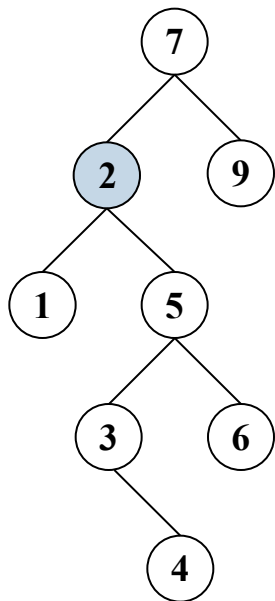
Fall 4: „Knoten hat zwei Kinder“

- a. Ersetze den Knoten **key** durch den Knoten mit dem kleinsten Schlüssel **key_{min}** im rechten Teilbaum von **key**.
- b. Lösche den Knoten **key_{min}**.
 - Da der kleinste Knoten **key_{min}** kein linkes Kind hat, kann das Löschen von **key_{min}** wie im Fall „Knoten hat nur ein Kind“ bzw. „Knoten hat keine Kinder“ behandelt werden.

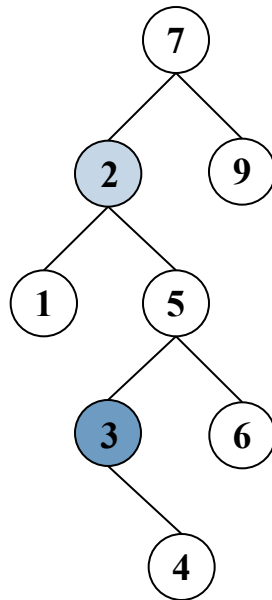
§4.3 Binäre Suchbäume

Algorithmen

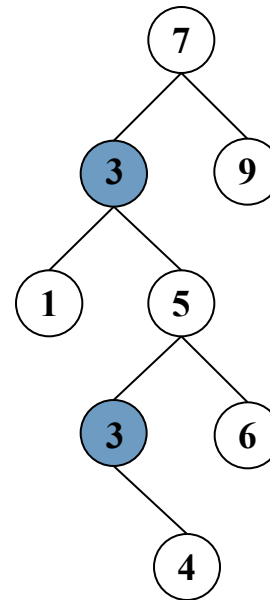
Beispiel: `remove(2, root);`



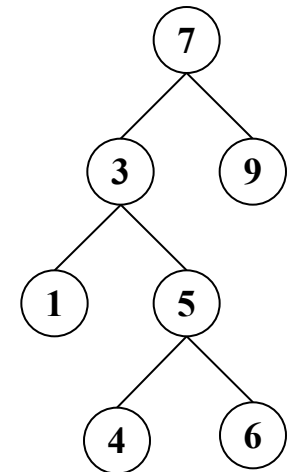
Suche 2.



Suche Knoten mit kleinstem Schlüssel im rechten Teilbaum von 2.



Knoten 2 wurde durch kleinsten Knoten ersetzt.

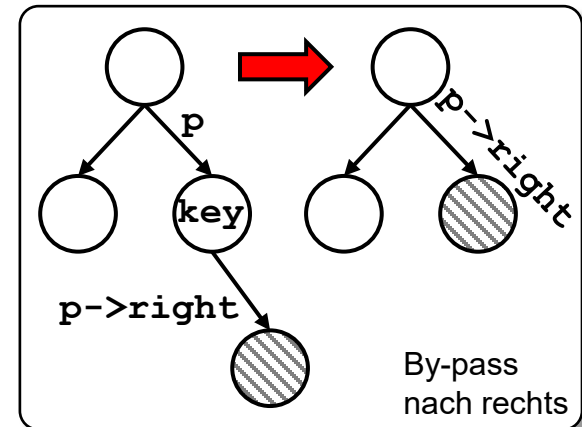
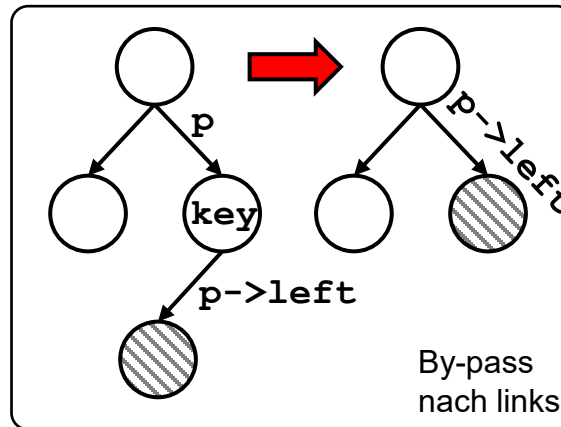
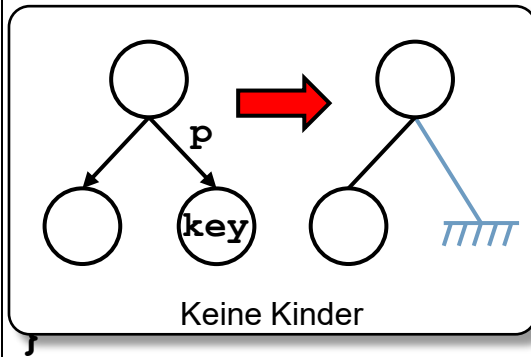


Kleinsten Knoten wurde gelöscht.

§4.3 Binäre Suchbäume

Algorithmen

```
bool remove(KeyType key, Node*& p)
{
    if (p == 0) return false;           // key nicht gefunden
    else if ( key < p->key ) return remove(key,p->left);
    else if ( key > p->key ) return remove(key,p->right);
    else if ( p->left == 0 || p->right == 0 ) { // 0&1 Kind
        Node* tmp = p;
        if ( p->left != 0 ) p = p->left; // by-pass nach links
        else                p = p->right; // by-pass nach rechts
        delete tmp;
        return true;
    }
}
```



§4.3 Binäre Suchbäume

Algorithmen

```
bool remove(KeyType key, Node*& p)
{
    if (p == 0) return false;           // key nicht gefunden
    else if ( key < p->key ) return remove(key,p->left);
    else if ( key > p->key ) return remove(key,p->right);
    else if ( p->left == 0 || p->right == 0 ) { // 0&1 Kind
        Node* tmp = p;
        if ( p->left!=0 ) p = p->left; // by-pass nach links
        else                p = p->right; // by-pass nach rechts
        delete tmp;
        return true;
    }
    else {                               // 2 Kinder
        Node* min = searchMin(p->right);
        p->val      = min->val;
        p->key      = min->key;
        return remove(min->key,p->right); // min hat 0/1 Kind
    }
}
```


§4.3 Binäre Suchbäume

Algorithmen

Operation searchMin

- Suche im Teilbaum **p** nach dem Knoten mit dem kleinsten Schlüssel und liefere Zeiger auf diesen zurück.
 - Suche den am weitesten links stehenden Knoten im Teilbaum.

```
Node* searchMin(const Node* p)
{
    if ( p->left == 0 ) return p;          // Minimum gefunden
    else return searchMin(p->left);
}
```

Bemerkung

- In der Operation **remove** im Fall „**Knoten hat zwei Kinder**“ wird beim Aufruf von **searchMin** und von **remove** jeweils vom rechten Kind **p->right** zum kleinsten Knoten gelaufen.
 - ▶ Es wird zweimal vom rechten Kind **p->right** zum kleinsten Knoten gelaufen.
- ▶ Dies lässt sich beheben, indem **searchMin** noch zusätzlich das Löschen des kleinsten Elements übernimmt.
 - Der rekursive Aufruf von **remove** ist dann überflüssig.

Worst case

- Im schlechtesten Fall degeneriert ein binärer Suchbaum mit n Knoten zu einer Liste der Höhe $n - 1$,
 - z.B. wenn die Daten vorsortiert mit `insert(...)` eingefügt werden.
- ▶ Die Operationen zum Suchen, Einfügen und Löschen von einem Datensatz haben eine maximale Suchlänge von n .
- ▶ **Laufzeit** für Suchen/Einfügen/Löschen von einem Datensatz: $O(n)$.

§4.3 Binäre Suchbäume

Laufzeit

Average case (Ottmann/Widmayer)

- Bäume mit n Knoten entstehen durch eine Folge von **insert**-Operationen von n unterschiedlichen Elementen.
 1. Annahme: Jede der $n!$ Anordnungen der n Elemente sind gleich wahrscheinlich.
 - ▶ Es gibt $n!$ Bäume, über die die Suchlänge gemittelt wird.
 - ▶ Mittlere Laufzeit für das **Einfügen** eines Elements: $O(\log_2 n)$.
 - ▶ Für die **Konstruktion** eines binären Suchbaums durch eine Folge von **insert**-Operationen von n unterschiedlichen Elementen ergibt sich eine mittlere Laufzeit von $O(n \log n)$.
 2. Annahme: Alle strukturell verschiedenen binären Suchbäume mit n Elementen sind gleich wahrscheinlich.
 - ▶ Die mittlere Suchlänge ist dann etwa $\sqrt{\pi n}$.
 - ▶ Mittlere Laufzeit für das **Einfügen** eines Elements: $O(\sqrt{n})$.

§4.3 Binäre Suchbäume

Probleme

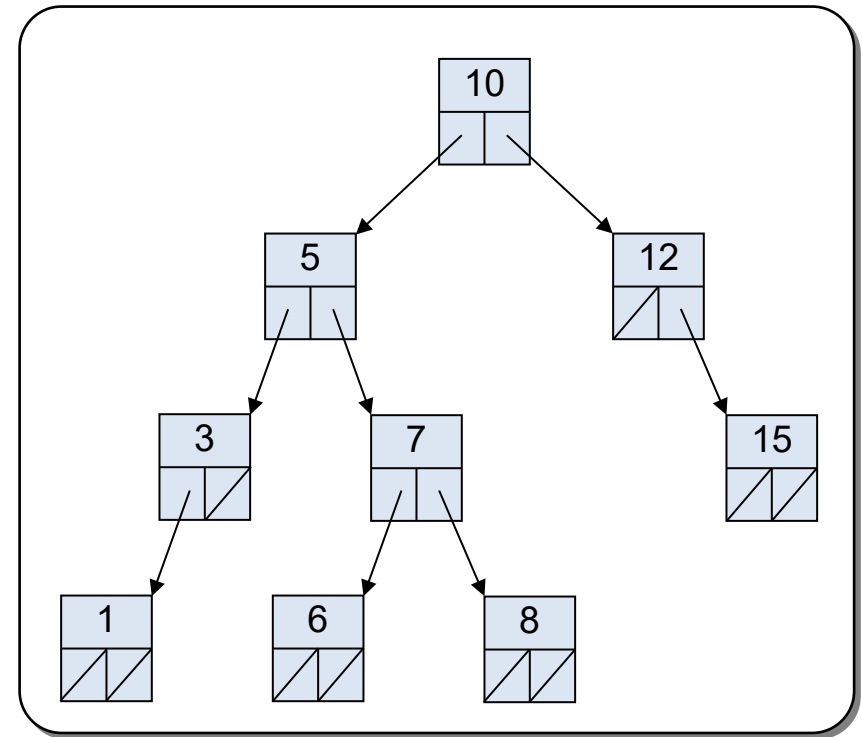
Probleme

- Oft werden die Daten unabsichtlich vorsortiert eingegeben.
- **remove** favorisiert Bäume mit tieferen linken Unterbäumen, da ein gelöschter Knoten immer durch den minimalen Knoten des rechten Unterbaums ersetzt wird.
- In binären Suchbäumen gibt es für einen Knoten im allgemeinen keinen effizienten Zugriff auf seinen InOrder-Vorgänger bzw. -Nachfolger.
- ...

§4.3 Binäre Suchbäume

Problem

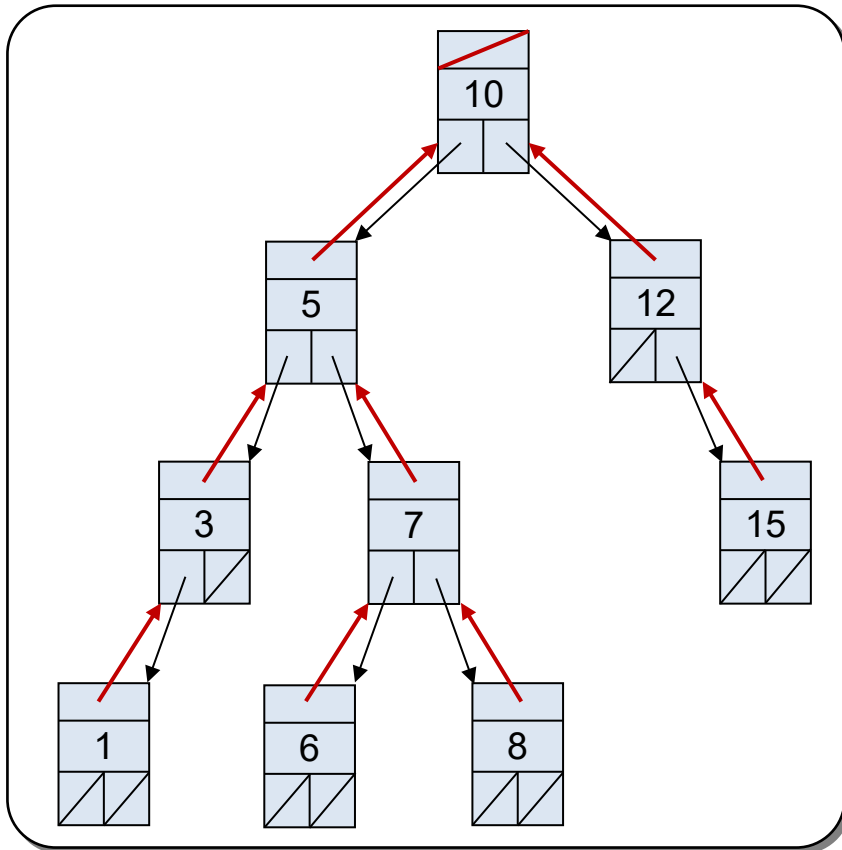
- In binären Suchbäumen gibt es für einen Knoten im allgemeinen keinen effizienten Zugriff auf seinen In-Order-Vorgänger bzw. -Nachfolger.
- ▶ Daher ist mit der bisherigen Datenstruktur für Suchbäume keine effiziente Vorwärts- bzw. Rückwärtstraversierung mit Iteratoren machbar.
- Beispiel:
 - Wie erreicht man den Nachfolger von 8?
 - Wie erreicht man den Vorgänger von 6?



§4.3 Binäre Suchbäume

Lösung

- Erweitere jeden Knoten um einen **Zeiger auf den Elternknoten**.



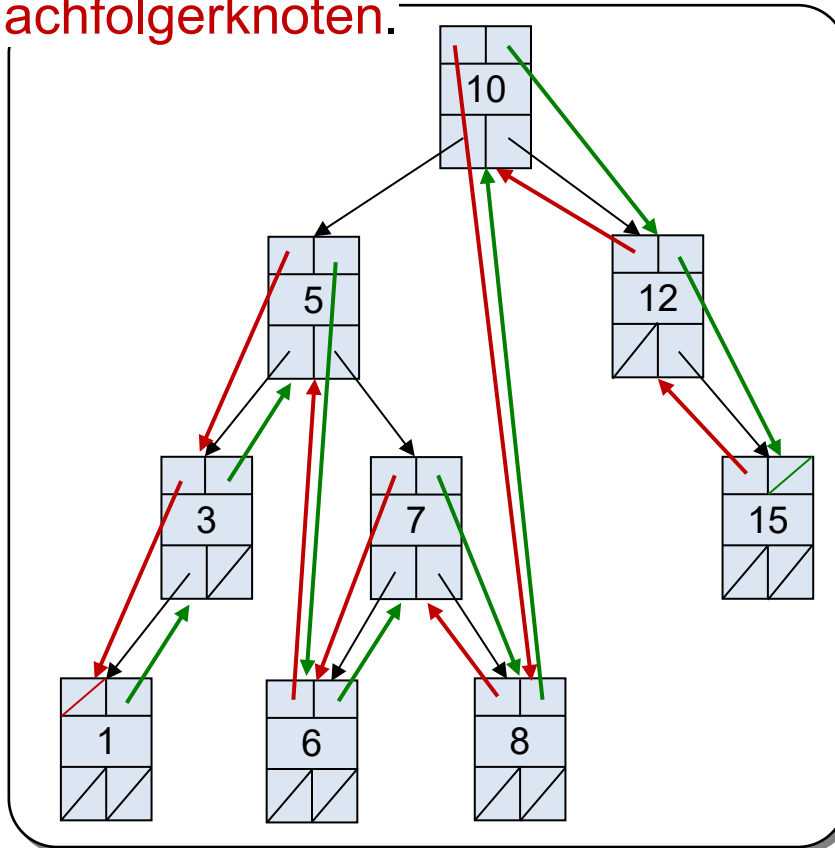
```
struct Node {
    KeyType    key;
    ValueType  val;
    Node*      left;
    Node*      right;
    Node*      parent;
};

Node::Node (...) {
    ...
    left    = null_ptr;
    right   = null_ptr;
    parent  = null_ptr;
}
```

§4.3 Binäre Suchbäume

Lösung (besser)

- Erweitere jeden Knoten um einen Zeiger auf den **Vorgängerknoten** und **Nachfolgerknoten**.



```
struct Node {
    KeyType    key;
    ValueType  val;
    Node*      left;
    Node*      right;
    Node*      succ;
    Node*      pred;
};

Node::Node (...) {
    ...
    left  = null_ptr;
    right = null_ptr;
    succ  = null_ptr;
    pred  = null_ptr;
}
```

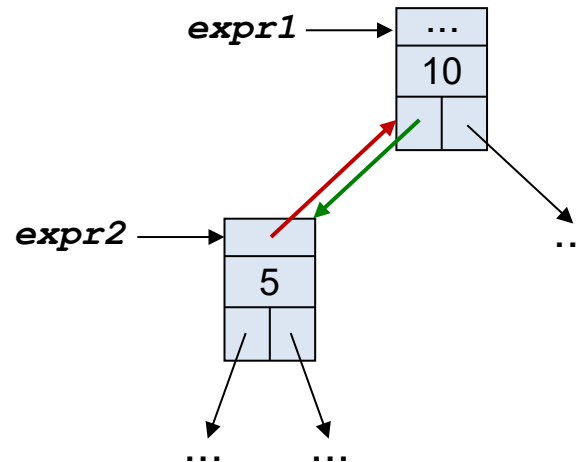
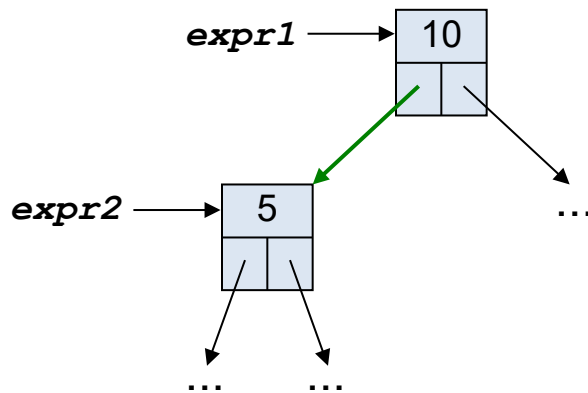

§4.3 Binäre Suchbäume

Lösung

- Überall, wo die **Struktur des Baums geändert** wird, werden die zusätzlichen Zeiger neu gesetzt.
- Z.B.

```
expr1.left = expr2;
```

```
expr1.left = expr2;  
if (expr1.left != null)  
    expr1.left.parent = expr1;
```

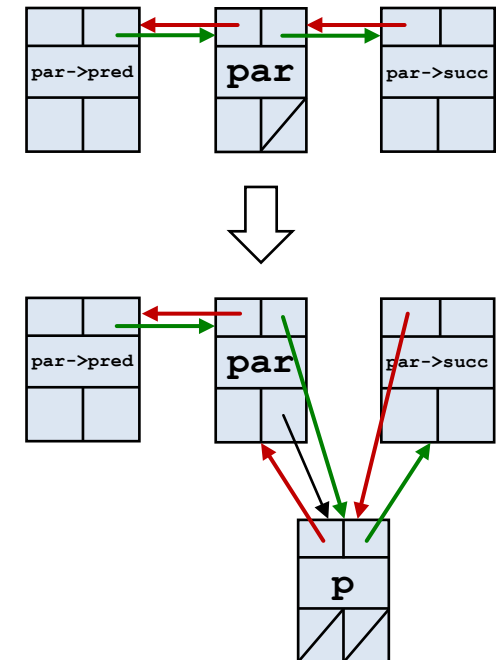


expr1 und **expr2** sind beliebige Ausdrücke vom Typ Node.

§4.3 Binäre Suchbäume

Algorithmen

```
bool insert(KeyType key, ValueType& val, Node*& par, Node*& p)
{
    if (p == 0) {                                     // key nicht vorhanden
        p = new Node(key, val);
        if (par->key < key) {
            p ->pred = par;
            p ->succ = par->succ;
            par->succ = p;
            if (p->succ) p->succ->pred = p;
        } else {
            p ->succ = par;
            p ->pred = par->pred;
            par->pred = p;
            if (p->pred) p->pred->succ = p;
        }
        return true;
    } ... // Rest wie auf Seite 22
}
```



§4.3 Binäre Suchbäume

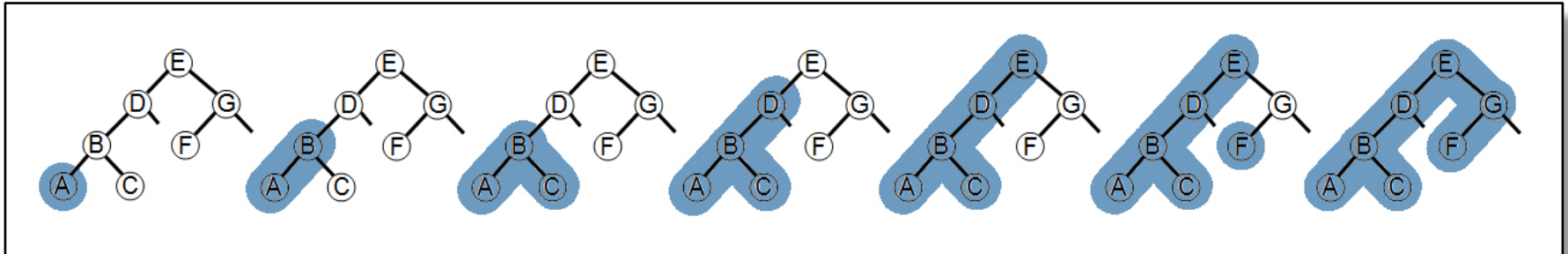
Algorithmen

```
bool remove(KeyType key, Node*& p)
{
    if (p == 0) return false;           // key nicht gefunden
    else if ( key < p->key ) return remove(key,p->left);
    else if ( key > p->key ) return remove(key,p->right);
    else if ( p->left == 0 || p->right == 0 ) { // 0&1 Kind
        Node* tmp = p;
        if ( p->pred != 0 ) p->pred->succ = p->succ;
        if ( p->succ != 0 ) p->succ->pred = p->pred;
        if ( p->left !=0 ) p = p->left; // by-pass nach links
        else                p = p->right; // by-pass nach rechts
        delete tmp;
        return true;
    } else {                             // 2 Kinder
        Node* min = p->succ;
        p->val      = min->val;
        p->key      = min->key;
        return remove(min->key,p->right); // min hat 0/1 Kind
    } }
}
```

§4.3 Binäre Suchbäume

Iterative In-Order-Traversierung

```
void InOrder(Node* p)
{
    while ( p != 0 ) {
        bearbeite(p->data);
        p = p->succ;
    }
}
```



Inhalt der Vorlesung

1. Problemstellung und Motivation
2. Binäre Bäume
3. Binäre Suchbäume
4. **AVL-Bäume**
5. Rot-Schwarz-Bäume
6. Heaps
7. B-Bäume*

* nicht prüfungsrelevant

§4.4 AVL-Bäume

Definition

Ein Suchbaum heißt **balancierter Suchbaum**, wenn er eine maximale Höhe von $O(\log n)$ hat.

- ▶ Alle Dictionary-Operationen (search, insert, remove) können in $O(\log n)$ ausgeführt werden.

Definition

Ein binärer Suchbaum heißt **AVL-Baum**, falls sich für jeden Knoten k die Höhen der beiden Teilbäume um höchstens 1 unterscheiden.

- Die Abkürzung AVL geht zurück auf **A**delson-**V**elskij und **L**andis.
- Ein AVL-Baum hat eine maximale Höhe von ca. $1.5 \log_2 n$. [Ottmann/Widmayer].

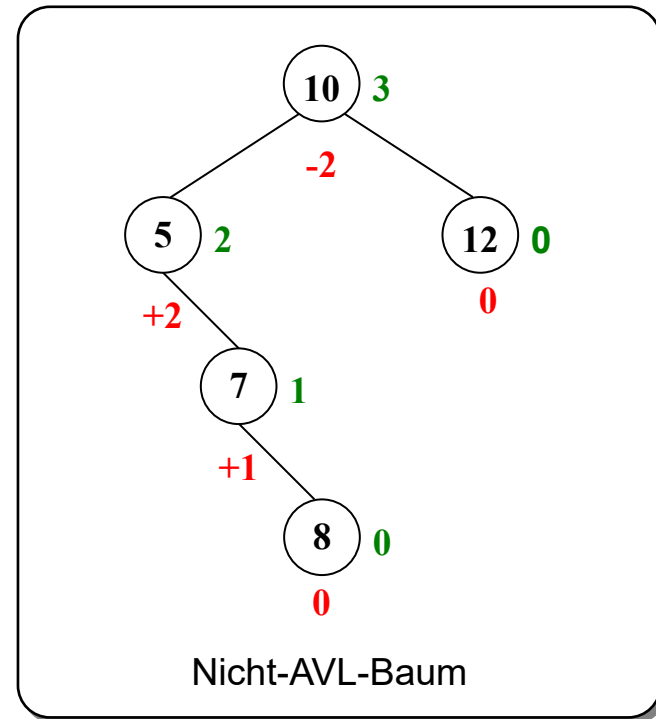
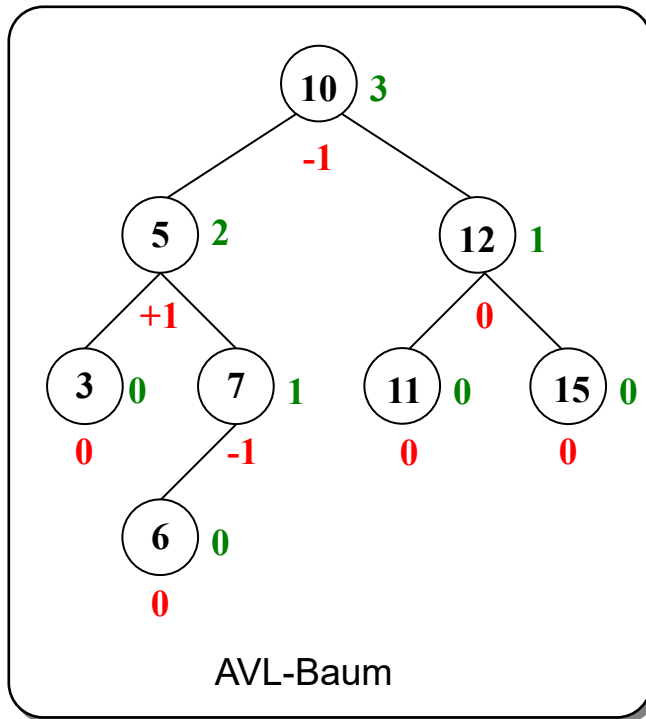
§4.4 AVL-Bäume

Beispiele:

Höhe des Teilbaums

Höhenunterschied = HöheVomRechtenTeilbaum – HöheVomLinkenTeilbaum

Höhe eines leeren Teilbaums ist -1



§4.4 AVL-Bäume

Was ist die Höhe h eines AVL-Baums mit n Knoten?

- Ein vollständiger Baum der Höhe h ist ein AVL-Baum mit maximaler Anzahl Knoten, d.h. $\log_2 n \leq h$.
- Ein AVL-Baum mit minimaler Anzahl Knoten bei gegebener Höhe h kann rekursiv konstruiert werden, z.B. durch

- Für die Anzahl $N(h)$ Knoten eines minimalen AVL-Baums der Höhe h folgt:

$$N(0) = 1$$

$$N(1) = 2$$

$$\begin{aligned} N(h) &= 1 + N(h-1) + N(h-2) \\ &= \dots = \text{Fib}(h+3) - 1 \end{aligned}$$

► $h \leq 1.44 \log_2(N(h) + 2) - 1.33$

Höhe $h = 0$



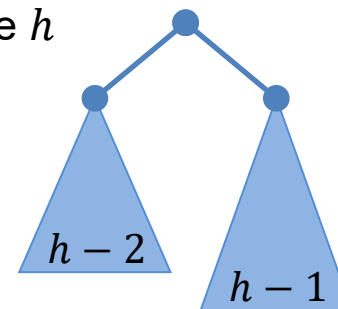
Höhe $h = 1$



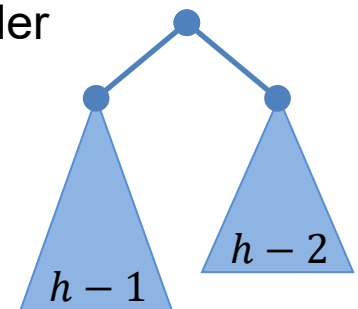
oder



Höhe h



oder



§4.4 AVL-Bäume

Eigenschaft von AVL-Bäumen

1. Für die Höhe h eines AVL-Baums mit n Knoten gilt:

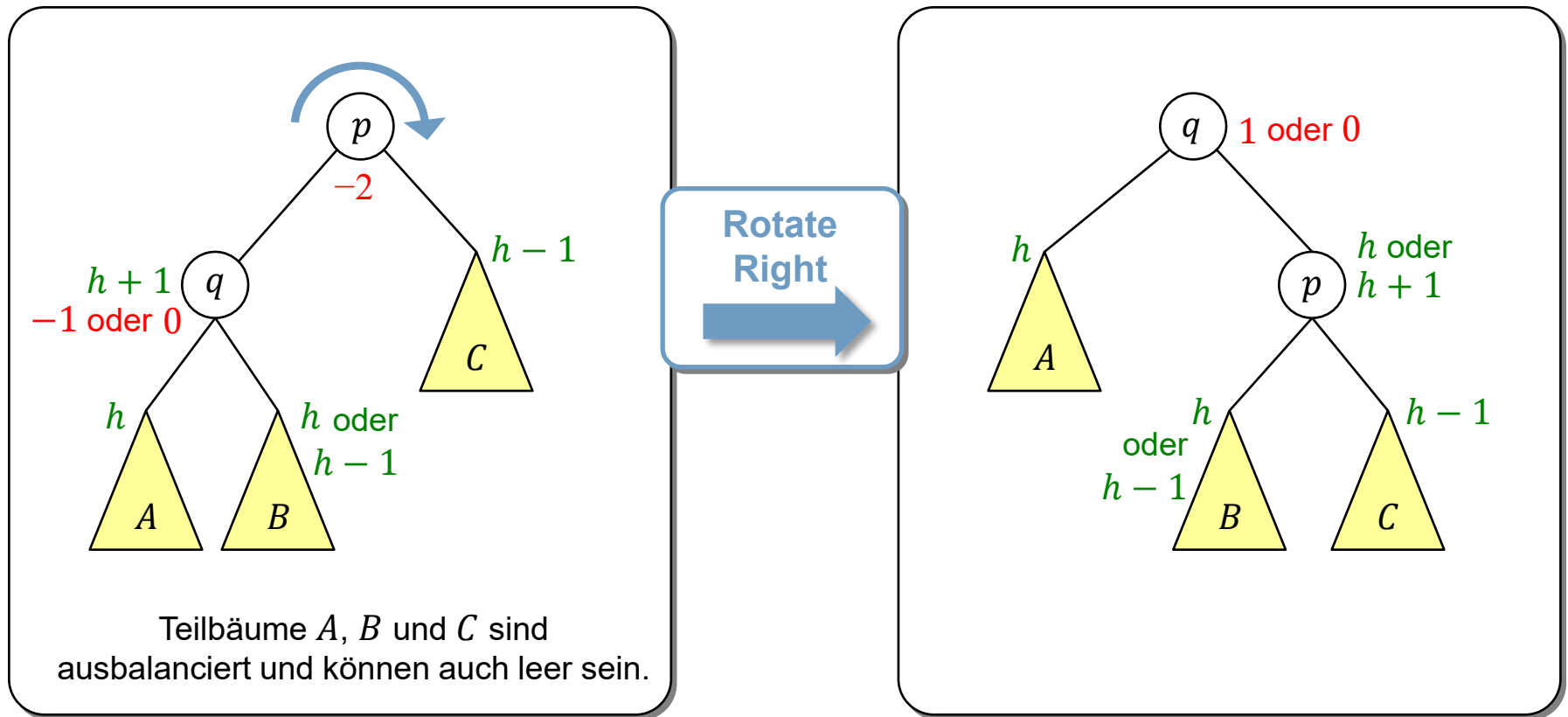
$$\log_2 n \leq h \leq 1.5 \cdot \log_2 n .$$

2. Alle Dictionary-Operationen (search, insert, remove) können in $O(\log n)$ realisiert werden
3. Ein unbalancierter Baum, der
 - einen Höhenunterschied von
 - -2 (linkslastiger Baum, aka left heavy) oder
 - $+2$ (rechtslastiger Baum, aka right heavy) hat und
 - dessen Teilbäume ausbalanciert sind,lässt sich durch so genannte **Rotationsoperationen** ausbalancieren.
 - Dabei treten vier verschiedene Fälle auf...

§4.4 AVL-Bäume

Fall A: Der Baum ist linkslastig, d.h. der Höhenunterschied ist -2 .

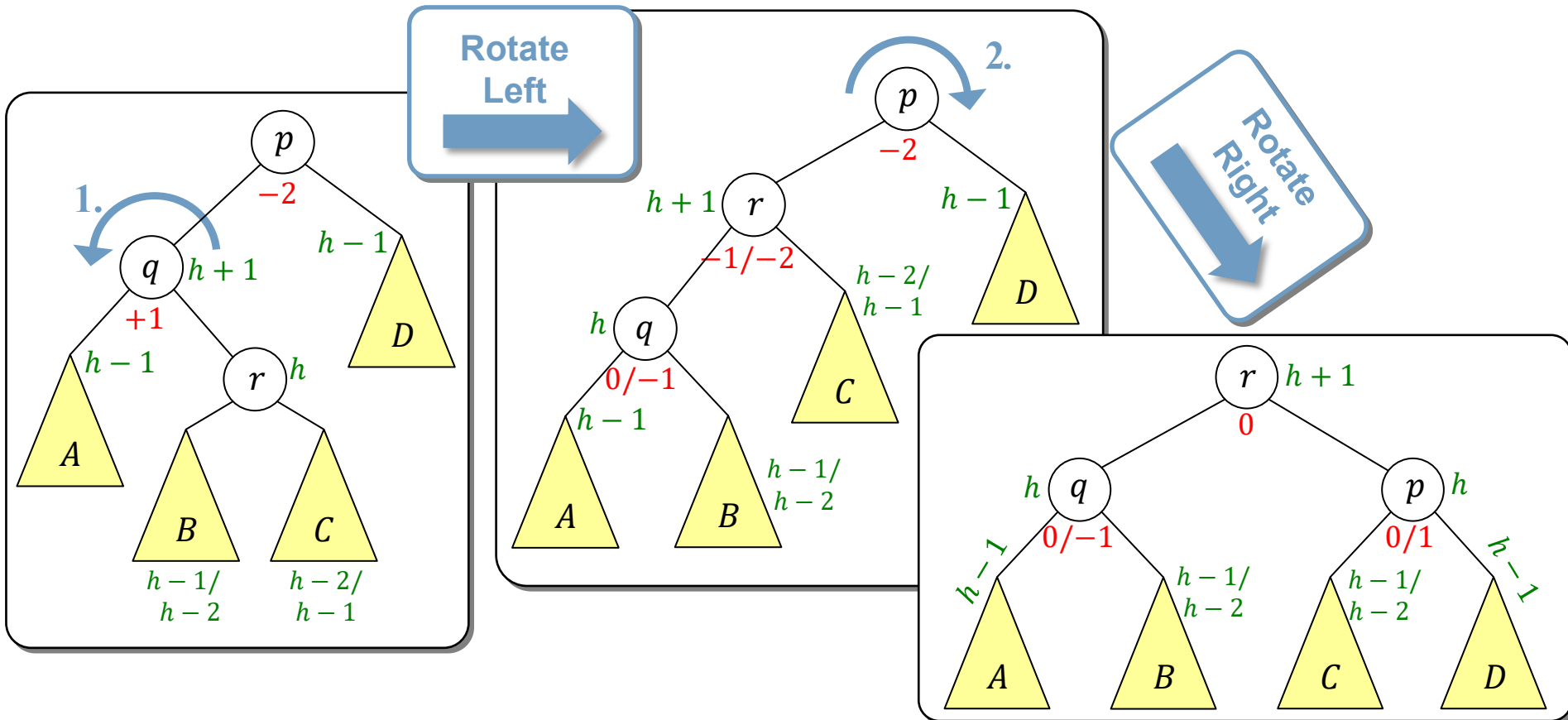
Fall A1: Der linke Teilbaum hat Höhenunterschied 0 oder -1 .



§4.4 AVL-Bäume

Fall A: Der Baum ist linkslastig, d.h. der Höhenunterschied ist -2 .

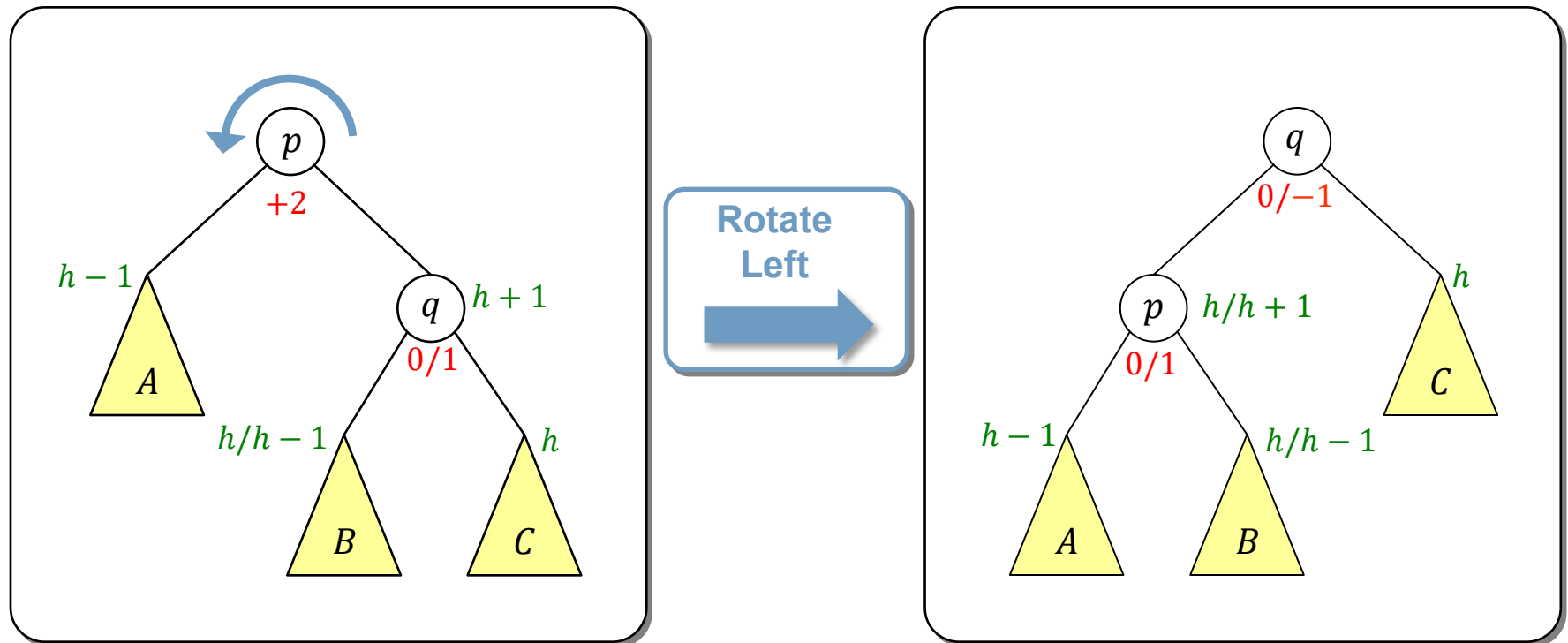
Fall A2: Der linke Teilbaum hat Höhenunterschied $+1$.



§4.4 AVL-Bäume

Fall B: Der Baum ist rechtslastig, d.h. der Höhenunterschied ist **+2**.

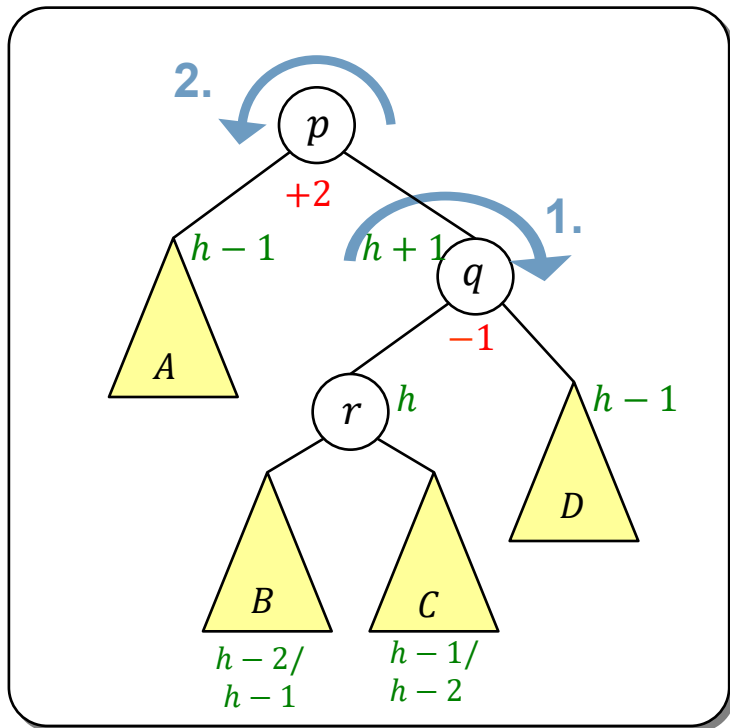
Fall B1: Der rechte Teilbaum hat Höhenunterschied **0** oder **+1**.



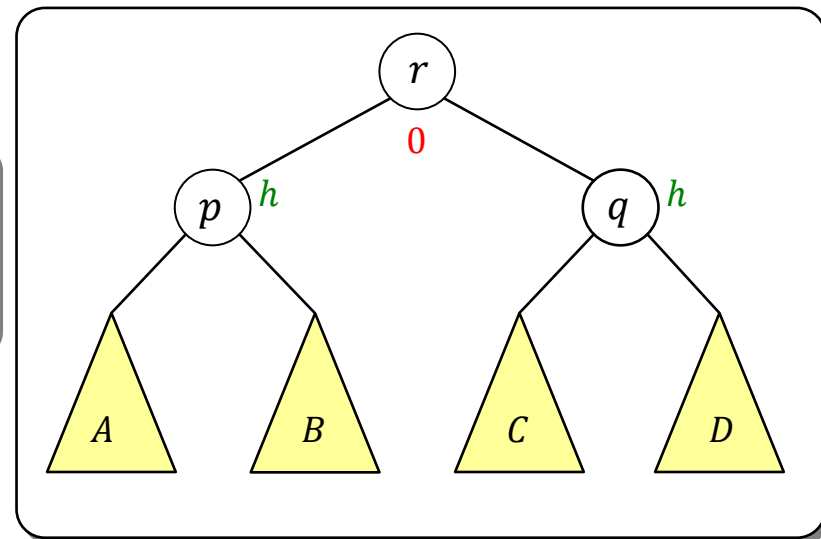
§4.4 AVL-Bäume

Fall B: Der Baum ist rechtslastig, d.h. der Höhenunterschied ist $+2$.

Fall B2: Der rechte Teilbaum hat Höhenunterschied -1 .



Rotate
Right Left
➔



Prinzip

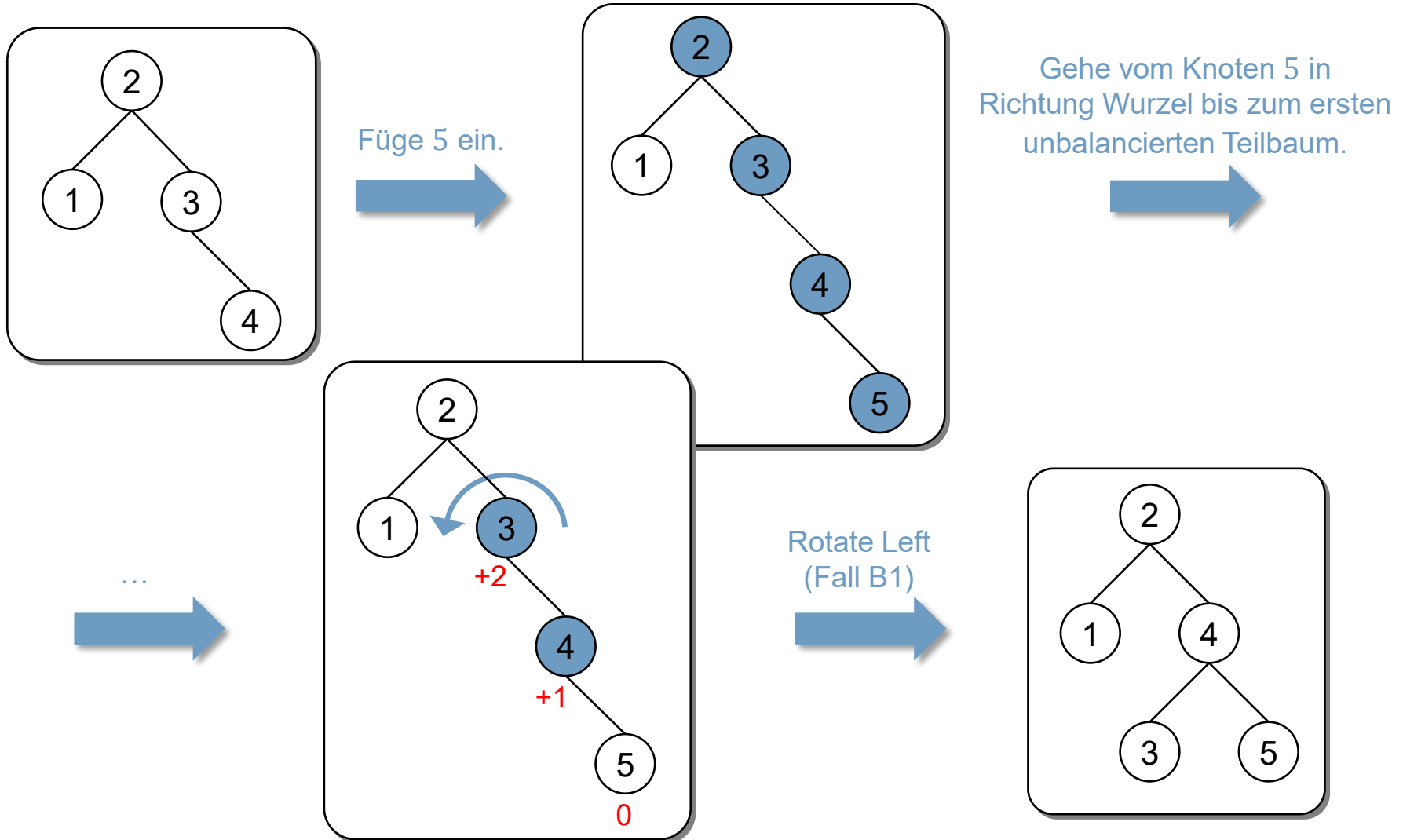
1. Einfügen bzw. Löschen wie bei binären Suchbäumen.
2. Gehe dann von der Einfüge- bzw. Löschstelle bis zur Wurzel (aka bottom-up) und balanciere, falls notwendig, mit Rotationsoperationen lokal aus.

Bemerkungen:

- Da der Baum vor dem Einfügen bzw. Löschen ausbalanciert war, haben alle Teilbäume einen Höhenunterschied von -1 , 0 oder $+1$.
 - Damit können nach dem Schritt 1. nur die Fälle A1, A2, B1 oder B2 auftreten.
 - Beim Einfügen ist in Schritt 2. **maximal eine** Rotationsoperation notwendig.
 - Beim Löschen sind in Schritt 2. i.A. **mehrere** Rotationsoperation auf dem Pfad zur Wurzel notwendig, siehe Beispiel auf Seite 57f.

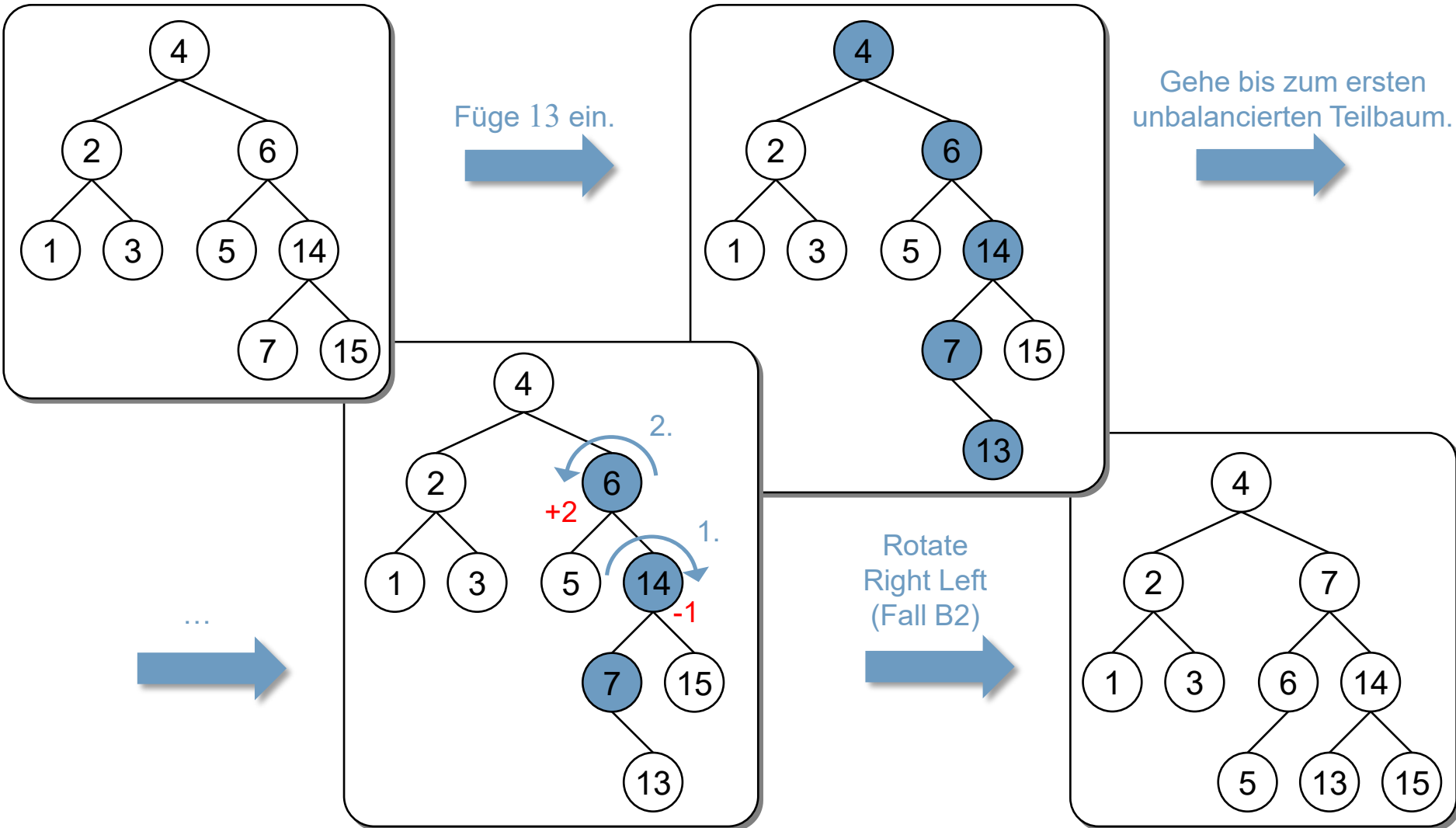
§4.4 AVL-Bäume

Beispiele: Einfügen



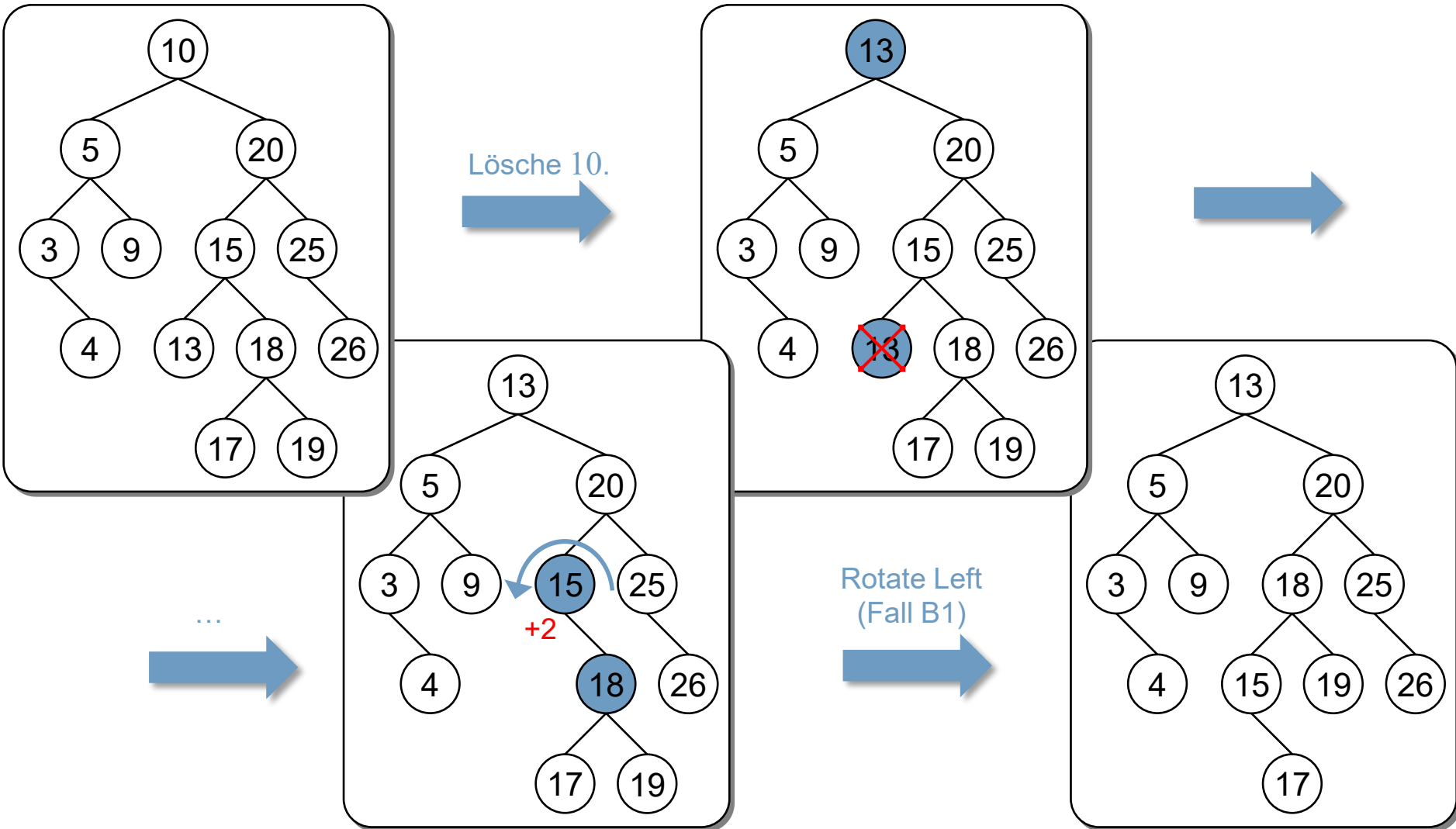
§4.4 AVL-Bäume

Beispiele: Einfügen



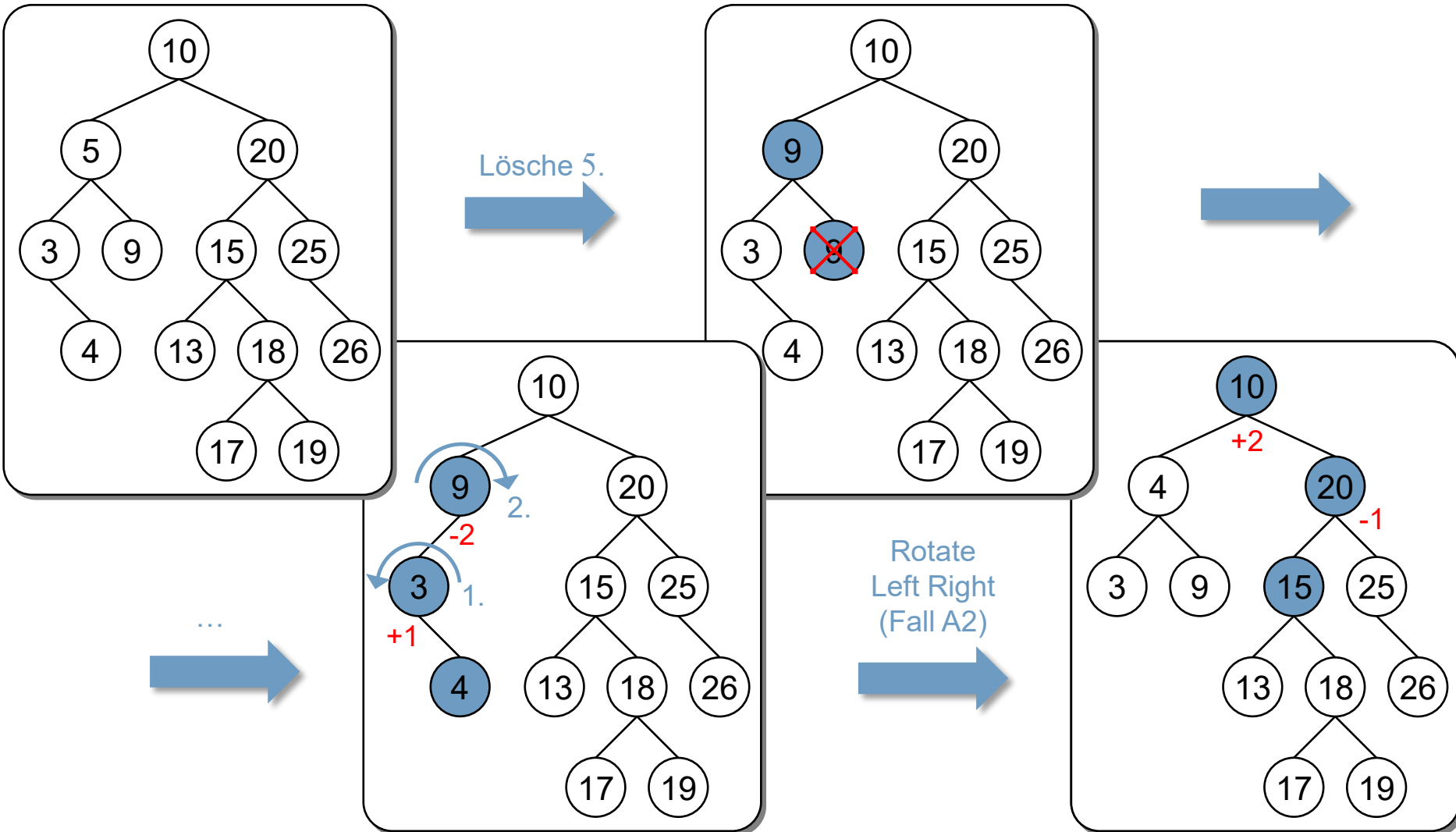
§4.4 AVL-Bäume

Beispiele: Löschen



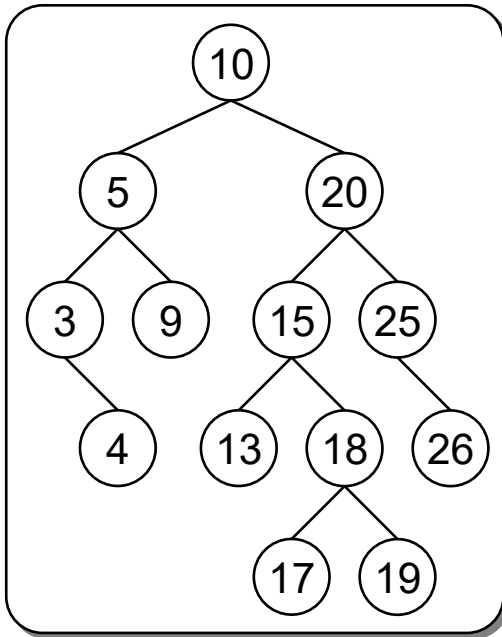
§4.4 AVL-Bäume

Beispiele: Löschen



§4.4 AVL-Bäume

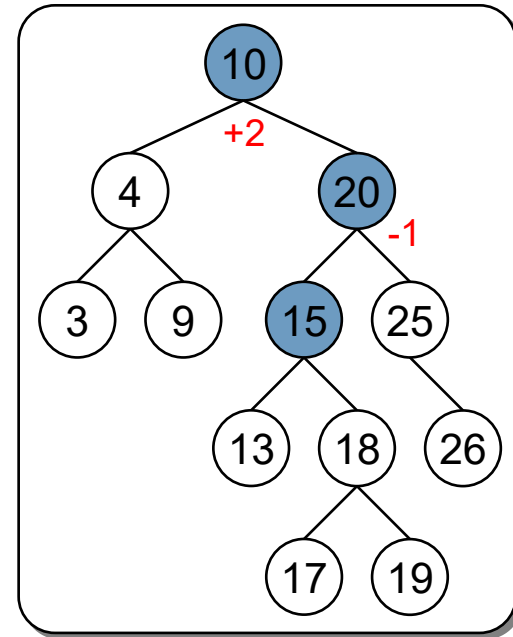
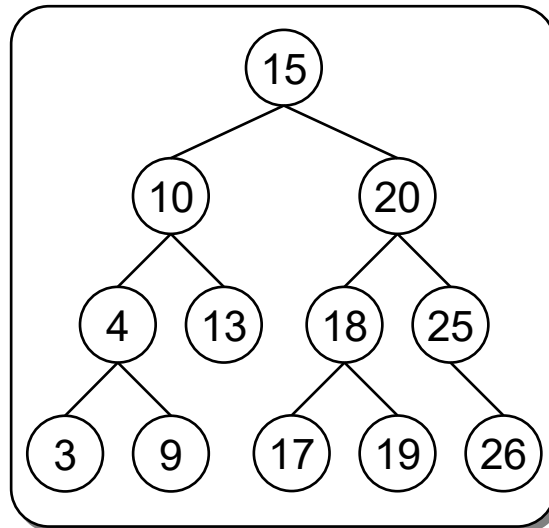
Beispiele: Löschen



Lösche 5.

Rotate
Left Right
(Fall A2)

Rotate
Right Left
(Fall B2)



§4.4 AVL-Bäume

Algorithmen

Datenstruktur

- Wie bei binären Suchbäumen.
- Zusätzlich wird für jeden Knoten noch die Höhe des zugehörigen Teilbaums gespeichert.

```
struct AVLNode
{
    int          height;
    KeyType      key;
    ValueType    val;
    Node*        left;
    Node*        right;
};
```

Algorithmen

- Erweitere die **insert**-Operation für binäre Suchbäume um Balancieroperationen, siehe nächste Seite.
- Erweitere die **remove**-Operation für binäre Suchbäume analog.
- Die **search**-Operation für binäre Suchbäume bleibt unverändert.

§4.4 AVL-Bäume

Algorithmen

```
bool insert(KeyType key, ValueType& v, AVLNode*& p)
{
    bool result;
    if (p == null) {                // key nicht vorhanden
        p = new AVLNode;
        p->key = key;
        p->val = v;
        p->height = 0;
        result = true;
    }
    else if ( key < p->key ) result = insert(key,v,p->left) ;
    else if ( key > p->key ) result = insert(key,v,p->right) ;
    else result = false; // key vorhanden
    if (result)
        balance(p) ;
    return result;
}
```

Ein Blatt hat die Höhe 0.

Knoten wurde eingefügt:
Höhe aktualisieren und
ausbalancieren, falls notwendig.

§4.4 AVL-Bäume

Algorithmen

```
void balance (AVLNode*& p)
{
    if (p == null) return;

    // Höhe anpassen.
    p->height = max(getHeight(p->left), getHeight(p->right)) + 1;

    if      (getBalance(p) == -2) // Baum ist linkslastig.
        if      (getBalance(p->left) <= 0) rotateRight(p); // A1
        else rotateLeftRight(p); // A2
    else if (getBalance(p) == +2) // Baum ist rechtslastig.
        if      (getBalance(p->right) >= 0) rotateLeft(p); // B1
        else rotateRightLeft(p); // B2
}
```

Aufwand: $O(1)$, falls alle Subroutinen konstanten Aufwand haben.

§4.4 AVL-Bäume

Algorithmen

Hilfsroutinen:

```
int getHeight(const AVLNode* p)
{
    if (p == null) return -1; // Leerer Baum hat Höhe -1
    else          return p->height;
}
```

```
int getBalance(const AVLNode* p)
{
    if (p==null) return 0; // Leerer Baum hat Höhenunterschied 0
    else        return getHeight(p->right) - getHeight(p->left);
}
```

Aufwand: $O(1)$.

§4.4 AVL-Bäume

Algorithmen

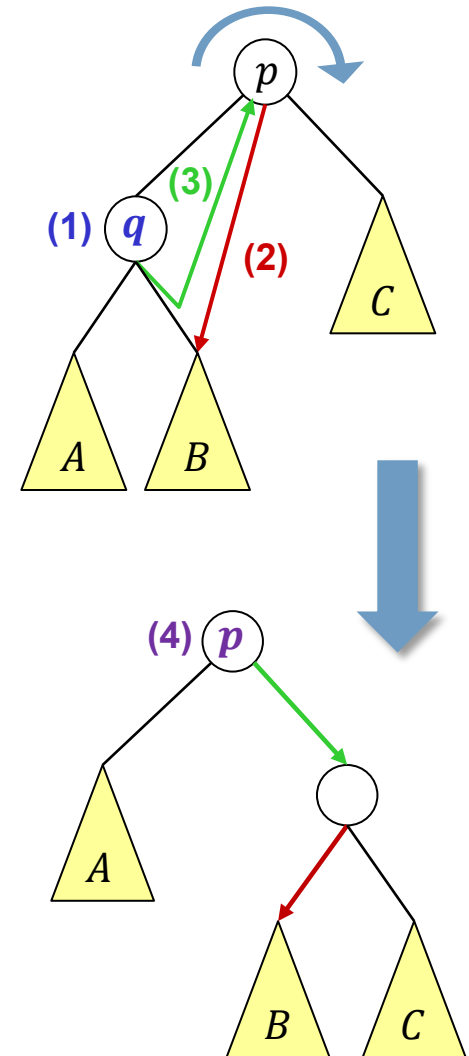
Rotationen:

```
void rotateRight(AVLNode*& p)
{
    // Vor.: p->left!=0, sonst wäre p nicht
    //          linkslastig unbalanciert.

    AVLNode* q = p->left;      // (1)
    p->left     = q->right;      // (2)
    q->right    = p;            // (3)

    p->height   = max( getHeight(p->left ),
                     getHeight(p->right) )+1;
    q->height   = max( getHeight(q->left ),
                     getHeight(q->right) )+1;
    p          = q;            // (4)
}
```

Aufwand: $O(1)$.



§4.4 AVL-Bäume

Algorithmen

Rotationen:

```
void rotateLeft(AVLNode*& p)
{ // Vor.: p->right!=0, ...
  // analog zu rotateRight
}
```

```
void rotateLeftRight(AVLNode*& p)
{ // Vor.: p->left!=0, ...
  rotateLeft (p->left);
  rotateRight (p);
}
```

```
void rotateRightLeft(AVLNode*& p)
{ // Vor.: p->right!=0, ...
  rotateRight (p->right);
  rotateLeft (p);
}
```

Laufzeit

- **search:** $O(\log n)$, weil ein AVL-Baum balanciert ist.
- **Rotationen und Balancieren**
 - **rotateRight:** $O(1)$.
 - **rotateLeft:** $O(1)$.
 - **rotateLeftRight:** $O(1)$.
 - **rotateRigthLeft:** $O(1)$.
 - **balance:** $O(1)$.
- **insert:** $O(\log n)$.
- **remove:** $O(\log n)$.

Inhalt der Vorlesung

1. Problemstellung und Motivation
2. Binäre Bäume
3. Binäre Suchbäume
4. AVL-Bäume
5. **Rot-Schwarz-Bäume**
6. Heaps
7. B-Bäume*

* nicht prüfungsrelevant

§4.5 Rot-Schwarz-Bäume

Prinzip von AVL-Bäumen

1. Einfügen bzw. Löschen wie bei binären Suchbäumen.
2. Gehe dann von der Einfüge- bzw. Löschstelle bis zur Wurzel und balanciere, falls notwendig, mit Rotationsoperationen bottom-up (d.h. von den Blättern zur Wurzel) lokal aus.

Bemerkungen:

- Beim Einfügen ist in Schritt 2. max. eine Rotationsoperation notwendig.
- Beim Löschen sind in Schritt 2. i.a. mehrere Rotationsoperation auf dem Pfad zur Wurzel notwendig.
- ▶ Dies ist unsymmetrisch.
- ▶ Ungünstige Ausführungszeit bei vielen Einfüge-/Lösche-Operationen.

§4.5 Rot-Schwarz-Bäume

Prinzip von Top-Down-Rot-Schwarz-Bäumen

1. Gehe von der Wurzel bis zu der Einfüge- bzw. Löschstelle wie bei binären Suchbäumen und
 - a. balanciere den Baum auf diesem Weg prospektiv aus.
2. *entfällt bei top-down*

Bemerkungen:

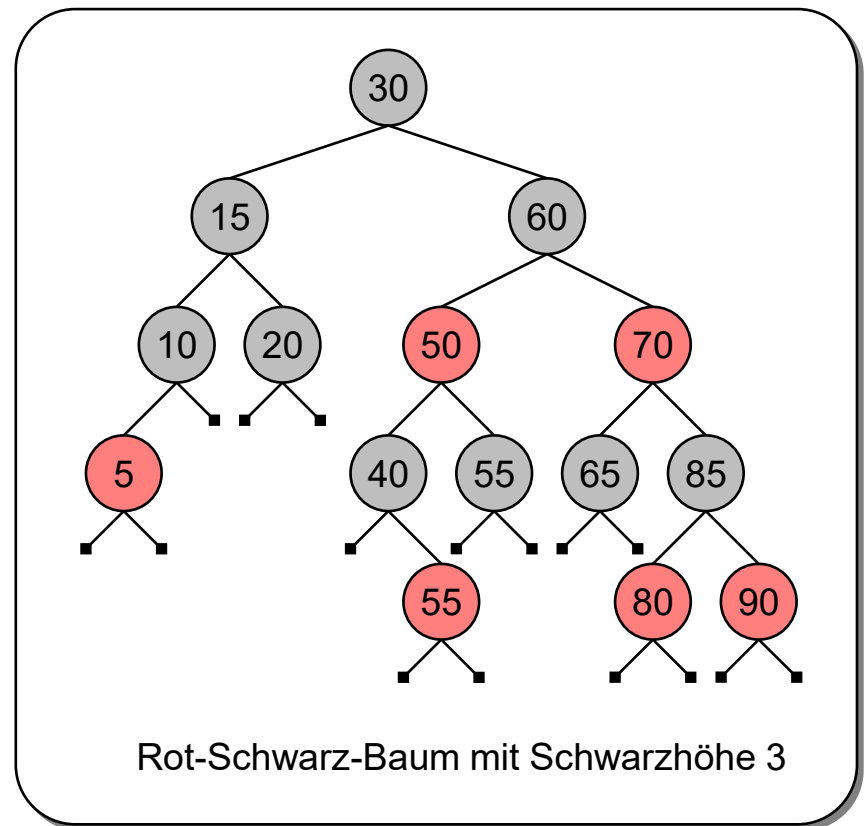
- Rot-Schwarz-Baum = red-black-tree
- Die meisten red-black-trees sind bottom-up implementiert.
 - Viele Fallunterscheidungen notwendig!
- **Top-Down:** Vorhersage wie gut auf den tieferen Levels der Baum balanciert sein wird.
 - Unterscheidung von roten und schwarzen Knoten.

§4.5 Rot-Schwarz-Bäume

Definition Rot-Schwarz-Baum

Ein **Rot-Schwarz-Baum** ist ein binärer Suchbaum mit folgenden Färbungsregeln:

- (1) Jeder Knoten ist entweder rot oder schwarz.
- (2) Die Wurzel und jeder leere Knoten (**null**) ist immer schwarz.
- (3) Ein roter Knoten darf kein rotes Kind haben.
- (4) Jeder Pfad von der Wurzel zu einem **null**-Zeiger hat die gleiche Anzahl an schwarzen Knoten, aka **Schwarzhöhe**.



§4.5 Rot-Schwarz-Bäume

Was ist die Höhe h eines Rot-Schwarz-Baums mit n Knoten?

- Ein vollständiger Baum der Höhe h mit rote-gefärbter letzter Ebene ist ein Rot-Schwarz-Baum mit maximaler Anzahl Knoten, d.h. $\log_2 n \leq h$.
- Ein Rot-Schwarz-Baum mit minimaler Anzahl Knoten bei gegebener Höhe h kann konstruiert werden, z.B. durch

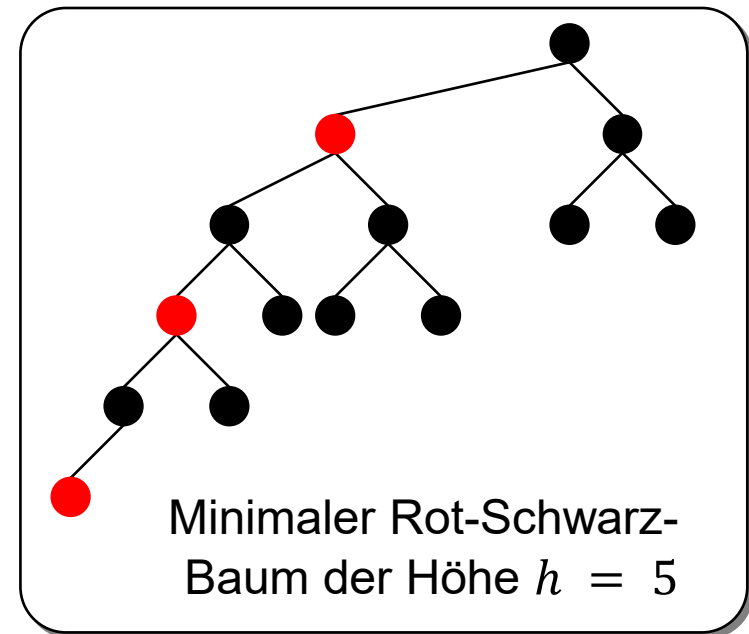
- einen Pfad der Länge h mit alternierenden Knotenfarben und
- sonst nur schwarze Knoten.
- Anzahl $N(h)$ Knoten eines minimalen Rot-Schwarz-Baum der Höhe h :

$$N(0) = 1,$$

$$N(h) = 1 + N(h-1) + 2^{\lfloor (h-1)/2 \rfloor}$$

$$= \dots \geq 2 \cdot 2^{h/2} - 1.$$

► $h \leq 2 \log_2(N(h) + 2) - 2.$



§4.5 Rot-Schwarz-Bäume

Eigenschaften von Rot-Schwarz-Bäumen

1. Für die Höhe h eines Rot-Schwarz-Baums mit n Knoten gilt:

$$\log_2 n \leq h \leq 2 \cdot \log_2 n.$$

2. Alle Dictionary-Operationen (search, insert, remove) können in $O(\log n)$ realisiert werden.
3. Die schwarzen Knoten repräsentieren eine globale Eigenschaft in Form der Schwarzhöhe des Baums.
4. Die roten Knoten repräsentieren eine lokale Eigenschaft in Form eines lokalen Pfadlängenunterschiedes.
5. Jeder AVL-Baum ist ein Rot-Schwarz Baum, ...
:

§4.5 Rot-Schwarz-Bäume

Algorithmen

Datenstruktur

- Wie bei binären Suchbäumen.
- Zusätzlich wird für jeden Knoten noch die Farbe des Knotens und ein Zeiger auf den Elternknoten gespeichert.

```
struct RBNode
{
    ColorType color;
    KeyType    key;
    ...
    Node*      left;
    Node*      right;
    Node*      parent;
};
```

Algorithmen

- Verwende iterative Varianten der Operationen, weil dies den Zugriff auf Vorgänger-Knoten erleichtert.
- Erweitere die iterative **insert**-Operation für binäre Suchbäume um Balancieroperationen, siehe nächste Seite.
- Erweitere die **remove**-Operation für binäre Suchbäume analog.
- Die **search**-Operation für binäre Suchbäume bleibt unverändert.

Einfügen eines neuen Knotens – Prinzip

- Ein neues Element wird als Blatt eingefügt.
- Ein neues Element wird an der Stelle eingefügt, an die erfolglose Suche nach diesem Element endet.
- Um die Schwarzhöhe nicht zu ändern (Regel (4)), wird ein rotes Element eingefügt.
- Damit es dabei nicht zu **red-red-violations** (Regel (3)) kommt, werden präventiv und prospektiv Farbwechsel nach oben im Baum durchgeführt.
 - Dabei kann es zu **red-red-violations** (Regel (3)) kommen, die durch Rotationen aufgelöst werden.

§4.5 Rot-Schwarz-Bäume

Algorithmen

```
bool RBTREE::insert(KeyType key)
{
    if (root==null) { root=new RBNODE(key,black); return true; }
    RBNODE* q = root, p = null; // q = current, p = parent
    while (q != null) {
        // 1. move red-color upwards in the tree and re-balance
        if (bothKidsRed(q)) { reColorUp(q); reBalance(key,p,q); }
        p = q;
        // 2. travers downwards in the tree
        if ( key < q->key ) q = q->left;
        else if ( key > q->key ) q = q->right;
        else return false; // key already present in tree
    }
    q = new Node(key, red, p);
    if (key < p->key) p->left = q; else p->right = q;
    reBalance(key,p,q); // re-balance only
    return true;
}
```

§4.5 Rot-Schwarz-Bäume

Algorithmen

```
bool RBTREE::bothKidsRed(RBNode* q)
{
    return isRed(q->left) && isRed(q->right);
}
```

```
bool RBTREE::isRed(RBNode* q)
{
    return q != null && q->color == red;
}
```

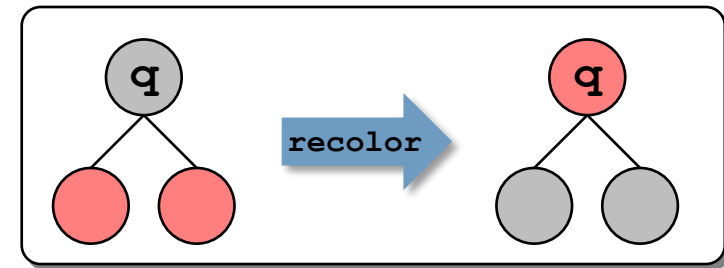
§4.5 Rot-Schwarz-Bäume

Algorithmen

```
void RBTree::reColorUp(RBNode*& q)
{ // Move red-color upwards in the tree.
  q->color = (q==root)? black: red;
  if (q->left != null) q->left ->color = black;
  if (q->right != null) q->right->color = black;
}
```

Farbwechsel (reColorUp)

- Für einen Knoten q mit zwei roten Kindern, führe einen Farbwechsel durch.
- Falls q die Wurzel ist, dann bleibt q schwarz.
- **Red-Red-Violation:** Dies kann zu zwei aufeinanderfolgenden roten Knoten führen, q und sein Elternknoten $q \rightarrow \text{parent}$!
 - ▶ Jetzt rotieren...



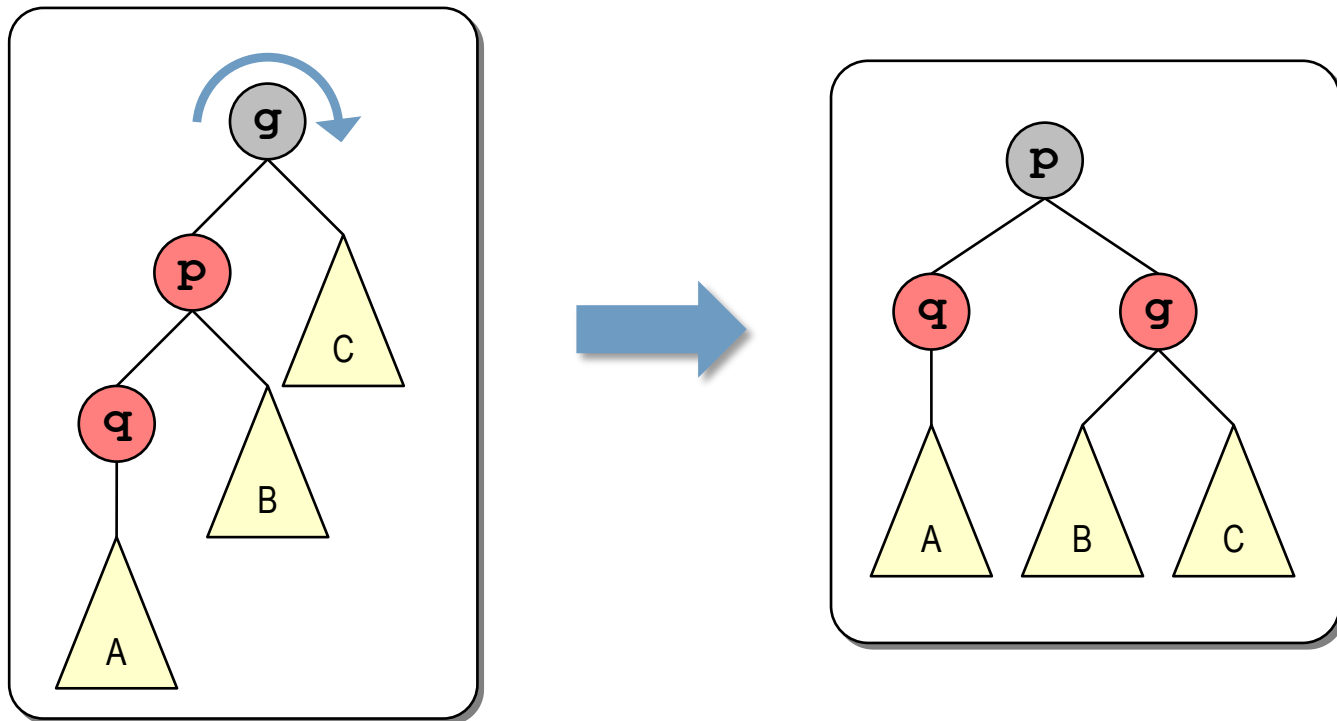
§4.5 Rot-Schwarz-Bäume

Algorithmen

Rotationen – vier Fälle

(A1) **rotateRight**: **p** (red) ist linkes Kind von **g** (black) und **q** (red) ist linkes Kind von **p**.

(B1) **rotateLeft**: analog.

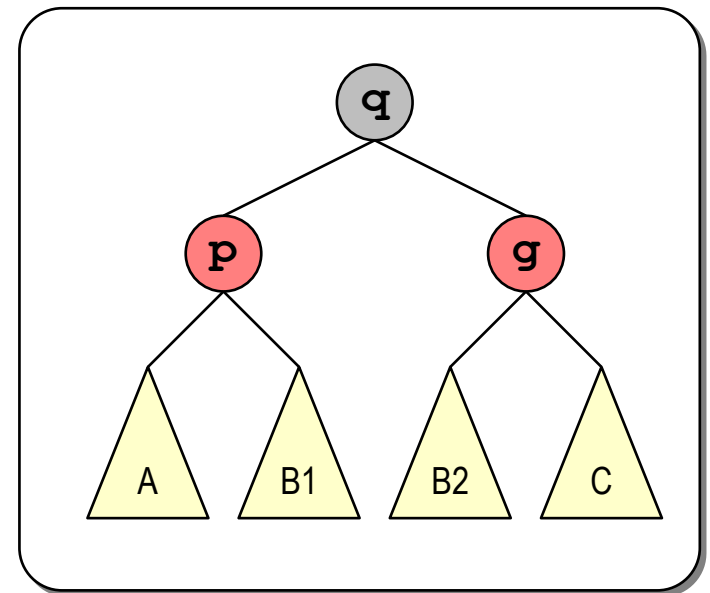
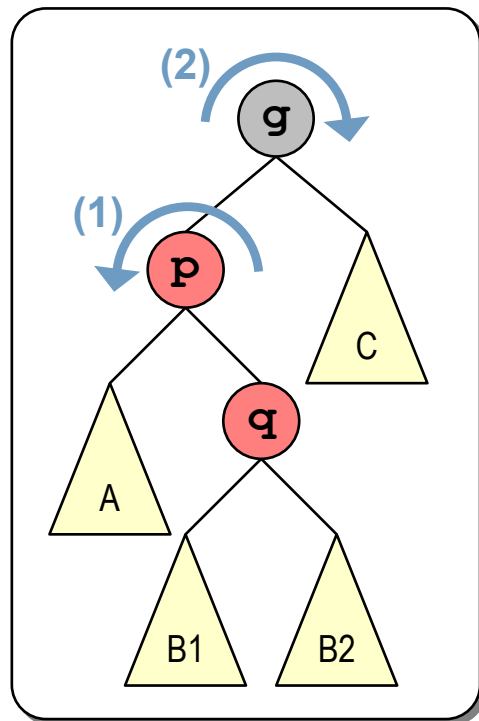


§4.5 Rot-Schwarz-Bäume

Algorithmen

(A2) **rotateLeftRight**: **p** (red) ist linkes Kind von **g** (black) und **q** (red) ist rechtes Kind von **p**.

(B2) **rotateRightLeft**: analog.



§4.5 Rot-Schwarz-Bäume

Algorithmen

```
bool RBTree::reBalance(KeyType key, RBNNode*& p, RBNNode*& q)
{ // Assumes: q->color==red
  if (isRed(p) && isRed(q)) { // red-red-violation
    RBNNode* g = p->parent; // g = grandparent
    g->color = red;
    if ( (key < g->key) != (key < p->key) )
      p = rotate(key, g);
    q = rotate(key, g->parent); // g->parent = greatgrandpa
    q->color = black;
  }
}
```

§4.5 Rot-Schwarz-Bäume

Algorithmen

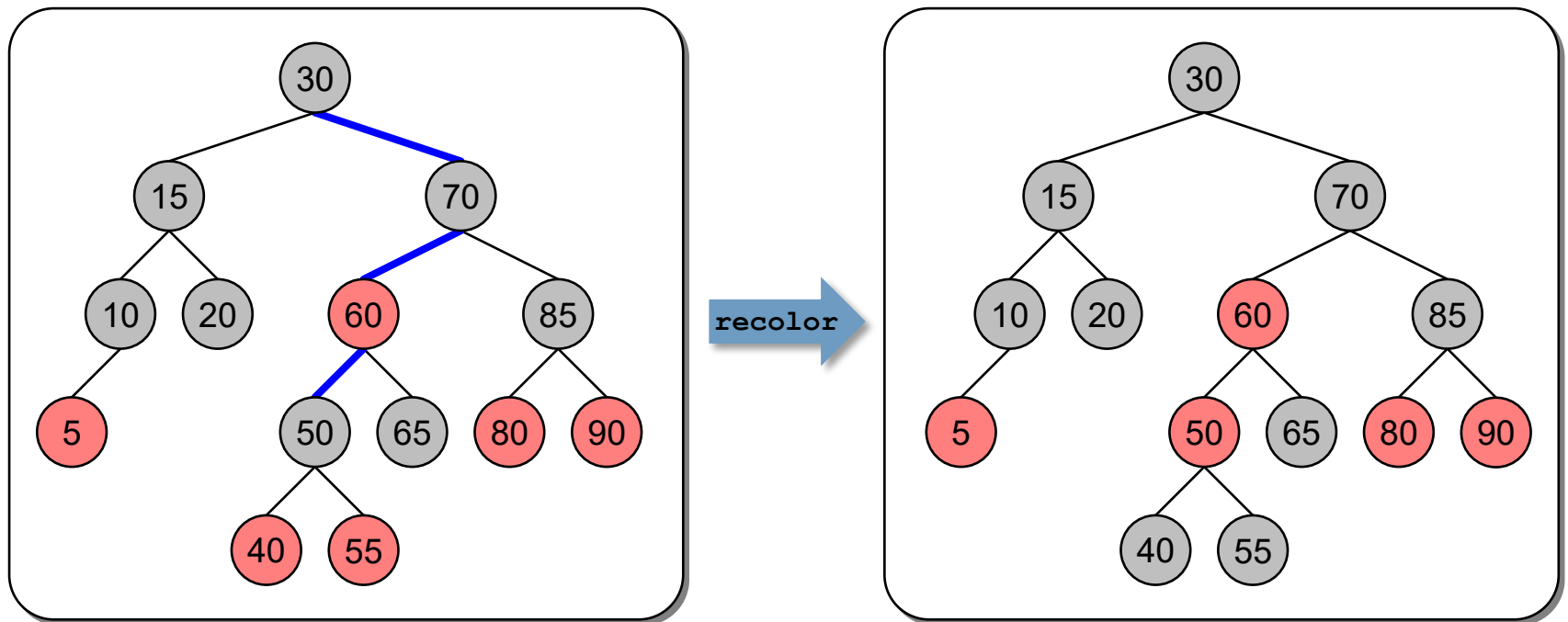
```
RBNode* RBTree::rotate(KeyType key, RBNode* par)
{
    if (key < par->key) {
        if (key < par->left->key) rotateLeft (par->left);
        else rotateRight (par->left);
        return par->left;
    } else {
        if (key < par->right->key) rotateLeft (par->right);
        else rotateRight (par->right);
        return par->right;
    }
}
```

§4.5 Rot-Schwarz-Bäume

Algorithmen

Beispiel

- Einfügen von 45.
 1. Der Baum wird von der Wurzel top-down durchlaufen: 30, 70, 60, 50.
 2. Farbwechsel bei 50.



§4.5 Rot-Schwarz-Bäume

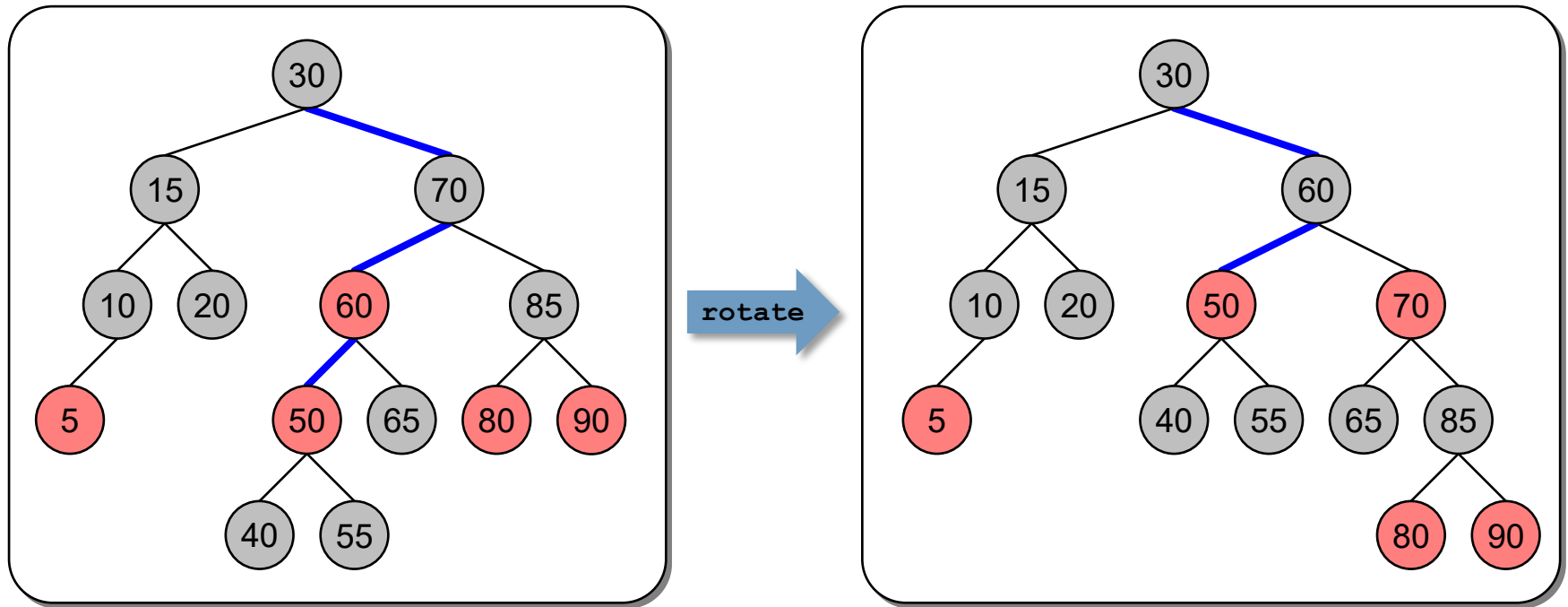
Algorithmen

Beispiel

- Einfügen von 45.

...

3. Rechtsrotation bei 70.

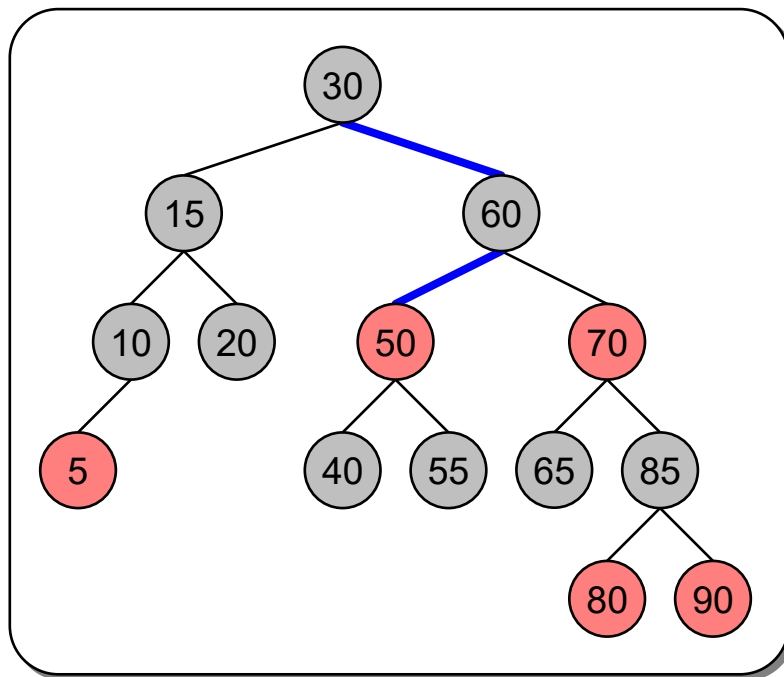


§4.5 Rot-Schwarz-Bäume

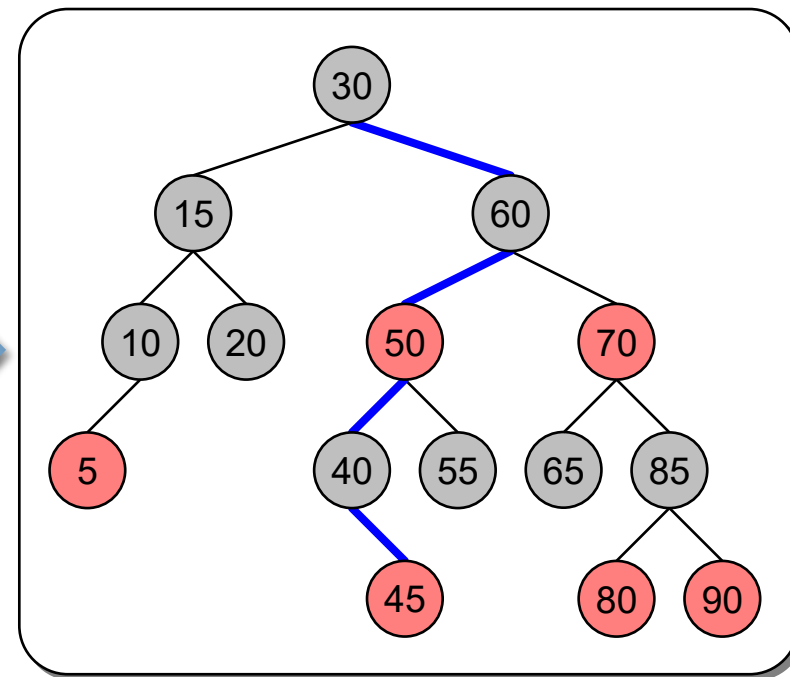
Algorithmen

Beispiel

- Einfügen von 45.
 4. Durchlaufen des Baums bei 50 fortsetzen.
 5. Knoten 45 als roter Knoten rechts von 40 einsetzen.



einfügen

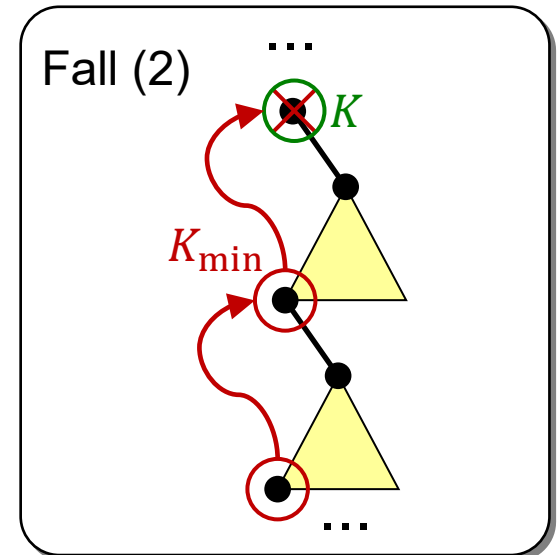
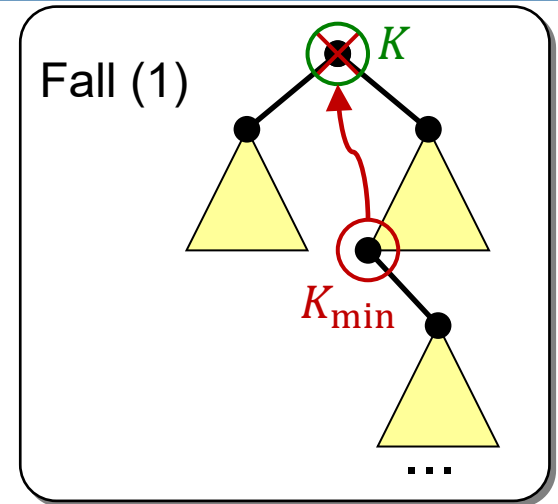


Löschen eines Knotens – Prinzip

- Um die Schwarzhöhe nicht zu ändern (Regel (4)), wird immer ein rotes Blatt gelöscht.
- Ersetze den zu löschenden Knoten mit einem geeigneten Nachkommen und lösche anschließend diesen.
 - Das Löschen eines Knoten wird so auf das sukzessive Löschen einer Folge von Knoten zurückgeführt, die immer tiefer im Baum liegen.
 - Letztendlich wird ein Blatt aus dem Baum gelöscht.
- Damit das zu löschende Blatt rot ist, wird während des Abstiegs die rote Farbe im Baum nach unten propagiert.
 - Dabei auftretende **red-red-violations** (Regel (3)) werden durch Farbwechsel nach unten und Rotationen aufgelöst.

Löschen eines Knotens

- Es wird immer ein rotes Blatt gelöscht.
- Suche den zu löschenden Knoten K durch iterativen top-down-Durchlauf und behandle 3 unterschiedliche Fälle:
 - (1) Hat der zu löschende Knoten K zwei Kinder, wird er durch den kleinsten Knoten $K_{\min}$ im rechten Teilbaum ersetzt.
 - ▶ $K_{\min}$ wird als nächstes gelöscht (Fall (2))
 - (2) Hat der zu löschende Knoten K genau ein rechtes Kind, wird er durch den kleinsten Knoten $K_{\min}$ im rechten Teilbaum ersetzt.
 - ▶ $K_{\min}$ wird als nächstes gelöscht.



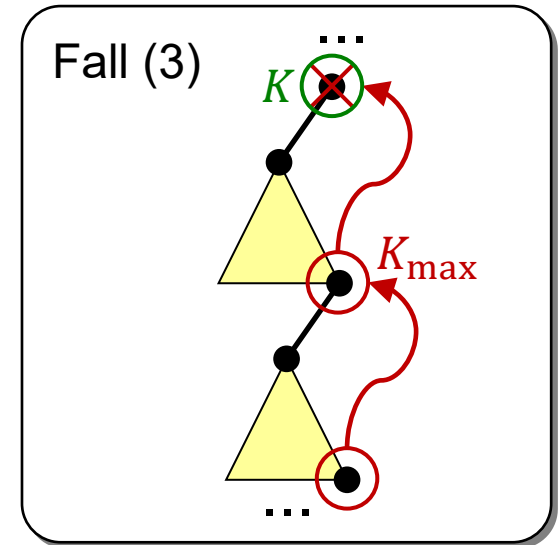
§4.5 Rot-Schwarz-Bäume

Algorithmen

Löschen eines Knotens (cont.)

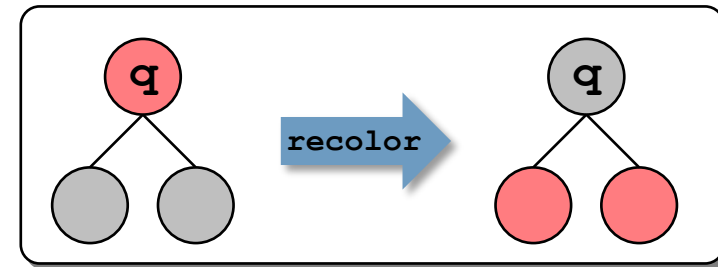
(3) Hat der zu löschende Knoten K genau ein linkes Kind, wird er durch den größten Knoten $K_{\max}$ im linken Teilbaum ersetzt.

► $K_{\max}$ wird als nächstes gelöscht.



Löschen eines Knotens (cont.)

- Das zu löschende Blatt muss rot sein.
 - Dann kann das Blatt einfach ausgehängt werden.
- Dazu wird beim iterativen top-down Durchlauf gewährleistet, dass der aktuelle Knoten q (außer der Wurzel) immer rot gefärbt wird.
- Dazu werden **Farbwechsel** und **Rotationsoperationen** durchgeführt.
 - **Farbwechsel (reColorDown)**
 - ▶ **Red-Red-Violation:** Dies kann zu zwei aufeinanderfolgenden roten Knoten führen von Kindern und Enkeln von q !
 - **Rotationsoperationen:**
 - ▶ 2x5 Fälle und zusätzlich 6 separate Fälle bei der Wurzel...
- Näheres siehe [Weiss 2011].



§4.5 Rot-Schwarz-Bäume

Vergleich zu AVL-Baum

Vergleich der Ausführungszeiten

Datenstruktur	N = 10 ⁶					N = 10 ⁷				
	N*insert	N*search	min	d _{av}	h	N*insert	N*search	min	d _{av}	h
Rot-Schwarz-Baum	0.68 sec	0.71 sec	15.0	18.4	24.4	10.66 sec	9.05 sec	18.0	21.8	28.8
AVL-Baum	0.81 sec	0.68 sec	14.5	18.3	23.0	12.93 sec	7.97 sec	17.0	21.7	27.0
Java TreeMap	0.66 sec	0.74 sec	n.a.	n.a.	n.a.	10.34 sec	8.98 sec	n.a.	n.a.	n.a.

- Zeiten sind über 10 Versuche gemittelt.
- N*insert: Es wurden N Zufallszahlen in einen leeren Baum eingefügt.
- N*search: Es wurden N Zufallszahlen in einem bereits mit N Zahlen gefüllten Baum gesucht (mit großer Wahrscheinlichkeit nicht erfolgreiche Suche!).
- min: Länge eines kürzesten Pfads von der Wurzel zu einem Blatt
- d_{av}: durchschnittliche Tiefe aller Knoten im Baum
- h: Höhe des Baums.

Inhalt der Vorlesung

1. Problemstellung und Motivation
2. Binäre Bäume
3. Binäre Suchbäume
4. AVL-Bäume
5. Rot-Schwarz-Bäume
6. Heaps
 - a. Binäre Heaps
 - b. Heap-Sort
 - c. Index-Heaps
7. B-Bäume*

* nicht prüfungsrelevant

§4.6 Heaps

- In einem binären Suchbaum gilt für einen Knoten **k** die Beziehung:

$$\mathbf{k.left.key < k.key < k.right.key.}$$

Diese Eigenschaft ermöglicht eine schnelle Suche nach einem bestimmten Schlüssel.

- Sucht man jedoch den maximalen (minimalen) Schlüssel, ist für einen Knoten **k** die folgende Beziehung besser:

$$\mathbf{k.key > k.left.key, k.right.key.}$$

Diese Eigenschaft heißt **(Max)-Heap-Eigenschaft**.

- ▶ Die Heap-Eigenschaft hat zur Folge, dass der Knoten mit dem größten (kleinsten) Schlüssel in der Wurzel steht.
- ▶ Dieser Schlüssel wird oft als Priorität interpretiert.
 - ▶ Alternative Namen: Priority queue, Prioritätsliste

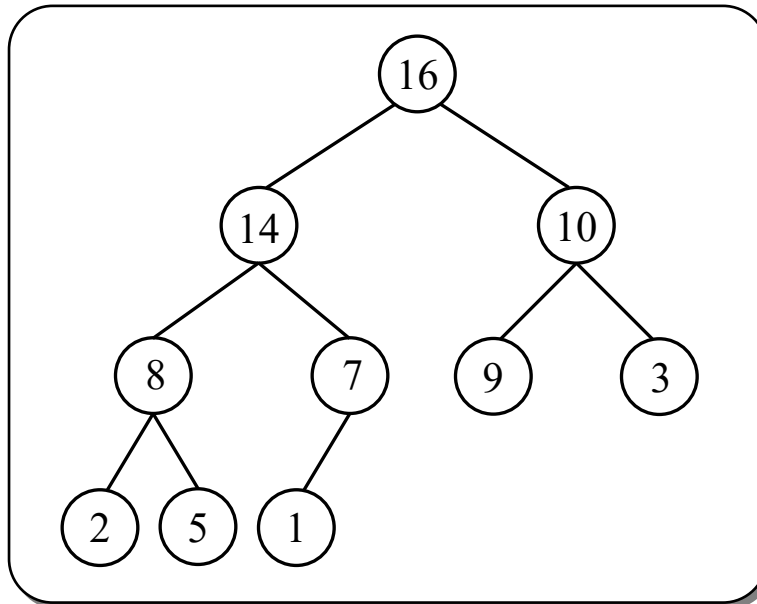
§4.6.a Binäre Heaps

Definition Ein binärer Baum,

1. in dem für jeden Knoten die (Max)-Heap-Eigenschaft erfüllt ist,
2. bei dem alle Schichten bis auf die letzte vollständig aufgefüllt sind und
3. die letzte Schicht linksbündig aufgefüllt ist,

heißt **(Max)-Heap** (dt.: Halde, Haufen).

Beispiel:



§4.6.a Binäre Heaps

Eigenschaften

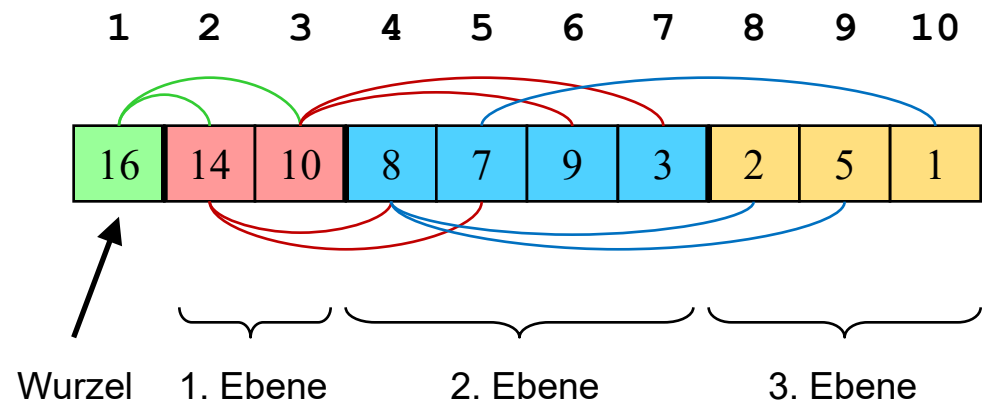
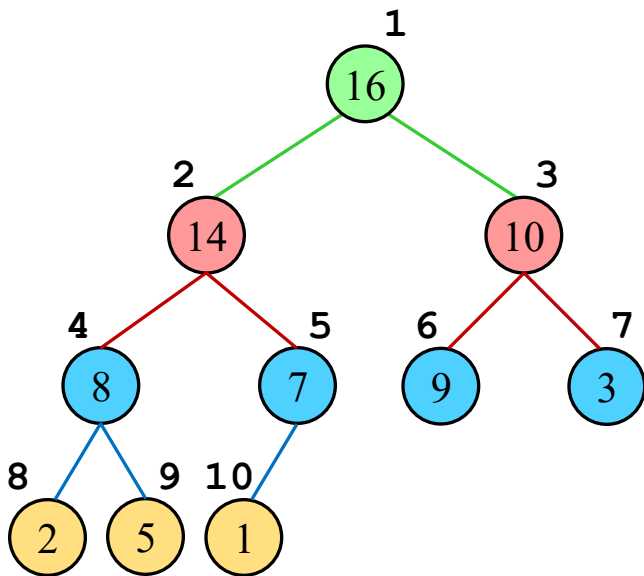
- In der Wurzel eines Max-Heaps steht der Knoten mit dem größten Schlüssel.
- In der Wurzel eines Min-Heaps steht der Knoten mit dem kleinsten Schlüssel.
- Weil ein Heap ein vollständiger binärer Baum ist, ist die Höhe eines Heaps mit n Einträgen $\Theta(\log n)$.

§4.6.a Binäre Heaps

Bemerkung

- Ein Heap kann auch mit einem Array $a[1..n]$ in-situ realisiert werden (siehe Übung).
- Die Heap-Eigenschaft hat dann die Form

$$a[i].key > a[2i].key, a[2i+1].key$$



Konstruktion eines Heap

- Ordne die n Datensätze in einem Array $\mathbf{a}[1..n]$ beliebig an.
 - Das entspricht einer beliebigen Anordnung in einem binären Baum mit den Eigenschaften 2. und 3.
- Anschließend muss noch die Heap-Eigenschaft hergestellt werden.
 - Jedes Blatt ist bereits ein einelementiger Heap.
 - Das Teilfeld $\mathbf{a}[(\lfloor n/2 \rfloor + 1) .. n]$ enthält nur Blätter.
 - Die Heap-Eigenschaft kann dann mit einer Hilfsfunktion **heapify** von den Blättern zur Wurzel (bottom-up) hergestellt werden.
- ...

§4.6.a Binäre Heaps

Konstruktion

- ...
- Anschließend muss noch die Heap-Eigenschaft hergestellt werden.
 - Jedes Blatt ist bereits ein einelementiger Heap.
 - Das Teilfeld $a[\lfloor n/2 \rfloor + 1 \dots n]$ enthält nur Blätter.
 - Die Heap-Eigenschaft kann dann mit einer Hilfsfunktion **heapifyDown** von den Blättern zur Wurzel (bottom-up) hergestellt werden.

```
void build_heap(Node a[])
{
    for (int i =  $\lfloor \text{length}(a) / 2 \rfloor$ ; i > 0; i--) {
        heapifyDown(a, i);
    }
}
```

§4.6.a Binäre Heaps

Konstruktion

- Die Hilfsfunktion **heapifyNN** arbeitet stellt lokal die Heap-Eigenschaft (wieder) her.
 - Sie kann von einem Blatt zur Wurzel aufsteigend (aka **heapifyUp**) oder von einem inneren Knoten zu einem Blatt absteigend (aka **heapifyDown**) verwendet werden.
 - **heapifyDown**
 - An einem zu bearbeitenden Knoten haben beide Kinder bereits die Heap-Eigenschaft.
 - Ist der Schlüssel größer als in beiden Kindknoten, ist die Heap-Eigenschaft erfüllt.
 - Anderenfalls muss der aktuelle Knoten
 - a. mit dem Kind mit dem größeren Schlüssel vertauscht werden und
 - b. in dem zugehörigen Teilbaum absinken, bis beide Nachfolger kleinere Schlüssel haben.

§4.6.a Binäre Heaps

Konstruktion

heapifyDown

```
void heapifyDown(Node a[], int i)
{
    int max = i;
    int li = 2*i;
    int re = li+1;
    if (a[li].key > a[max].key) max = li;
    if (a[re].key > a[max].key) max = re;
    if (i != max) {
        swap(a[i], a[max]);
        heapifyDown(a, max);
    }
}
```

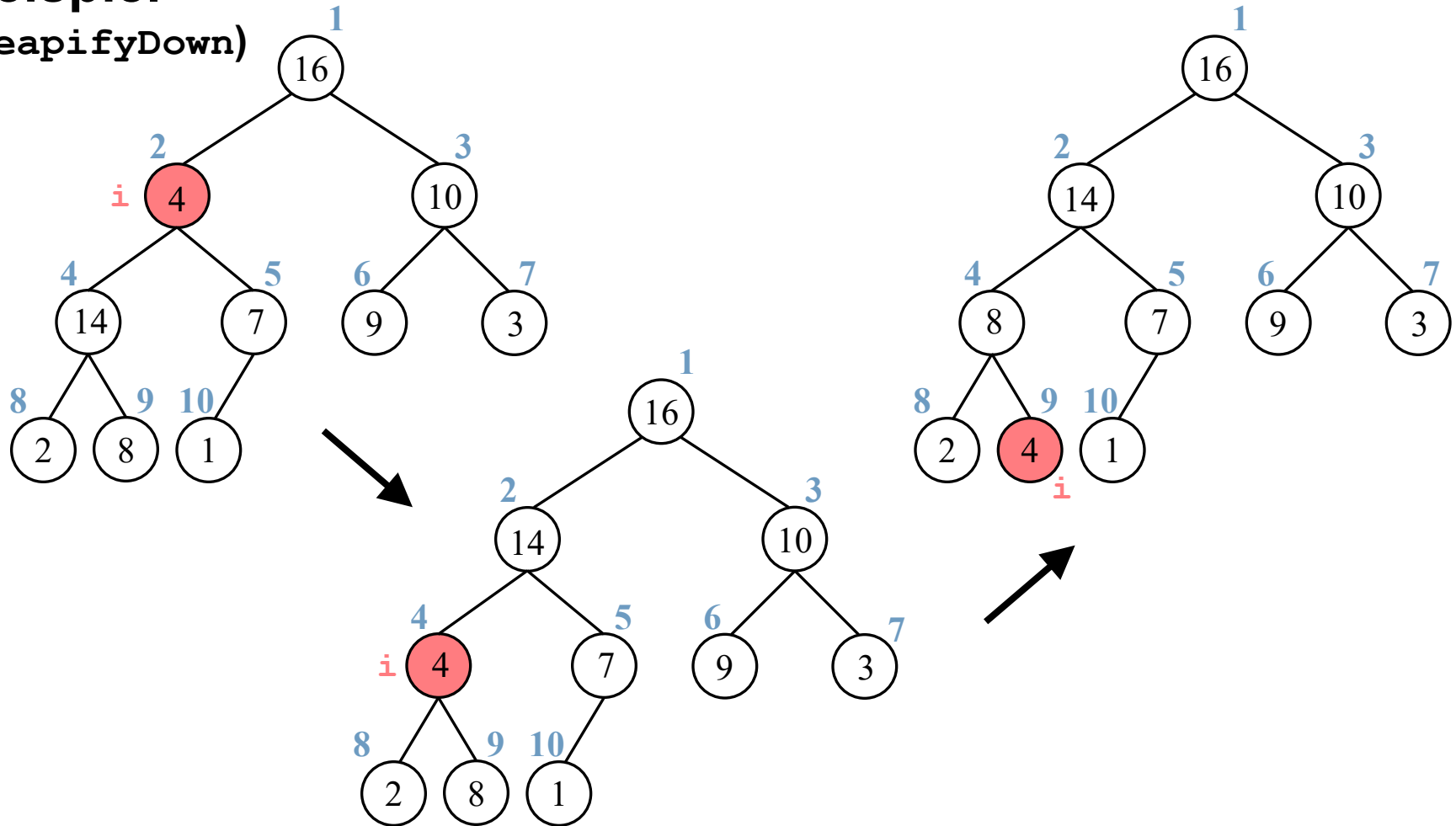
// Achtung: Hier müsste
noch getestet werden,
ob li und re gültige
Indizes sind!

§4.6.a Binäre Heaps

Konstruktion

Beispiel

(heapifyDown)

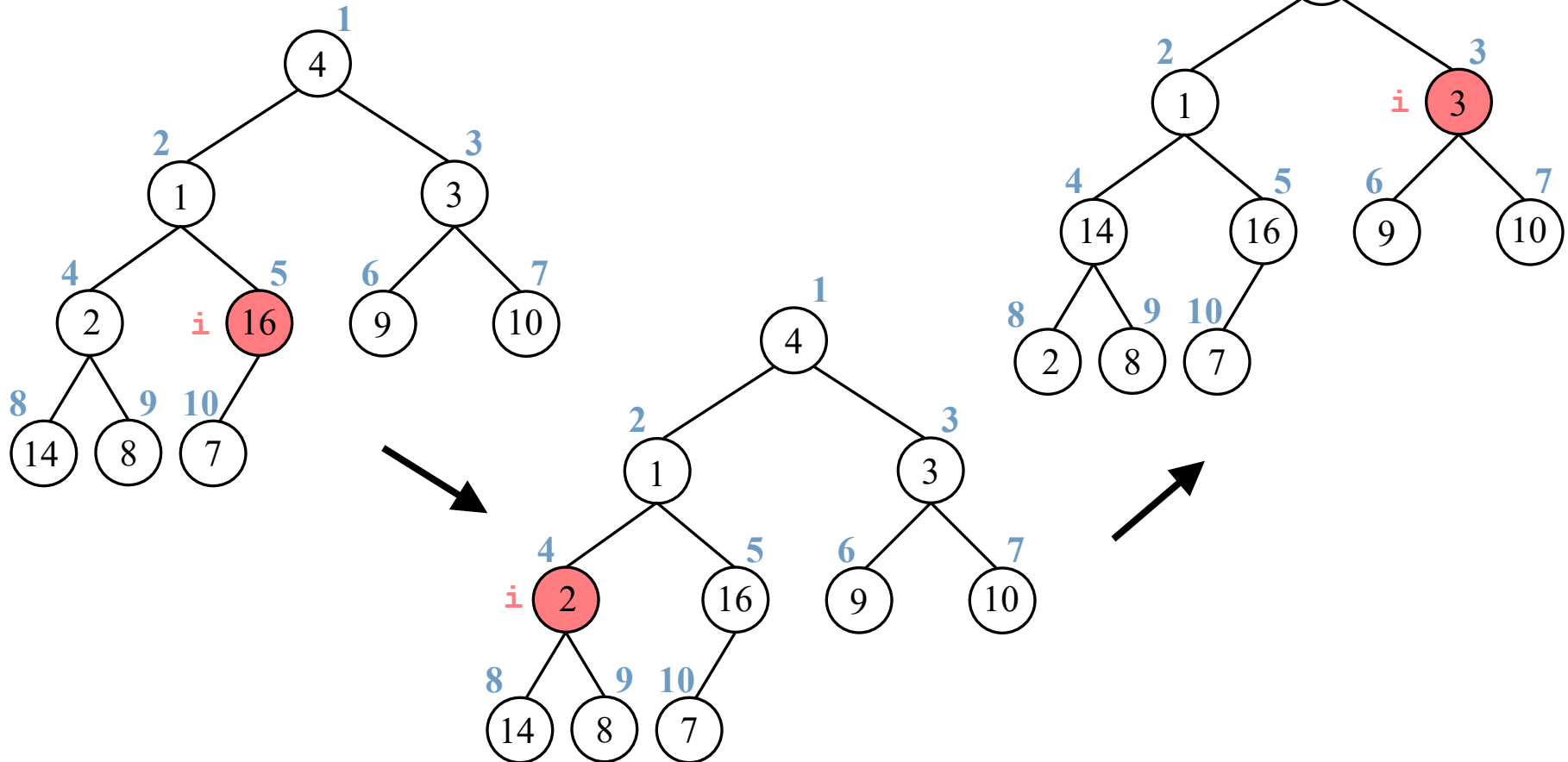


§4.6.a Binäre Heaps

Konstruktion

Beispiel (build_heap)

4	1	3	2	16	9	10	14	8	7
---	---	---	---	----	---	----	----	---	---

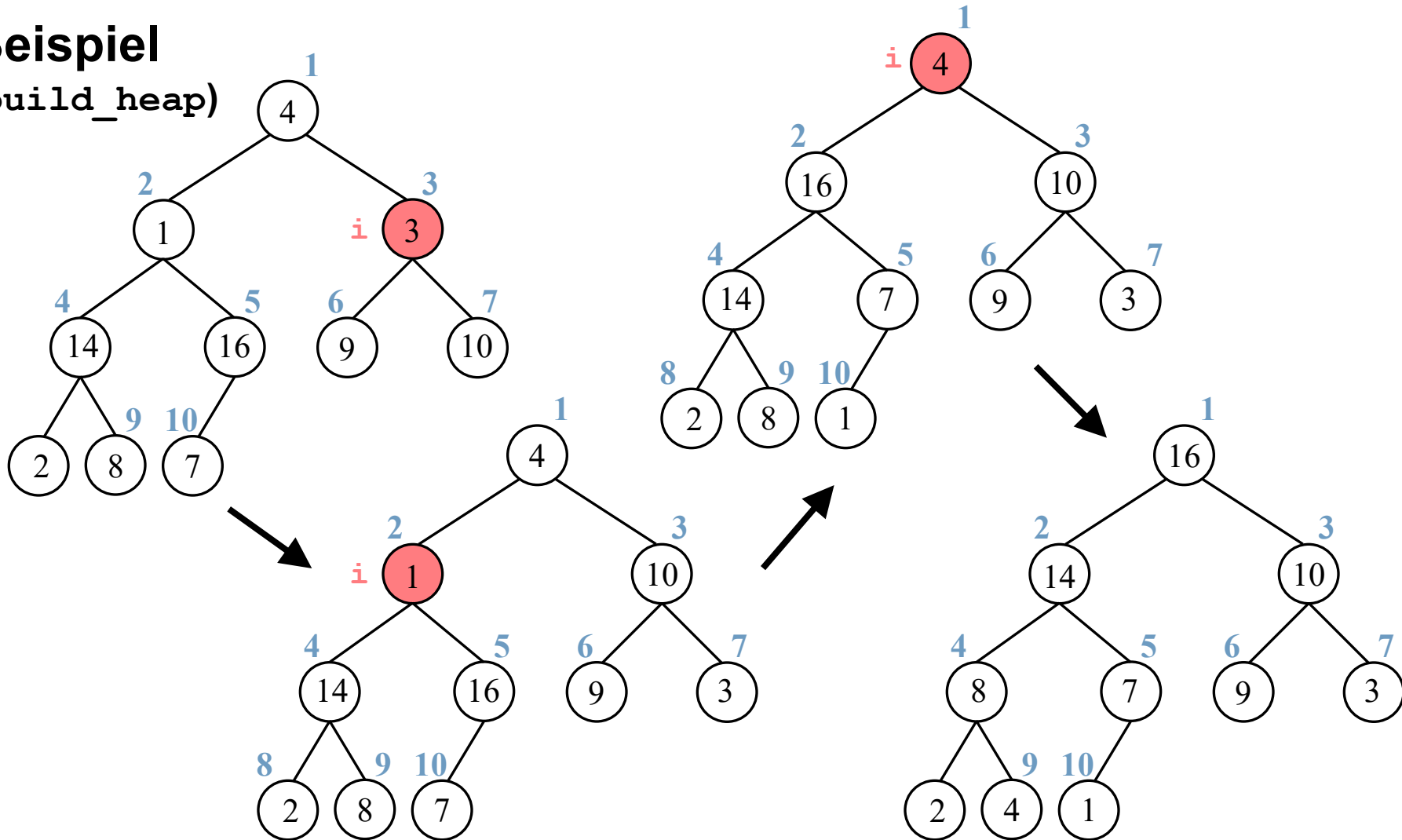


§4.6.a Binäre Heaps

Konstruktion

Beispiel

(build_heap)



Laufzeit der Konstruktion (**heapify**)

- Laufzeit von **heapifyDown** auf einem Teilbaum der Größe n .
 - Bestimmung der Beziehungen von $a[i]$, $a[le]$, $a[re]$: $\Theta(1)$.
 - Ausführung von **heapifyDown** auf einem Teilbaum.
 - Ein Teilbaum hat maximal die Größe $2n/3$, falls die letzte Ebene des Baumes genau halb gefüllt ist.
 - Laufzeit von **heapifyDown**: $T(n) \leq T(2n/3) + \Theta(1)$.
 - ▶ $T(n) = O(\log n)$.
- Alternativ: Laufzeit von **heapifyDown** für einen Knoten der Höhe h : $O(h)$.

Laufzeit der Konstruktion (`build_heap`)

- Laufzeit von `build_heap` für ein Feld der Größe n .
 - Obere Schranke: $O(n)$ Aufrufe von `heapifyDown`
 - ▶ $O(n \log n)$.
 - Schärfere Schranke: Die Laufzeit von `heapifyDown` variiert mit der Höhe des Teilbaums und ist meist klein.
 - Ein n -elementiger Heap hat die Höhe $\lfloor \log n \rfloor$.
 - Es gibt höchstens $\lfloor n/2^{h+1} \rfloor$ Knoten der Höhe h .
 - Gesamtlaufzeit:

$$\sum_{h=0}^{\lfloor \log n \rfloor} \left\lfloor \frac{n}{2^{h+1}} \right\rfloor O(h) = O\left(n \sum_{h=0}^{\lfloor \log n \rfloor} \frac{h}{2^{h+1}}\right) \subseteq O\left(n \underbrace{\sum_{h=0}^{\infty} \frac{h}{2^{h+1}}}_{=2}\right) = O(n).$$

Einfügen eines Elements in einen binären Heap

- (1) Ein neues Element wird an das Ende des Heap gespeichert.
 - ▶ Im allgemeinen ist dann die Heap-Eigenschaft verletzt.
- (2) Um die Heap-Ordnung wieder herzustellen, wird das neue Element nach oben verschoben mit **heapifyUp**.

```
void insert(Node x) { // Indizierung 1..n
    a[++n] = x;        // evtl. enlarge dyn. array
    heapifyUp(n);
}
```

```
void heapifyUp(int k) { // max-heap
    if (k > 0 && a[k/2] < a[k]) {
        swap(k, k/2);
        heapifyUp(k/2);
    }
}
```

▶ Laufzeit: $O(\log n)$.

Entfernen des Maximums aus einem max-Heap

- (1) Ersetze das erste Element durch das letzten Element **n** im Heap,
 - ▶ Im allgemeinen ist dann die Heap-Eigenschaft verletzt.
- (2) Heap verkürzen durch **n--** und
- (3) verschiebe das erste Element mit **heapifyDown** nach unten.

```
void removeMax() {  
    a[1] = a[n--]; // evtl. shrink dyn. array  
    heapifyDown(1);  
}
```

▶ Laufzeit: $O(\log n)$.

Zusammenfassung für binäre Heaps

- Aufbauen eines Heap: $O(n)$.
- Finden des Maximums/Minimums: $O(1)$.
- Entfernen des Maximums/Minimums: $O(\log n)$.
- Einfügen eines neuen Elements: $O(\log n)$.

Anwendung?

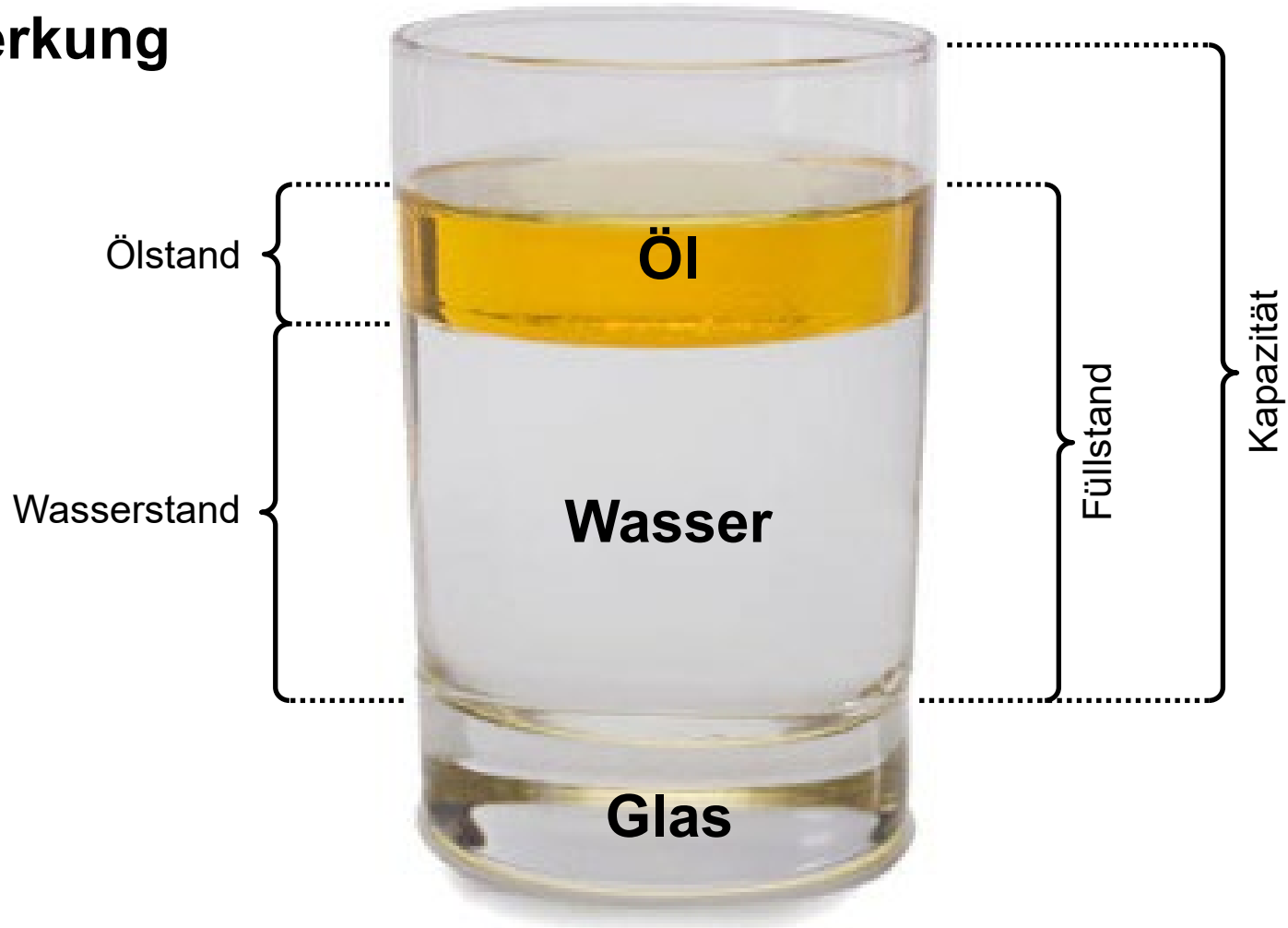
- Heap-Sort: $O(n \log n)$.

Wünsche?

- Ändern eines Prioritätswerts: $O(n)$.
- Löschen eines beliebigen Elements: $O(n)$.

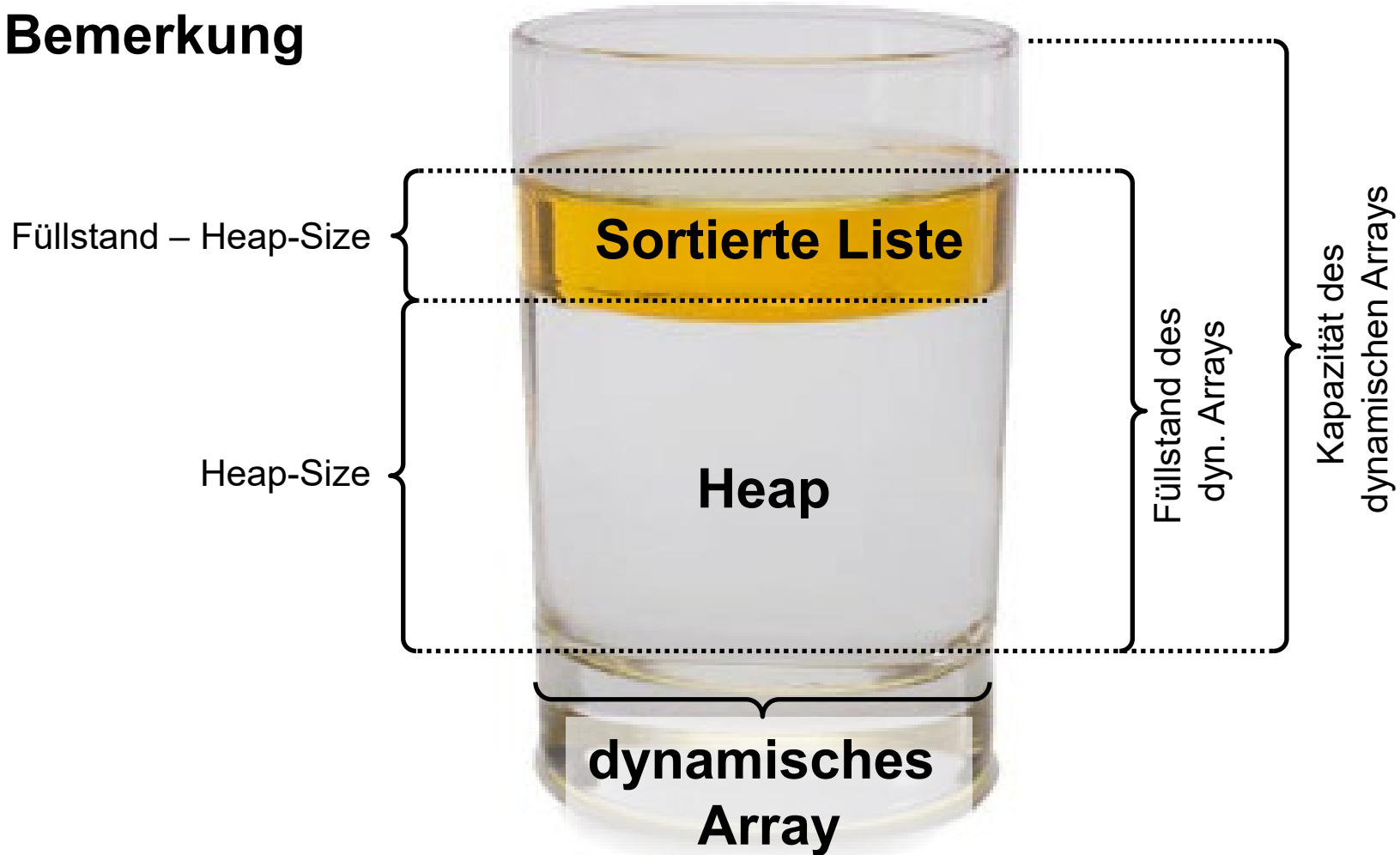
§4.6.b Heap-Sort

Bemerkung



§4.6.b Heap-Sort

Bemerkung



§4.6.b Heap-Sort

Idee (Heap-Sort)

1. Erzeuge ein Heap aus dem zu sortierenden Feld.
2. Vertausche das maximale Element mit dem letzten Element (Selectionsort).
3. Verkleinere den Heap und stelle die Heap-Eigenschaft wieder her (**heapify**).
4. Wiederhole 2. & 3.



```
void heap_sort(Node a[])
{
    build_heap(a);
    for (int i=length(a); i>=2; i--) {
        swap(a[1],a[i]);
        decrease_heap_size(a);
        heapifyDown(a,1);
    }
}
```

§4.6.b Heap-Sort

Laufzeit von Heap-Sort

- Die Laufzeit von Heap-Sort setzt sich zusammen aus
 - einem Aufruf von **build_heap** mit Laufzeit von $O(n)$
 - und $n - 1$ Aufrufen von **heapifyDown** mit einer Laufzeit von $O(\log n)$.
 - Gesamtlaufzeit: $O(n \log n)$.

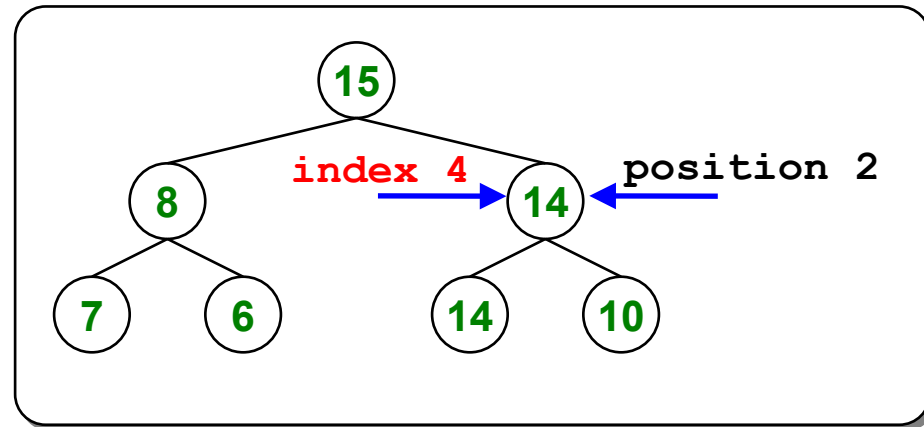
Probleme von binären Heaps

- Elemente haben neben der Priorität auch einen Index aus dem ursprünglichen Feld, wo ggf. auch die Nutzdaten liegen.
- Wie ändert man die Priorität, wenn sie die Nutzdaten ändern, ohne den gesamten Heap zu durchsuchen?
- Wie entfernt man ein Element aus dem Heap, das nicht das Maximum ist, ohne den gesamten Heap zu durchsuchen?
- Dabei sollen Aufbau, Einfügen und Maximum-Entfernen weiterhin effizient beibehalten werden.

§4.6.c Index-Heaps

Probleme

- Das Element mit **index 4** hat den **priority-Wert 14** und steht im **HEAP-Feld** an der **position 2**.



HEAP	position p	0	1	2	3	4	5	6
	heap[p]	15	8	14	7	6	14	10

PRIO	index i	0	1	2	3	4	5	6
	priority[i]	8	14	15	6	14	10	7

- Ein einfaches **HEAP-Feld** enthält direkt die **priority-Wert**.
 - Das **PRIO-Feld** ordnet jedem **index** den zugehörigen **priority-Wert** zu.
- Wie kann für das Element mit **index 4** der **priority-Wert** effizient von **14** auf **20** erhöht werden.
 - ▶ Element muss im **HEAP-Feld** gefunden werden!

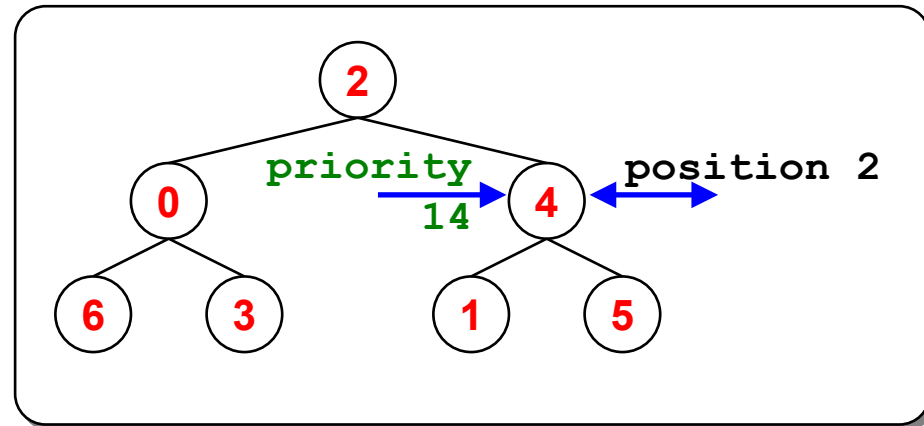
§4.6.c Index-Heaps

Lösung

- Das **HEAP**-Feld enthält den **index** der Elemente statt der Prioritäten.
- Es wird zusätzlich ein **POS**-Feld benötigt, das für jeden **index** die **position** im Heap angibt.

HEAP	position p	0	1	2	3	4	5	6
	heap[p]	2	0	4	6	3	1	5

PRIO & POS	index i	0	1	2	3	4	5	6
	priority[i]	8	14	15	6	14	10	7
	position[i]	1	5	0	4	2	6	3



- **HEAP**-Feld enthält **index** anstatt der **priority**-Wert.
 - Das **PRIO**-Feld ordnet jedem **index** den zugehörigen **priority**-Wert zu.
 - Das **POS**-Feld ordnet jedem **index** die **position** im **HEAP**-Feld zu.
- Zugriff auf die **priority**-Wert: **priority[heap[p]]**
 - Es gilt: **position[heap[p]] = p** und **heap[position[i]] = i**

Einfügen eines Elements in einen Index-Heap

- Notwendige Anpassungen

```
void insert(index i, priority p) {  
    if (position[i] != -1) return;  
    heap    [++n] = i;  
    priority[ i] = p;  
    position[ i] = n;  
    heapifyUp(n) ;  
}
```

► Laufzeit: $O(\log n)$.

```
void heapifyUp(position k) {  
    index i = heap[k];  
    if (k > 0 && priority[heap[k/2]] < priority[i]) {  
        heap[k] = heap[k/2]; position[heap[k]] = k;  
        heap[k/2] = i ; position[ i ] = k/2;  
        heapifyUp(k/2) ;  
    }  
}
```

Ändern der Priorität eines Elements in einem Index-Heap

```
void change(index i, priority p) {  
    if (position[i] == -1) return; // Element nicht gefunden  
    priority[i] = p;  
    if (Priorität gestiegen) heapifyUp (position[i]);  
    else                      heapifyDown(position[i]);  
}
```

► Laufzeit: $O(\log n)$.

Löschen eines Elements aus einem Index-Heap

```
void remove(index i) {
    if (position[i] == -1) return; // Element nicht gefunden
    oldPos          = position[i];
    oldPrio         = priority[i];
    position        [    i] = -1; // Element aus heap gelöscht
    heap            [oldPos] = heap[n];
    position[heap[oldPos]] = oldPos;
    newPrio         = priority[heap[oldPos]];
    n--;            // Heap verkleinert
    if (oldPrio < newPrio) heapifyUp  (oldPos);
    else             heapifyDown(oldPos);
}
```

► Laufzeit: $O(\log n)$.

Zusammenfassung für Index-Heaps

- Aufbauen eines Heap: $O(n)$.
- Finden des Maximums/Minimums: $O(1)$.
- Entfernen des Maximums/Minimums: $O(\log n)$.
- Einfügen eines neuen Elements: $O(\log n)$.
- Ändern eines Prioritätswerts: $O(\log n)$.
- Löschen eines beliebigen Elements: $O(\log n)$.

Inhalt der Vorlesung

1. Problemstellung und Motivation
2. Binäre Bäume
3. Binäre Suchbäume
4. AVL-Bäume
5. Rot-Schwarz-Bäume
6. Heaps
7. **B-Bäume***

* nicht prüfungsrelevant

Motivation

- Bisher wurde angenommen, dass die gesamte Datenstruktur des Baumes im Hauptspeicher Platz findet.
- Wird ein Teil oder die gesamte Datenstruktur auf der Festplatte gespeichert, ist die bisherige Laufzeitanalyse unzureichend, weil Plattenzugriffe viel länger dauern als Speicherzugriffe:
 - Ein Plattenzugriff dauert so lange wie mehrere 100.000 Speicherzugriffe.
 - **Beispiel:** Bei einem binären Suchbaum mit 10 Mio. Einträgen würde eine durchschnittliche Suche 32–100 Plattenzugriffe (ca. 5–16 sek) brauchen.
 - **Beispiel:** Bei einem AVL-Baum kommt man durchschnittlich nahe an $\log n$, also typischerweise 25–35 Plattenzugriffe (ca. 4–6 sek).
- **Ansatz:** Minimiere die Anzahl von Plattenzugriffen durch Bäume mit Knoten, die mehr als zwei Kinder haben können.

§4.7 B-Bäume*

Definition

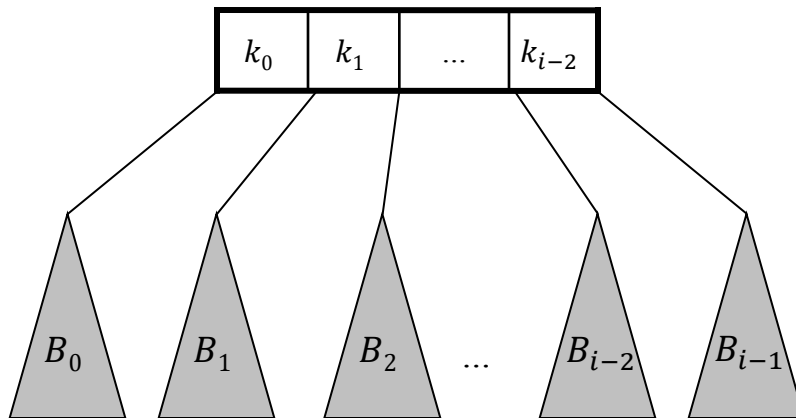
Ein **B-Baum der Ordnung m** ist ein Baum mit den folgenden Eigenschaften:

1. **Perfekte Balance:** Jedes Blatt hat die gleiche Tiefe.
2. **Anzahl Kinder:**
 - Die Wurzel hat zwischen 2 und m Kinder.
 - Alle anderen inneren Knoten haben zwischen $\lceil m/2 \rceil$ und m Kinder.
3. **Anzahl Schlüssel:**
 - Jeder Knoten mit i Kindern hat $i - 1$ Schlüssel.
 - Jedes Blatt hat zwischen $\lceil m/2 \rceil - 1$ und $m - 1$ Schlüssel.
4. **Schlüsselordnung:**
 - siehe nächste Seite...

§4.7 B-Bäume*

- 4. Schlüsselordnung:**
- In jedem Knoten sind die Schlüssel sortiert und
 - für jeden Knoten K mit i Kindern und $i-1$ Schlüsseln $k_0, k_1, \dots, k_{i-2}$ gilt die Beziehung:

Schlüssel in $B_0 < k_0 < \text{Schlüssel in } B_1 < k_1 < \dots$
 $< \text{Schlüssel in } B_{i-2} < k_{i-2} < \text{Schlüssel in } B_{i-1}.$

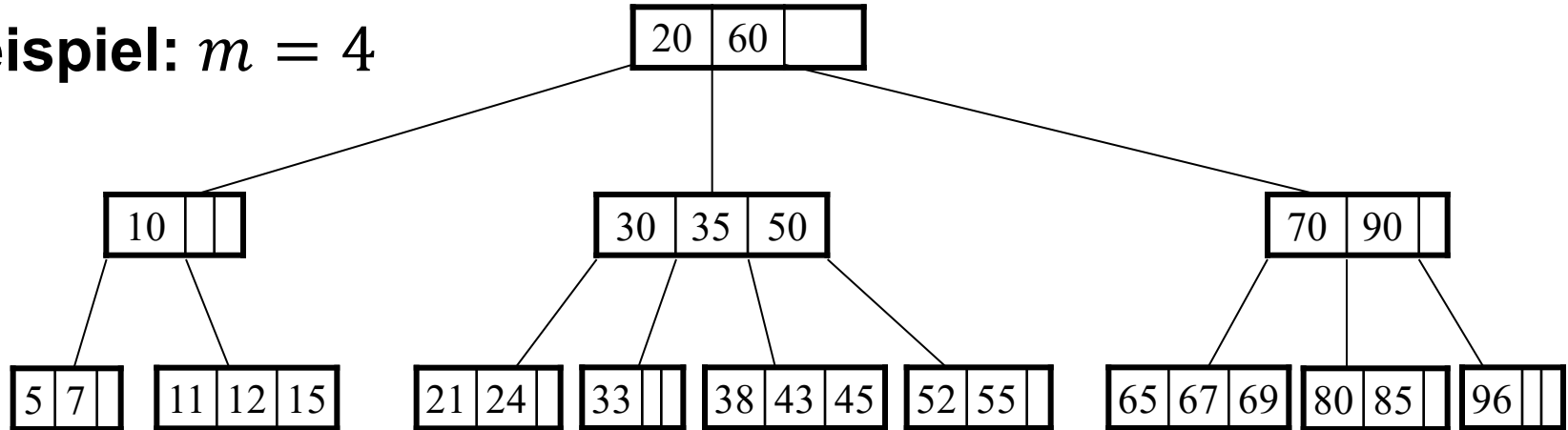


Knoten K mit $i-1$ Schlüsseln

i Kinder mit ihren Teilbäumen B_i

§4.7 B-Bäume*

Beispiel: $m = 4$



Vergleich mit der Definition:

- Der Baum ist perfekt balanciert: Jedes Blatt hat die gleiche Tiefe 2.
- Anzahl Kinder: Die Wurzel hat 3 Kinder, jeder andere innere Knoten hat zwischen $\lceil m/2 \rceil = 2$ und $m = 4$ Kinder.
- Anzahl Schlüssel: Jeder Knoten mit i Kindern hat $i - 1$ Schlüssel und die Blätter haben zwischen $\lceil m/2 \rceil - 1 = 1$ und $m - 1 = 3$ Schlüssel.
- Die Schlüsselordnung ist für jeden Knoten erfüllt.

§4.7 B-Bäume*

Algorithmen

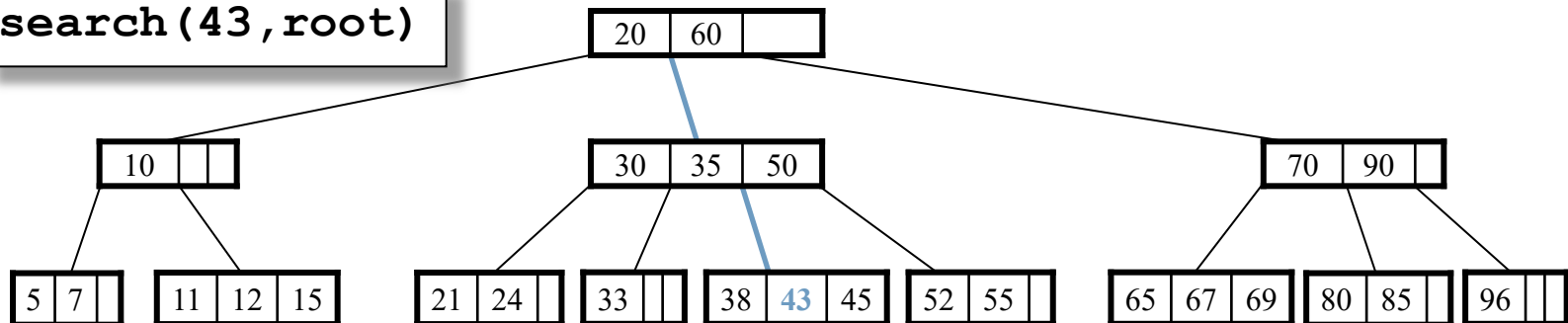
Suchen in B-Bäumen:

```
bool search(KeyType key, Node* p)
{
    if (p == 0)           return false; // key nicht gefunden

    Suche key in Schlüsselmenge von p mit binärer Suche;
    if (key gefunden) return true;

    return search(key, Teilbaum-in-dem-key-liegen-könnte);
}
```

Beispiel: `search(43, root)`



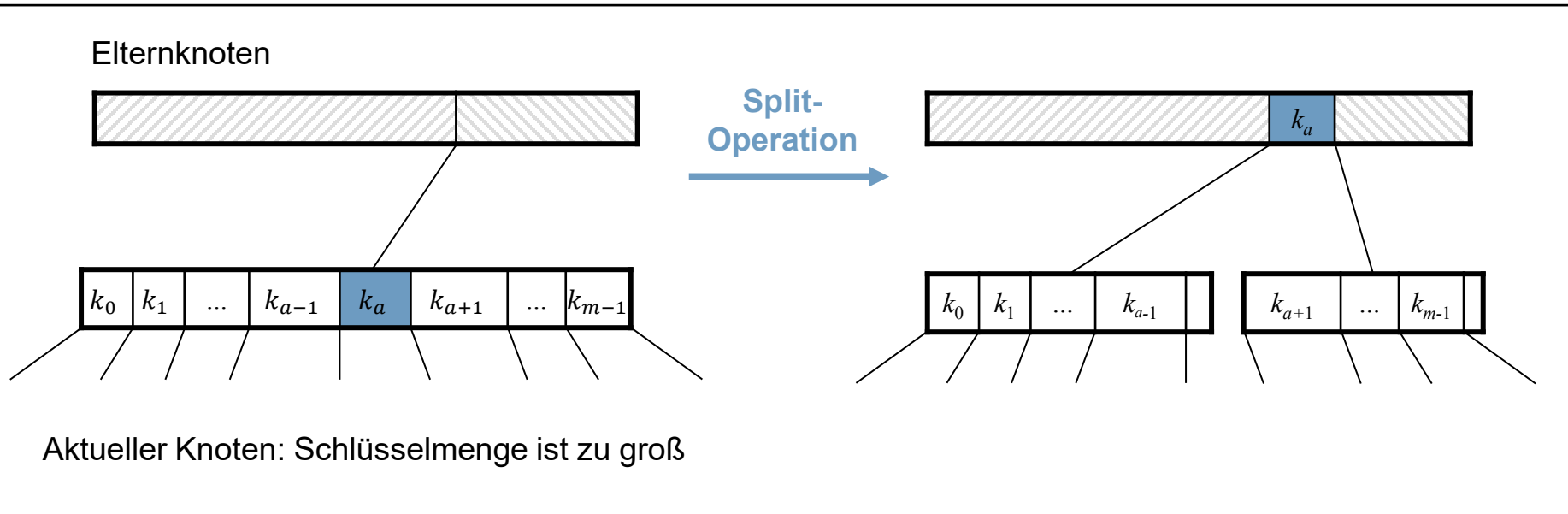
Einfügen in B-Bäumen:

```
bool insert(KeyType key, Node* p)
{
    if (search(key,p)) return false; // key vorhanden

    Füge key in dem Blatt ein, an dem die Suche endete;
    if (Schlüsselüberlauf) { // d.h. Anzahl Schlüssel == m
        Führe vom aktuellen Knoten bis zur Wurzel
        Split-Operationen durch, falls die jeweilige
        Schlüsselmenge zu groß ist.
        return true;
    }
}
```

Split-Operation:

Teile Schlüsselmenge an der Stelle $a = m/2$.



Bemerkungen:

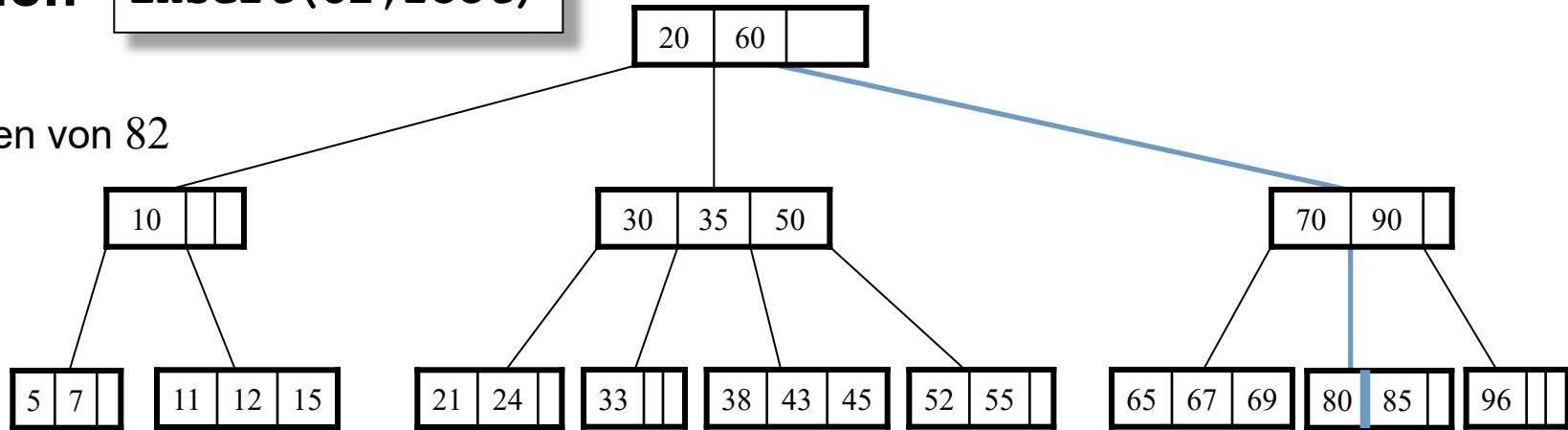
1. Es muss ggf. auch die Wurzel gesplittet werden.
 - ▶ Die Höhe des Baumes wächst um Eins.
2. Es ist in der Regel günstiger, jeden bei der Suche besuchten Knoten zu teilen, der maximal voll ist.
 - ▶ Man spart den rekursiven Aufstieg im Baum, um innere Knoten zu splitten.
 - ▶ Beim Einfügen muss höchstens noch das gefundene Blatt geteilt werden.

§4.7 B-Bäume*

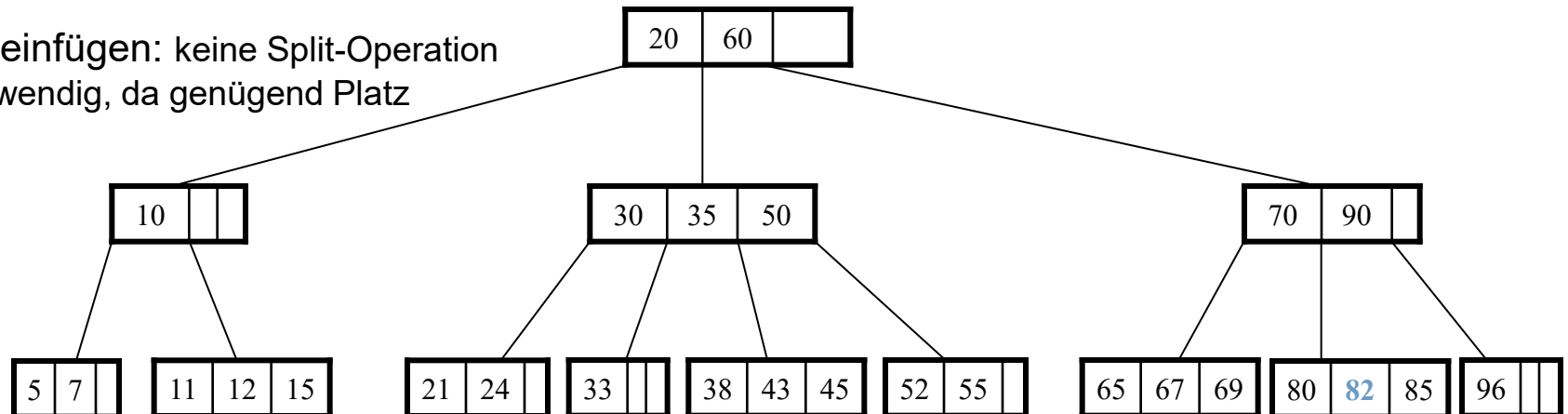
Algorithmen

Beispiel: `insert(82, root)`

1) Suchen von 82



2) 82 einfügen: keine Split-Operation
notwendig, da genügend Platz

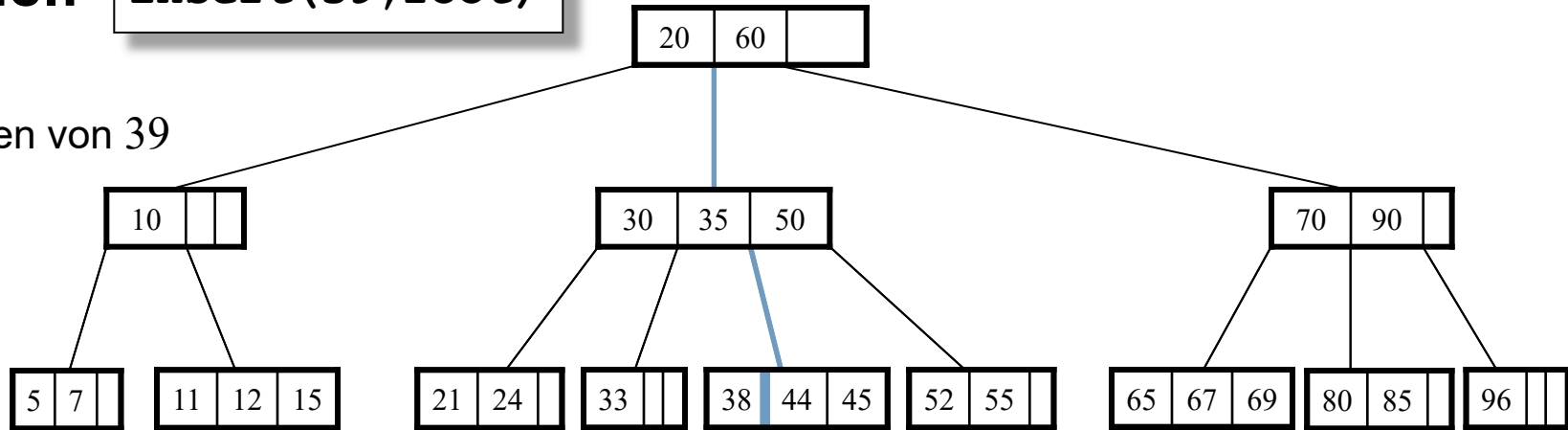


§4.7 B-Bäume*

Algorithmen

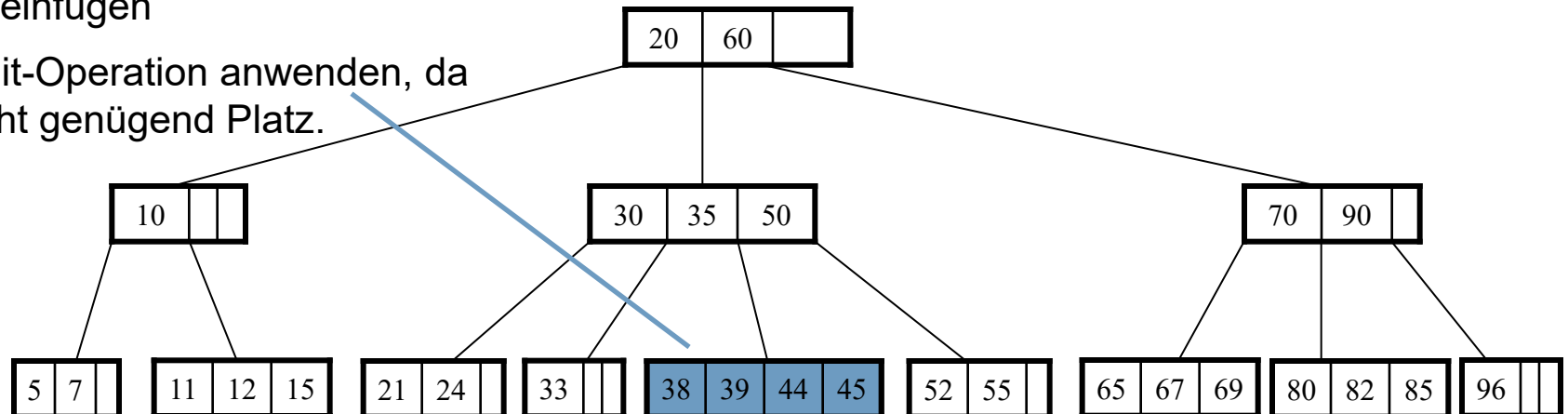
Beispiel: `insert(39, root)`

1) Suchen von 39



2) 39 einfügen

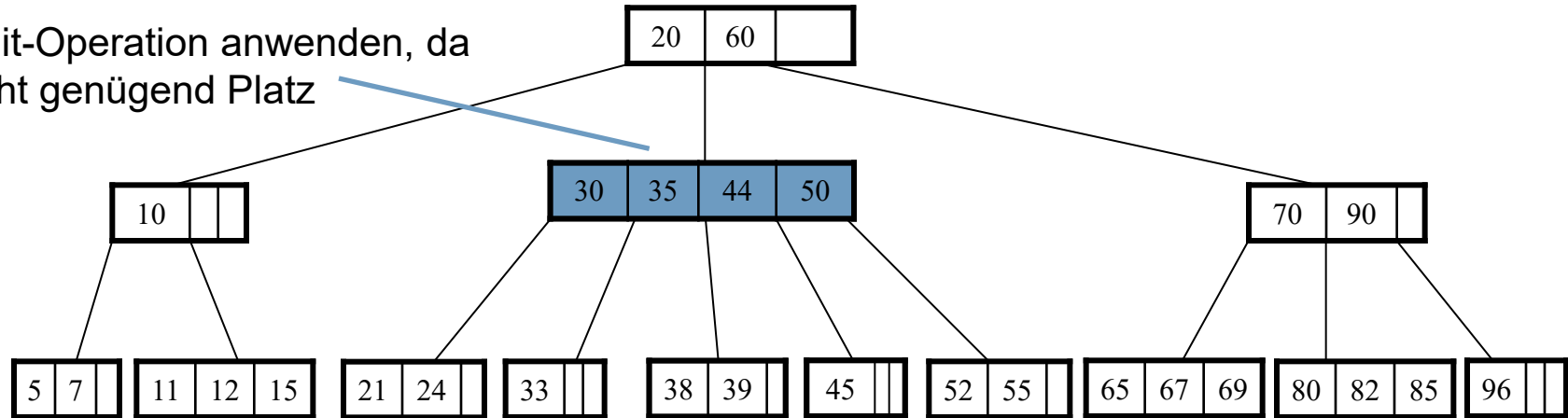
3) Split-Operation anwenden, da nicht genügend Platz.



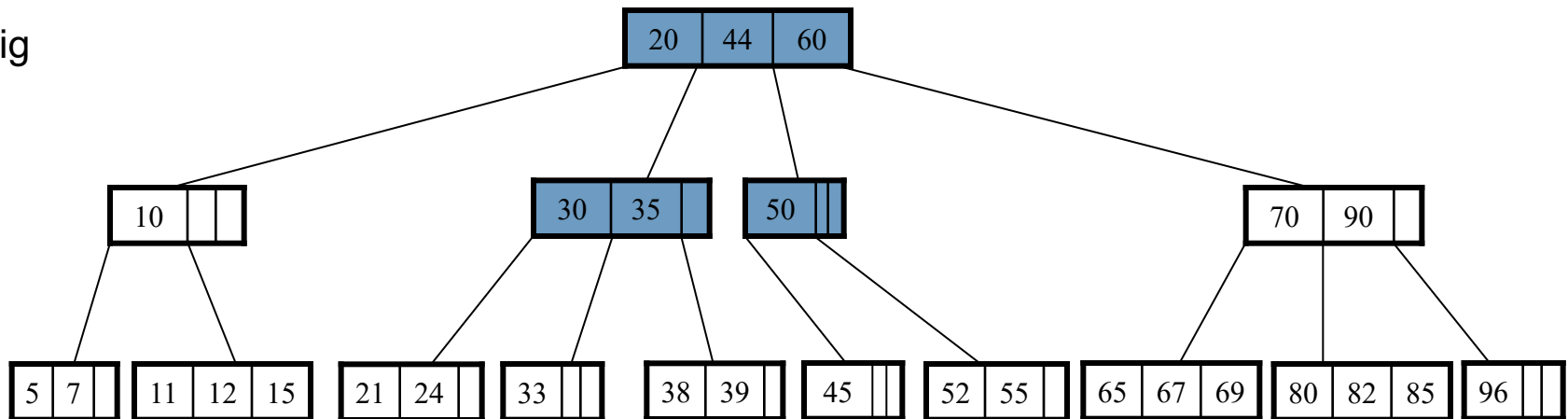
§4.7 B-Bäume*

Algorithmen

4) Split-Operation anwenden, da nicht genügend Platz



5) fertig



§4.7 B-Bäume*

Algorithmen

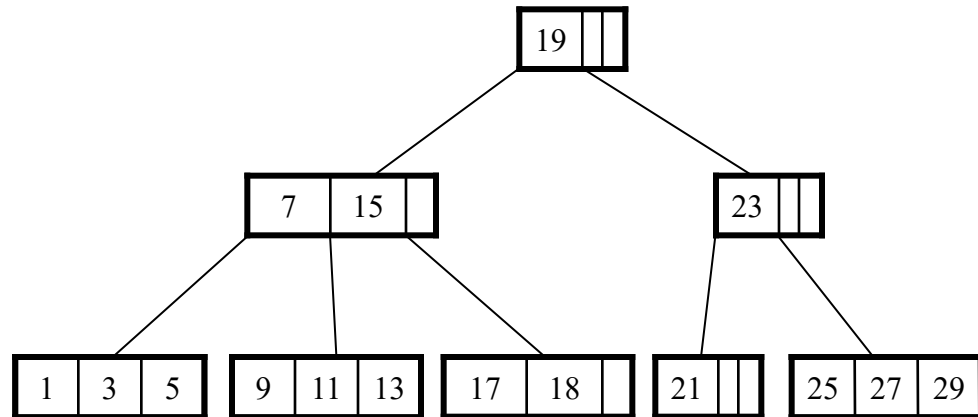
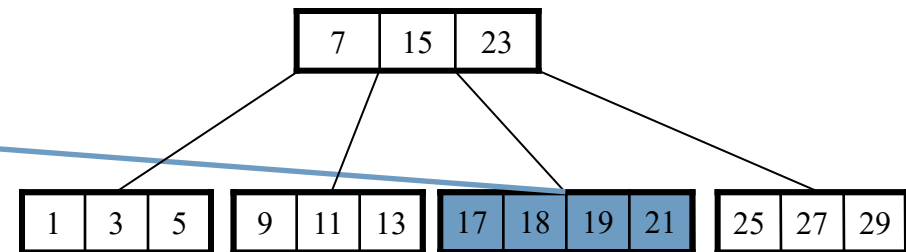
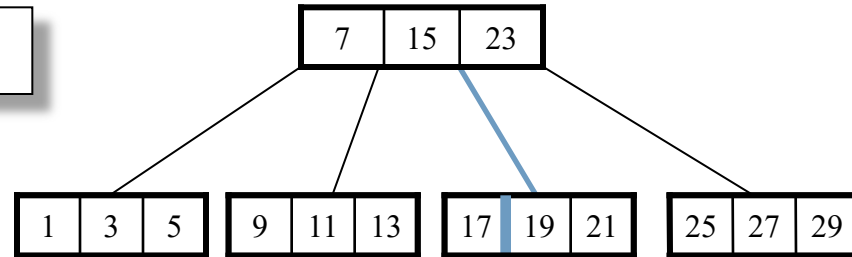
Beispiel: `insert(18, root)`

1) Suchen von 18

2) 18 einfügen

3) Split-Operation anwenden, da nicht genügend Platz.

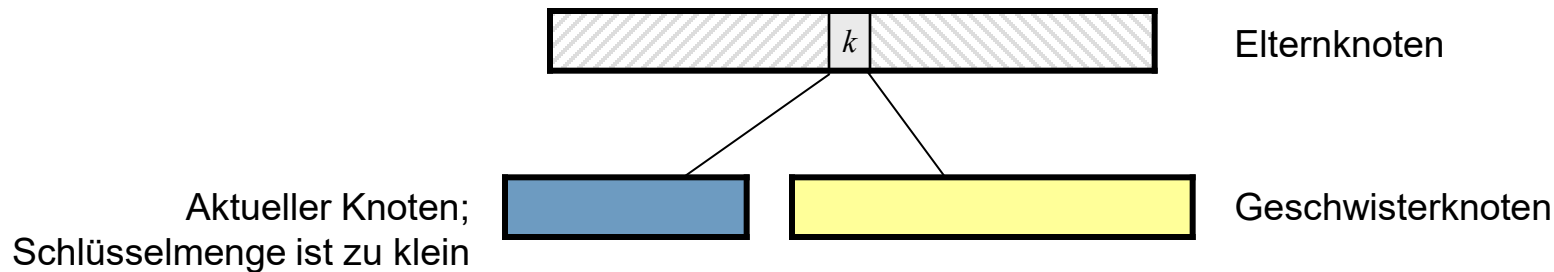
4) Wurzel splitten.



Löschen aus B-Bäumen:

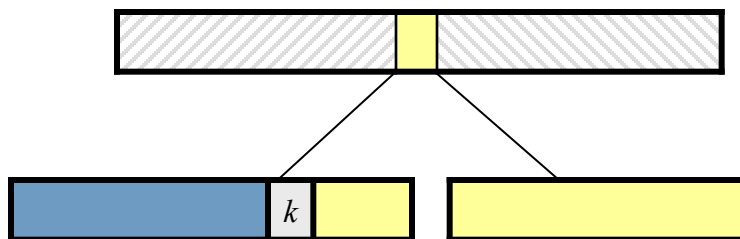
```
bool remove(KeyType key, Node* p)
{
    if (!search(key,p)) return false; // key nicht vorhanden
    if (key befindet sich in einem Blatt) lösche key;
    else Ersetze key durch nächst größeren Schlüssel key' aus
        dem linksten Blatt des nach key hängenden Teilbaums
        und lösche key';
    if (Schlüsselunterlauf) // d.h. Anzahl Schlüssel <  $\lceil m/2 \rceil - 1$ 
        // (0 bei Wurzel)
        Führe vom aktuellen Knoten bis zur Wurzel eine
        Merge-Operation mit einem Geschwisterknoten oder eine
        Übernahme von Schlüsseln von einem Geschwisterknoten
        durch, falls die jeweilige Schlüsselmenge zu klein ist.
    return true;
}
```

Behandlung des Schlüsselunterlaufs



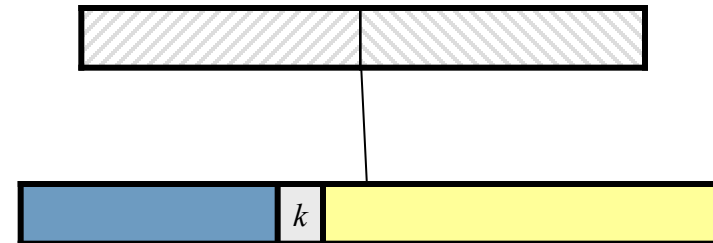
Übernahme von Schlüssel

Linker bzw. rechter Geschwisterknoten kann genügend Schlüssel abgeben.



Merge-Operation

Beide Geschwisterknoten haben zu wenig Schlüssel, dann fasse aktuellen Knoten mit linkem bzw. rechten Geschwisterknoten zusammen.



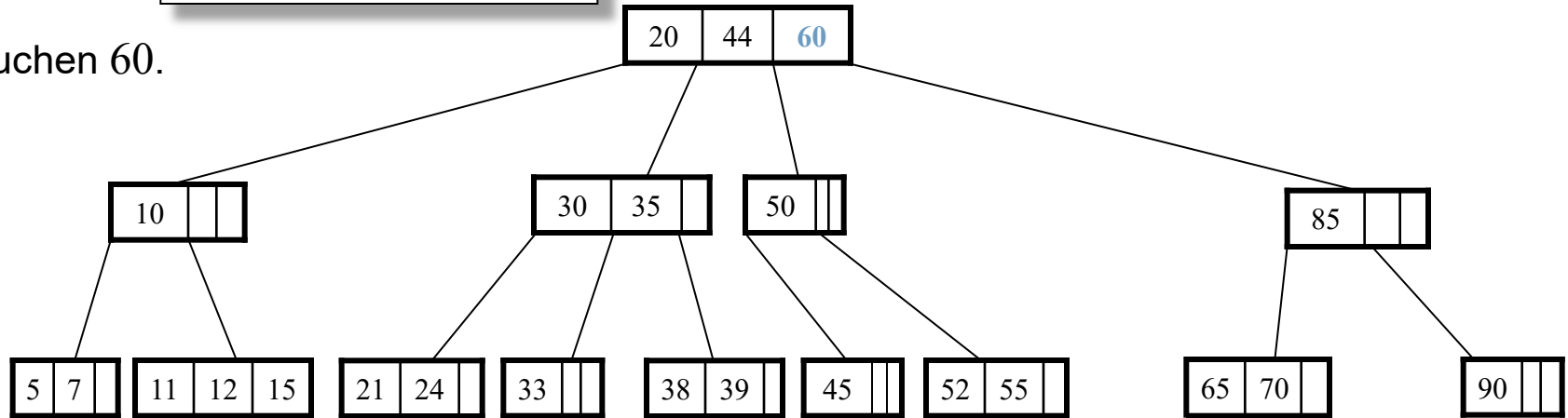
- a) Elternknoten ist um einen Schlüssel kleiner geworden.
- b) Wende Unterlaufbehandlung auf Elternknoten an.

§4.7 B-Bäume*

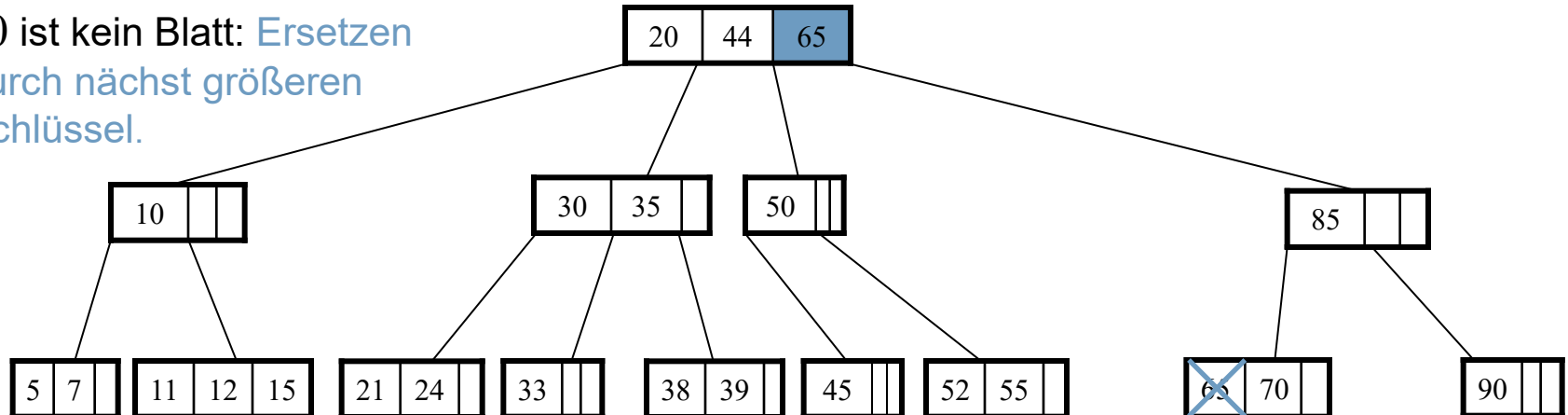
Algorithmen

Beispiel: `remove(60, root)`

1) Suchen 60.



2) 60 ist kein Blatt: Ersetzen
durch nächst größeren
Schlüssel.

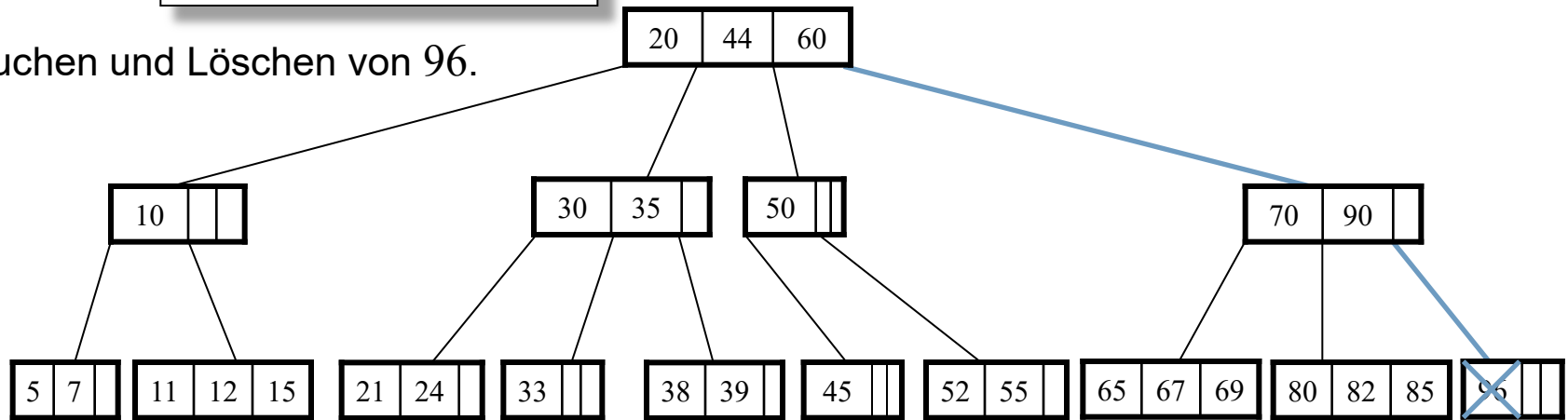


§4.7 B-Bäume*

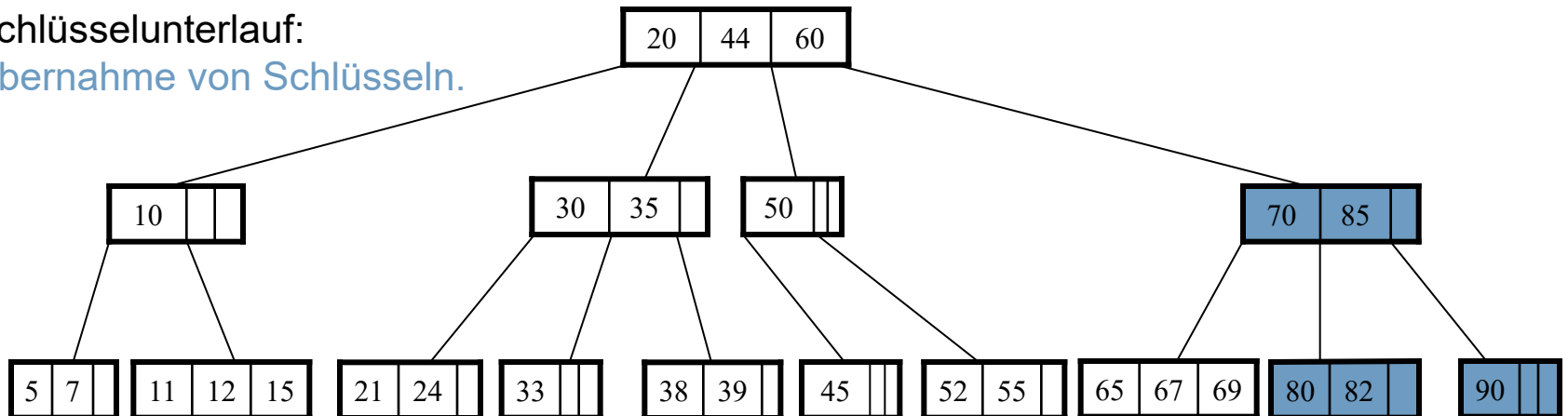
Algorithmen

Beispiel: `remove(96, root)`

1) Suchen und Löschen von 96.



2) Schlüsselunterlauf:
Übernahme von Schlüsseln.

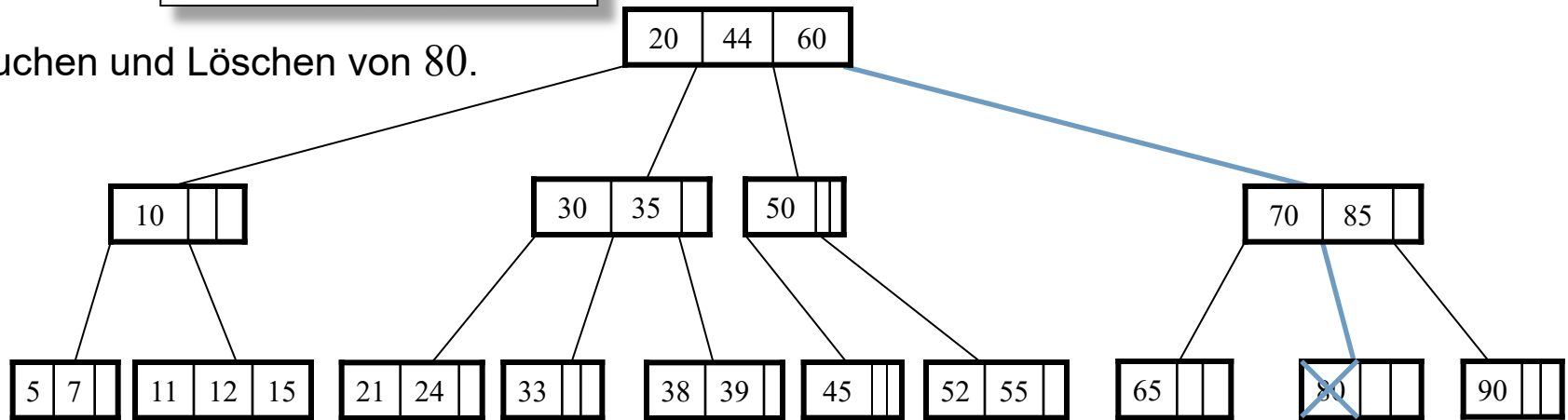


§4.7 B-Bäume*

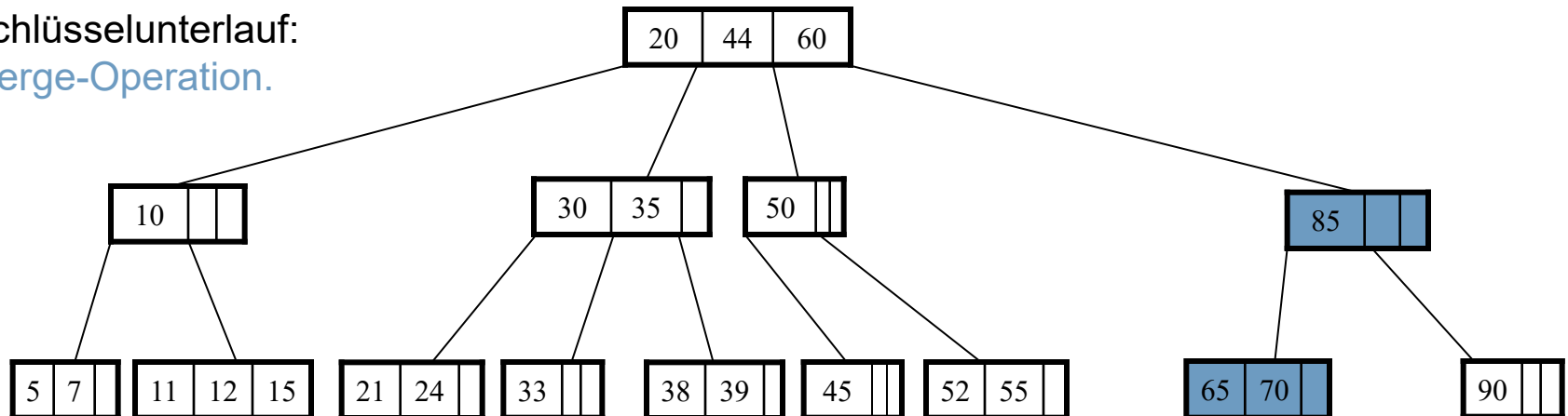
Algorithmen

Beispiel: `remove(80, root)`

1) Suchen und Löschen von 80.



2) Schlüsselunterlauf:
Merge-Operation.



Höhe eines B-Baums mit n Schlüsseln

- Im schlechtesten Fall hat die Wurzel zwei Kinder und jeder andere innere Knoten $\lceil m/2 \rceil$ Kinder.
- ▶ Damit ergibt sich eine maximale Höhe von [Ottmann/Widmayer]:

$$h \leq 1 + \log_{\lceil \frac{m}{2} \rceil} \left(\frac{n+1}{2} \right).$$

- ▶ Laufzeit für Suchen, Einfügen und Löschen:

$$O(\log_{\lceil \frac{m}{2} \rceil} (n)).$$

Speicherplatzausnutzung

- Implementierung eines B-Baums: Jeden Knoten hat ein statisches Feld der Größe m für die Schlüssel und Zeiger auf die Kinder.
 - ▶ Wie groß ist die Speicherplatzausnutzung, wenn jeder Knoten (außer der Wurzel) zwischen $\lceil m/2 \rceil - 1$ und $m - 1$ viele Schlüssel enthält?
 - ▶ Wenn eine zufällig gewählte Folge von n Schlüsseln in einen anfangs leeren B-Baum der Ordnung m eingefügt werden, ist eine Speicherplatzausnutzung zu erwarten von: $\ln(2) \approx 69 \%$.
 - Wird eine ab- oder aufsteigend sortierte Folge in einen leeren B-Baum eingefügt, ergibt sich eine besonders schlechte Speicherplatzausnutzung von ca. 50 %.
 - ▶ Etwas verbessern lässt sich dies, indem nicht bei jedem Knotenüberlauf eine Split-Operation durchgeführt wird, sondern erst geprüft wird, ob Schlüssel an Geschwisterknoten abgegeben werden können.

Externe Suchverfahren

- Anwendungen von B-Bäume: externe Suchverfahren, z.B. Datenbanken.
 - Die Menge der Datensätze ist so groß, dass sie nur auf dem Hintergrundspeicher (z.B. Festplatte) gespeichert werden kann.
 - Die Knotengröße m wird so gewählt, dass mit einem Festplattenzugriff die gesamten Daten eines Knotens (d.h. Schlüssel und Zeiger; Indexseite) gelesen werden können.
 - Die Datensätze selbst sind auf der Festplatte gespeichert.

Beispiel: Für $m = 256$ und $n = 10^9$ Datensätze ergibt sich ein B-Baum der Höhe ca. 4.

- Speicherung der beiden obersten Ebenen des B-Baums (d.h. Wurzel und seine Kinder) im Hauptspeicher (d.h. $256 + 256 \cdot 256 \approx 65.000$ Schlüssel), dann braucht eine Suchoperation im worst case drei Plattenzugriffe:
zwei Knotentabellen lesen und den Datensatz selbst lesen.

Weitere Baum-Typen:

■ Balancierte Bäume:

- Bruder-Bäume, Treaps, Tries, Splay-Bäume, 2-3-Bäume, Schwarz-Rot-Bäume, B*-Bäume, B⁺-Bäume, Patricia-Bäume, Fibonacci-Heaps, etc.

■ Verwaltung geometrischer Daten:

- Quadrees (2d), Octrees (3d), BSP-Trees, *kd*-Bäume, Range-Trees, etc.

Am Ende dieser Lerneinheit sollten Sie ...

- ... die Definition eines (binären) Suchbaumes und seiner Bestandteile kennen,
- ... die Definition eines balancierten Baumes kennen,
- ... die Eigenschaften und Algorithmen für AVL-Bäume kennen,
- ... die Eigenschaften und Algorithmen für B-Bäume kennen,
- ... die Eigenschaften und Algorithmen für Heaps kennen,
- ... Heapsort implementieren und analysieren können.

Web-Ressourcen zum Nachschlagen:

- <http://fbim.fh-regensburg.de/~saj39122/bruhi/index.html>
- <http://slady.net/java/bt/>